목차

- Windows 앱 만들기
  - 데스크톱 앱 만들기
    - 개요
      - 앱 플랫폼 선택하기
      - 세팅하기
        - 장치에서 개발할 수 있도록 하기
        - 개발자 모드 기능 및 디버깅
        - 개발자 계정 생성
    - Windows 앱을 위한 Visual Studio 템플릿
    - 데스크 톱 앱 현대화하기
      - 개요
      - Windows 런타임 API 호출
        - 개요
        - 가용 Windows 런타임 API
      - WinRT XAML controls 호스팅(XAML Islands)
        - 개요
        - WPF 와 Windows Forms apps 에서 XAML Islands
          - 표준 WinRT XAML control 호스트
          - 커스텀 WinRT XAML control 호스트
        - C++ Win32 앱에서 XAML Islands
          - WinRT XAML hosting API 사용
          - 표준 WinRT XAML control 호스트
          - 커스텀 WinRT XAML control 호스트
          - 응용 시나리오
      - Visual layer (Windows.UI.Composition) 사용하기
        - 개요
        - WPF
        - Windows Forms
        - Win32
      - 패키지 ID 가 필요한 기능
        - 개요
        - 패키지 확장자로 통합
        - 파일 탐색기로 패키지된 데스크톱 앱 통합

            - 패키지된 앱에 대한 활성화 정보 가져오기
            - UWP 구성 요소로 확장
            - 패키지된 앱 배포
        - 튜토리얼: WPF 앱 현대화하기
            - 개요
            - Contoso Expenses 앱을 .NET Core 3 로 마이그레이션
            - XAML Islands 를 이용하여 UWP 잉크캔버스 컨트롤 추가
            - XAML Islands 를 이용하여 UWP 캘린더뷰 컨트롤 추가
            - Windows 10 사용자 활동과 알림 추가
            - MSIX 로 패키지/배포
        - 패키징되지 않은 데스크톱 앱에 ID 부여
    - Windows UI 라이브러리(WinUI)
        - 개요
        - Windows UI Library 3 Preview 4
            - 개요 및 릴리스 노트
            - WinUI 3 으로 데스크톱 앱 만들기 시작하기
            - WinUI 3 으로 UWP 앱 만들기 시작하기
            - C++/WinRT 를 사용하는 WinUI 3 앱에 대한 XAML 컨트롤 템플릿
            - C#를 사용하는 WinUI 3 앱에 대한 XAML 컨트롤 템플릿
        - Windows UI 2.x 라이브러리
            - 개요
            - 셋업
                - WinUI 2.x 시작해 보기
                - NuGet 패키지 목록
            - 릴리스 노트
                - 개요
                - WinUI 2.5
                - WinUI 2.4
                - WinUI 2.3
                - WinUI 2.2
                - WinUI 2.1
                - WinUI 2.0
        - WinUI API 레퍼런스
    - Project Reunion 으로 Windows 앱 제작(사전 릴리스)

- 개요
- MRT Core
- DWriteCore

- Windows 10X 개발
  - Windows 10X 개발 FAQ
- Fluent Design System
- 접근성
- 스피치
- 기능 및 기술
- 최근 문서 업데이트
- Microsoft Build 2020 관련 문서

Windows PC 데스크톱 앱 개발하기

2/18/2021

이 자료에서는 Windows 용 데스크톱 앱을 만들거나 Windows 10 의 최신 환경을 적용하기 위해 기존 데스크톱 앱을 업데이트하는 데 필요한 정보를 제공합니다.

데스크톱 앱 플랫폼

Windows PC 용 데스크톱 앱을 구축하는 데는 네 가지 주요 플랫폼이 있습니다. 각 플랫폼은 앱의 라이프사이클을 정의하는 앱 모델, Word, Excel, Photoshop 과 같은 데스크톱 앱을 생성하고 Windows 기능을 사용하기 위해 관리되는 API 또는 기본 API 에 액세스할 수 있는 UI 프레임워크 및 UI 컨트롤 세트를 제공합니다.

추가적인 자료와 함께 세세한 비교를 보려면, 앱 플랫폼 선택하기 부분을 보십시오.

플랫폼	설명	문서 등 자료
Universal Windows Platform (UWP)	Windows 10 앱 및 게임을 위한 최첨단 플랫폼. UWP 컨트롤과 API 를 독점적으로 사용하는 UWP 앱을 구축하거나, 다른 플랫폼 중 하나를 사용하여 구축된 데스크톱 앱에서 UWP 컨트롤과 API 를 사용할 수 있습니다.	시작하기 API 레퍼런스 샘플
C++/Win32	Windows 및 하드웨어에 직접 액세스해야 하는 기본 Windows 앱의 플랫폼입니다.	시작하기 API 레퍼런스 샘플
WPF	XAML UI 모델을 사용하여 그래픽 기반으로 관리되는 Windows 애플리케이션을 위한 확립된 .NET 기반 플랫폼. .NET Core 3 또는 모든 .NET Framework 를 대상으로 할 수 있습니다.	시작하기 API 레퍼런스(.NET) 샘플
Windows Forms	경량 UI 모델로 관리되는 LOB(Line-of-Business) 애플리케이션용으로 설계된 .NET 기반 플랫폼. .NET Core 3 또는 모든 .NET Framework 를 대상으로 할 수 있습니다.	시작하기 API 레퍼런스(.NET)

향후 로드맵

앞으로 우리는 Windows UI Library(WinUI) 및 Project Reunion 을 통해 Windows 앱 개발 플랫폼을 발전시키고 있습니다.

- **WinUI** 는 Windows 10 앱용 기본 UX(사용자 환경) 프레임워크입니다. WinUI 는 Windows 10 의 하위 버전을 대상으로 하는 UWP 앱에 대한 WinRT 컨트롤의 새로운 버전과 업데이트된 버전을 제공하는 툴킷으로 시작했습니다. 현재 WinUI 3(현재 프리뷰)인 WinUI 는 UWP, .NET 및 Win32 앱 플랫폼 전반에 걸쳐 Windows 10 앱을 위한 UI(기본 사용자 인터페이스) 프레임워크로 성장하고 있습니다. 추가적인 정보는 Windows UI 라이브러리 (WinUI) 자료를 보십시오.
- **Project Reunion**(현재 프리뷰)은 Windows 앱 개발 플랫폼의 다음 진화를 나타내는 광범위한 새로운 개발자 구성 요소 및 도구의 태명입니다. Project Reunion 은 광범위한 대상 윈도우즈 10 OS 버전의 모든 앱에서 일관된 방식으로 사용할 수 있는 통합 API 및 툴 세트를 제공합니다. Project Reunion 은 UWP 및 네이티브 Win32, dotNET 과 같은 기존 Windows 앱 플랫폼과 프레임워크를 개발자들이 이러한 플랫폼 전반에 걸쳐 의존할 수 있는 공통 API 및 툴셋으로 보완합니다. 추가적인 정보는 Project Reunion 자료를 보십시오.

기존 데스크톱 앱을 Windows 10 용으로 업데이트

기존 WPF, Windows Forms 또는 기본 Win32 데스크톱 앱이 있는 경우 Windows 10 및 UWP(Universal Windows Platform)는 앱에서 최신 환경을 제공하는 데 사용할 수 있는 많은 기능을 제공합니다. 이러한 기능 중 대부분은 다른 플랫폼을 위해 앱을 다시 작성할 필요 없이 자신만의 속도로 앱에서 채택할 수 있는 모듈형 구성 요소로 제공됩니다.

기존 데스크톱 앱을 개선하는 데 사용할 수 있는 몇 가지 기능:

- MSIX 를 사용하여 데스크톱 앱을 패키징 및 배포합니다. MSIX 는 모든 Windows 앱에 범용 패키지 환경을 제공하는 최신 Windows 앱 패키지 형식입니다. MSIX 는 MSI, .appx, App-V 및 ClickOnce 설치 기술의 가장 좋은 측면을 결합하며, 안전하고 신뢰할 수 있도록 구축되었습니다.
- **패키지 확장**을 사용하여 데스크톱 앱을 Windows 10 과 통합할 수 있습니다. 예를 들어, 앱의 타일을 시작하거나, 앱을 공유 대상으로 만들거나, 앱에서 토스트 알림을 보냅니다.

  XAML Islands 를 사용하여 UWP XAML 컨트롤을 데스크톱 앱에서 호스팅합니다. 대부분의 최신 Windows 10 UI 기능은 UWP XAML 컨트롤에서만 사용할 수 있습니다.

추가적인 정보는 다음 자료를 보십시오.

자료	설명
데스크톱 앱 현대화하기	WPF, Windows Forms 및 C++ Win32 앱을 포함한 모든 데스크톱 앱에서 사용할 수 있는 최신 Windows 10 및 UWP 개발 기능에 대해 설명합니다.
튜토리얼: WPF 앱 현대화하기	UWP 잉크 및 캘린더 컨트롤을 앱에 추가하고 MSIX 패키지에 패키징하여 기존 WPF LOB(Line-of-Business) 샘플 앱을 현대화하려면 단계별 지침을 따르십시오.

새 데스크톱 앱 만들기

새 Windows 앱을 만들고자 한다면 다음 자료가 시작하는 데 도움이 될 수 있습니다.

자료	설명
앱 플랫폼 선택하기	주요 데스크톱 앱 플랫폼을 심층적으로 비교하고 사용자의 요구에 적합한 플랫폼을 선택하는 데 도움이 됩니다. 이 문서에서는 각 플랫폼에 대한 문서에 대한 유용한 링크도 제공합니다.
Windows 앱을 위한 Visual Studio 프로젝트 템플릿	Visual Studio 에서 C# 또는 C++를 사용하여 Windows 10 장치용 앱을 만드는 데 도움이 되는 프로젝트 및 항목 템플릿을 설명합니다.
데스크톱 앱 현대화하기	WPF, Windows Forms 및 C++ Win32 앱을 포함한 모든 데스크톱 앱에서 사용할 수 있는 최신 Windows 10 및 UWP 개발 기능에 대해 설명합니다.
기능 및 기술	각 기본 데스크톱 앱 플랫폼과 관련 문서에 대한 링크를 통해 액세스할 수 있는 Windows 기능에 대한 개요를 제공합니다.

관련 문서 및 기술

자료	설명
.NET Core 3.1	WPF/Windows Forms 앱을 위한 개선을 비롯한 .NET Core 3.1 의 최신 기능에 대하여 설명합니다.
.NET 5	이 자료에는 3.1 의 .NET Core 다음 릴리스인 .NET 5 에 포함된 내용이 자세히 나와 있습니다.
Desktop guide for WPF and .NET Core	모든 .NET 프레임워크가 아닌 .NET Core 대상 WPF 앱 개발
Azure	Azure 클라우드 서비스를 통해 애플리케이션의 범위를 확장하십시오.
Visual Studio	앱 및 서비스를 개발하기 위해 Visual Studio 사용법을 배우십시오.
MSIX	모든 Windows 앱을 최신 범용 패키지 형식으로 패키징 및 배포합니다.
Windows AI	Windows AI 를 사용하여 앱의 복잡한 문제에 대한 지능형 솔루션을 구축합니다.
Windows Containers	완전히 격리된 빠른 윈도우즈 환경에서 애플리케이션을 종속성으로 패키지화하십시오.
Progressive Web Apps	웹 앱을 Windows 10 에서 UWP 앱으로 배포하고 실행할 수 있는 진행형 웹 앱으로 변환합니다.
Xamarin	.NET 코드 및 플랫폼별 사용자 인터페이스를 사용하여 Windows, Android, iOS 및 macOS 용 크로스 플랫폼 앱을 구축합니다.
Docs archive for Windows 8.x and earlier	Windows 8.x 및 이전 버전의 앱 작성에 대한 보관된 문서에 액세스합니다.

Windows 앱 플랫폼 선택하기

2/18/2021

Windows PC 용 새 데스크톱 응용 프로그램을 만들려는 경우 가장 먼저 사용할 응용 프로그램 플랫폼을 결정합니다. Windows(윈도우)는 4 가지 주요 애플리케이션 플랫폼을 제공하며 각각 다른 강점을 가지고 있습니다.

- Universal Windows Platform (UWP): 이 플랫폼은 Windows 10 을 실행하는 모든 장치에 공통 유형 시스템, API 및 애플리케이션 모델을 제공합니다. 네이티브 또는 관리 가능 프로그램이 있습니다.
- WPF and Windows Forms: 이러한 .NET 기반 플랫폼은 중앙 관리 애플리케이션을 위한 공통 유형 시스템, API 및 애플리케이션 모델을 제공합니다.
- Win32: 이 플랫폼은 Windows 및 하드웨어에 직접 액세스해야 하는 기본 C/C++ Windows 애플리케이션을 위한 원래 플랫폼입니다. 따라서 Win32 API 는 최고 수준의 성능과 시스템 하드웨어에 직접 액세스해야 하는 애플리케이션을 위한 선택 플랫폼으로 자리 잡게 됩니다.

각 플랫폼에는 완벽한 UI 프레임워크와 Word, Excel 및 Photoshop 과 같은 데스크톱 앱을 생성하여 기존 Windows 데스크톱에서 실행되며 해당 환경의 특정 기능을 최대한 활용할 수 있는 UI 컨트롤 세트가 포함되어 있습니다. Windows 10 에서는 각 플랫폼에서도 Windows UI 라이브러리를 사용하여 사용자 인터페이스를 만들 수 있습니다.

이러한 플랫폼 중 일부는 일부 특성을 공유하며 특정 유형의 애플리케이션에 더 적합합니다. 예를 들어 UWP 와 .NET 은 Visual Studio 와 긴밀하게 통합됩니다. 이는 특히 개발자 생산성, 정교하고 사용자 정의가 가능한 UI 및 애플리케이션 보안 분야에서 많은 이점을 제공합니다. 이러한 프레임워크는 UI 를 빠르게 생성하는 시각 디자이너와 UI 마크업을 지원하기 때문에 특히 LOB(Line-of-Business) 애플리케이션에 적합합니다.

> **NOTE**
> 어떤 앱 플랫폼을 선택하든 많은 Windows 10 기능을 사용하여 앱에서 현대적인 경험을 제공할 수 있습니다. 예를 들어 WPF, Windows Forms 또는 Win32 API 를 사용하여 데스크톱 앱을 빌드한 경우에도 MSIX 패키지 배포를 계속 사용할 수 있습니다. 데스크톱 앱을 현대화하는 모든 방법에 대한 자세한 내용은 데스크톱 앱 현대화를 참조하십시오.

UWP

UWP 는 Windows 10 응용 프로그램과 게임을 위한 최첨단 플랫폼입니다. XAML 마크업을 사용하여 UI 와 코드(비즈니스 논리)를 구분하는 사용자 지정이 가능한 플랫폼입니다. UWP 는 정교한 UI, 스타일 사용자 지정 및 그래픽 집약적인 시나리오가 필요한 데스크톱 애플리케이션에 적합합니다. 또한 UWP 는 기본 UX 환경을 위한 Fluent Design System 을 기본적으로 지원하며 Windows Runtime(WinRT) API 에 대한 액세스를 제공합니다. Fluent 를 채택함으로써 UWP 는 잉크, 터치, 게임패드, 키보드, 마우스와 같은 일반적인 입력 방식을 자동으로 지원합니다.

UWP 를 사용하여 Windows PC 용 데스크톱 응용 프로그램을 만들 수 있을 뿐만 아니라 Xbox, HoloLens 및 Surface Hub 응용 프로그램에서 유일하게 지원되는 플랫폼입니다. UWP 는 최신 첨단 애플리케이션 플랫폼입니다.

추가적인 정보는 다음 자료들에 있습니다.

- 시작하기
- 프로젝트 템플릿
- 디자인과 UI
- 기술 및 기능
- API 레퍼런스
- 샘플

WPF

WPF 는 .NET Core 또는 full .NET Framework 에 액세스할 수 있는 관리되는 Windows 애플리케이션을 위한 플랫폼이며 XAML 마크업을 사용하여 UI 와 코드를 구분한다. 이 플랫폼은 정교한 UI, 스타일 사용자 지정 및 그래픽

집약적인 시나리오가 필요한 데스크톱 애플리케이션을 위해 설계되었습니다. WPF 개발 기술은 UWP 개발 기술과 유사하므로 WPF 에서 UWP 앱으로 마이그레이션하는 것이 Windows Forms 에서 마이그레이션하는 것보다 쉽습니다.

추가적인 정보는 다음 자료들에 있습니다.

- 시작하기(WPF)
- 프로젝트 템플릿
- 첫 앱 만들기 (.NET Core)
- 첫 앱 만들기 (.NET Framework)
- WPF 앱을 .NET Core 로 마이그레이션
- API 레퍼런스(.NET)
- 샘플

Windows Forms

Windows Forms 는 .NET Core 또는 full .NET 프레임워크에 대한 액세스와 가벼운 UI 모델이 있는 관리되는 Windows 응용 프로그램을 위한 원래 플랫폼입니다. 개발자가 플랫폼을 처음 사용하는 개발자에게도 애플리케이션 구축을 신속하게 시작할 수 있도록 지원하는 데 탁월합니다. 이 플랫폼은 시각적 및 비시각적 드래그 앤 드롭 제어 기능이 내장된 폼 기반 신속한 애플리케이션 개발 플랫폼입니다. Windows Forms 는 XAML 을 사용하지 않으므로 나중에 응용 프로그램을 UWP 로 확장하기로 결정하려면 UI 를 완전히 다시 써야 합니다.

추가적인 정보는 다음 자료들에 있습니다.

- Windows Forms 시작하기
- 프로젝트 템플릿
- 첫 Windows Forms 앱 만들기
- 튜토리얼: 사진 뷰어 만들기
- API 레퍼런스 (.NET)
- Windows Forms 앱 개선하기

Win32

Win32 API 를 C++와 함께 사용하면 WinRT 와 .NET 과 같은 관리 런타임 환경에서 가능한 것보다 관리되지 않은 코드로 대상 플랫폼을 더 많이 제어함으로써 최고 수준의 성능과 효율성을 달성할 수 있습니다. 그러나 애플리케이션 실행을 이러한 수준으로 제어하려면 올바르게 하려면 더 많은 주의를 기울여야 하며, 개발 생산성이 대체로 낮습니다.

다음은 고성능 애플리케이션을 구축할 수 있도록 Win32 API 및 C++가 제공하는 기능에 대한 몇 가지 강조 사항입니다.

- 리소스 할당, 개체 수명, 데이터 레이아웃, 정렬, 바이트 패킹 등에 대한 엄격한 제어를 포함한 하드웨어 수준 최적화
- 내장 기능을 통해 SSE 및 AVX 와 같은 성능 지향 명령 집합에 액세스
- 템플릿을 이용한 효율적이고 안전한 제네릭 프로그래밍
- 효율적이고 안전한 컨테이너/알고리즘
- DirectX, 특히 Direct3D 및 DirectCompute(UWP 는 DirectX interop 도 제공.)

추가적인 정보는 다음 자료들에 있습니다.For more information, see the following articles:

- 시작하기
- 프로젝트 템플릿
- 첫 Win32/C++ 앱 만들기
- 기술 및 기능
- API 레퍼런스
- 샘플

플랫폼 비교: UWP, WPF, and Windows Forms

다음 표는 위 플랫폼에 대한 더 자세한 비교를 보여줍니다.

기능/시나리오	UWP	WPF	WINDOWS FORMS
지원 버전	Windows 10	Windows 7 이후	Windows 7 이후
언어	C#, C++/WinRT, C++/CX, VB, JavaScript	C#, C++/CLI (C++ 확장), F#, VB	C#, C++/CLI (C++ 확장), F#, VB
UI 런타임	Native (C++/WinRT and C++/CX) and managed (.NET Native)	Managed (.NET Framework and .NET Core 3)	Managed (.NET Framework and .NET Core 3)
오픈 소스	Yes (Windows UI Library only)	Yes (.NET Core only)	Yes (.NET Core only)
XAML 지원	Yes	Yes	No
강점	- XAML markup for UI - 풍부한 커스텀 UX - 기존 코드 기반이 .NET 표준과 호환 - 고 DPI 지원 - Windows 장치 간 다중 입력 유형 지원(터치, 펜, 게임패드, 마우스, 키보드 등) - Xbox, HoloLens, IoT or Surface Hub 지원 - 선택적 밀집 UI - 네이티브 C++ 지원 - 배터리 수명 최적화 - 최신 접근성 지원(읽어주기 등) - 많은 유용한 텍스트 데이터 (빌트인 스펠체크 등) - 잉크 그림 지원 - 애플리케이션 컨테이너를 통한 안전한 실행(ex: 신뢰할 수 없는 컨텐츠는 안전모드)	- XAML markup for UI - 풍부한 커스텀 UX - 마이크로소프트 및 파트너의 대규모 제어 기능 모음 - 밀집 UI - Windows 7 지원 - 입력 확인을 위한 플랫폼 지원	- 빠른 개발 - UI 개발을 위한 WYSIWYG 에디터 - 마이크로소프트 및 파트너의 대규모 제어 기능 모음 - 밀집 UI - Windows 7 지원 - 키보드/마우스 입력

지원이 제한된 시나리오	- 다중 창 지원 [1] - 입력 확인을 위한 플랫폼 지원 [1] - Windows 7 미지원 - 몇몇 Windows 런타임 API 가 Windows 10 의 특정 버전만 지원함 - 플랫폼 완전 지원과 셸 통합(예를 들어 UWP 는 현재 시스템 트레이 통합 또는 모든 장치에 대한 전체 액세스를 지원하지 않음) - 디스크의 모든 파일 직접 접근 - ADO.NET - 비.NET 표준 또는 비 Windows App 인증서 Kit 호환 API 를 사용하는 기존 코드베이스 클래스 라이브러리 - 지역 네트워크 루프백 지원(즉, 대상 디바이스에 루프백 예외를 만들지 않고 앱이 localhost 와 통신해야 하는 경우) - 강력한 파일 입출력	- 고 DPI 지원 [2] - 터치 입력 [2]	- 그래픽/UX(터치/애니메이션 등) - 고 DPI 지원 [2] - 터치 입력 [2] - 뷰/데이터 모델의 추상화 수 - 커스텀 가능 UI

[1] 향후 Windows 10 릴리스에서 이 시나리오를 다룰 기능을 공개적으로 발표했습니다.

[2] 플랫폼에는 이 시나리오에 대한 1 급 API 지원이 부족하지만 개발자는 해결 방법을 통해 이 시나리오를 지원할 수 있습니다.

Windows UI 라이브러리를 Windows 앱에 사용하기

기본 Windows 앱 플랫폼을 보완하기 위해 앱에서 Windows UI 라이브러리를 사용할 수도 있습니다. WinUI 는 Windows 10 의 하위 버전을 대상으로 하는 UWP 앱에 대한 WinRT 컨트롤의 새로운 버전과 업데이트된 버전을 제공하는 툴킷으로 시작했습니다. 현재 윈 UI 3(프리뷰) WinUI 는 UWP, .NET 및 Win32 앱 플랫폼 전반에 걸쳐 Windows 10 앱을 위한 UI(기본 사용자 인터페이스) 프레임워크로 성장하고 있습니다. 다음과 같은 방법으로 사용할 수 있습니다.

- WinUI 2.x:
  - UWP 앱은 Windows SDK 에서 제공하는 WinRT 컨트롤 대신 WinUI 2.x 컨트롤을 사용할 수 있습니다. 이 WinUI 릴리스에는 Windows SDK 의 새 컨트롤과 기존 컨트롤의 업데이트된 버전이 모두 포함되어 있습니다.
  - XAML Islands 를 사용하여 WinUI 2.x 컨트롤을 호스팅하도록 기존 WPF, Windows Forms 및 C++/Win32 앱을 업데이트할 수 있습니다.
- WinUI 3 (프리뷰):

WinUI 3 부터는 완전히 WinUI 기반 UI 를 사용하는 dotNET 및 C++/Win32 앱과 UWP 앱을 생성할 수 있습니다. 이 릴리스에는 이러한 앱을 만드는 데 필요한 모든 것을 제공하는 Visual Studio 프로젝트 템플릿이 포함되어 있습니다.

> **NOTE**
> 3 판은 아직 프리뷰이며, 현재 생산성 앱에 사용할 수 없습니다.

다른 앱 플랫폼

Progressive Web Apps (PWAs)

개발자들은 PWA 를 통해 웹 사이트 코드를 패키징하여 Windows 10 PC 에 애플리케이션처럼 설치 및 실행할 수 있습니다. 자세한 내용은 여기를 참조하십시오.

Xamarin

Xamarin 을 사용하여 iOS 및 Android 에서도 실행할 수 있는 Windows 10 용 크로스 플랫폼 애플리케이션을 구축할 수 있습니다. 자세한 내용은 여기를 참조하십시오.

Uno Platform

Uno 플랫폼은 Windows UWP 기반 코드(C# 및 XAML)를 iOS, Android, MacOS, Linux 및 Web Assembly 에서 실행할 수 있게 합니다. 윈도우 10 2004(19041)의 UWP 에 대한 전체 API 정의와 UWP API 의 일부를 구현(Windows.UI.Xaml 등)하여 UWP 애플리케이션이 이러한 플랫폼에서 실행될 수 있도록 한다. 자세한 내용은 Uno 플랫폼 문서를 참조하십시오.

세팅하기

2/18/2021

Windows 10 개발에 입문하신 것을 환영합니다! 다음 안내를 따라 WinRT API(aka UWP)를 이용해 Windows 10 용 앱을 만들어 보세요.

Visual Studio 다운로드/업데이트

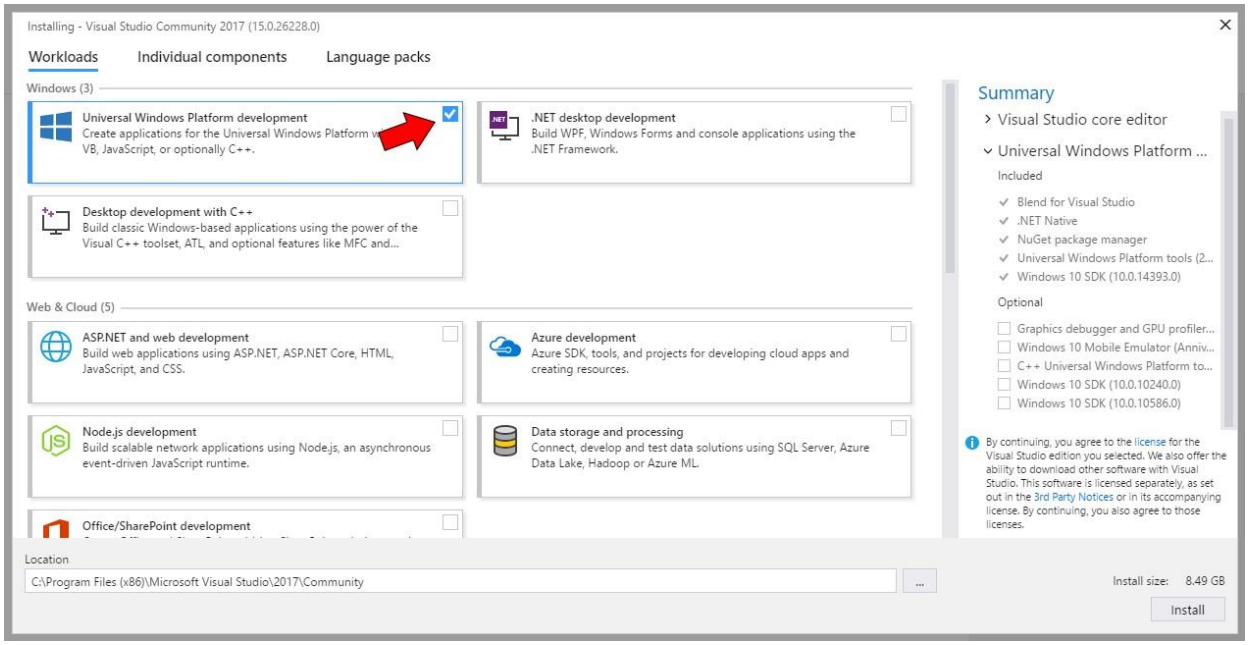
> **NOTE**
> 새 버전의 Visual Studio 는 다음 사진과는 다르게 생겼을 수 있습니다. 걱정하지 마세요! 그렇다 해도 다음 안내를 따르면 쉽게 설치할 수 있습니다.

Microsoft Visual Studio 2019 는 당신이 앱을 설계하고, 코드를 쓰고, 테스트하고, 디버깅하는 것을 도와줍니다.

지금 Visual Studio 2019 가 없다면, 무료인 Microsoft Visual Studio Community 2019 를 설치하실 수 있습니다. 이 다운로드에는 앱 테스트를 위한 장치 시뮬레이터가 포함됩니다.

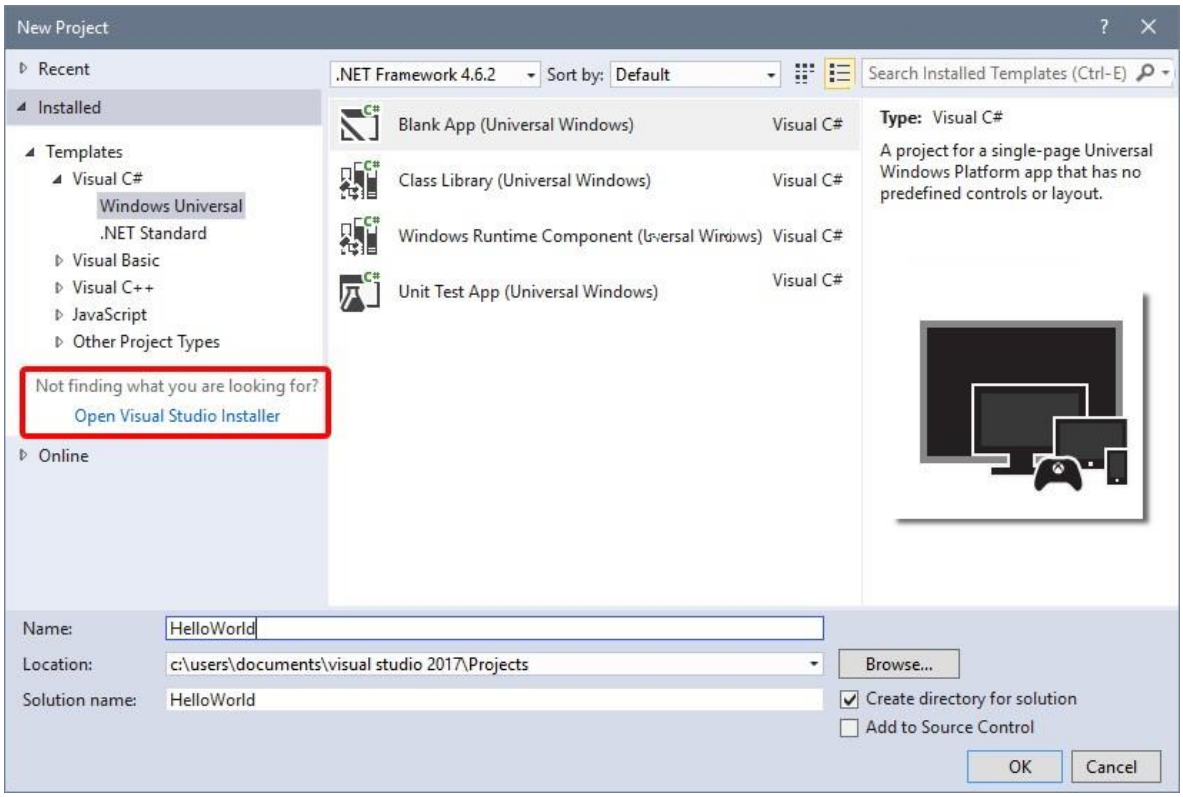
- Windows 10 개발자 도구 다운로드

설치할 때, 다음과 같이 '유니버설 Windows 플랫폼 개발'에 체크해 주세요.



비주얼 스튜디오 사용에 대한 안내는 여기를 참고해 주세요. 비주얼 스튜디오 시작하기

기존에 이미 비주얼 스튜디오를 사용하고 있었지만 구성 요소 몇 가지를 가지고 있지 않다면, '새 프로젝트' 대화 상자에서 위와 같은 설치 프로그램을 다시 열 수 있습니다.



장치에서 개발할 수 있도록 하기

실제 PC 나 스마트폰에서 UWP 앱을 테스트하는 것은 중요합니다. 만든 앱을 배포하기 전에 개발을 가능하게 해야 합니다.

- 자세한 설명은 장치에서 개발할 수 있도록 하기 부분을 보십시오.

앱 개발자로 등록하기

지금 개발을 시작할 수도 있지만, 스토어에 게시하려면 개발자 계정이 필요합니다.

- 개발자 계정을 만들려면 가입 페이지로 이동해 주세요.

다음은?

필요한 도구와 개발 옵션을 활성화하였다면, 그 다음은 시작에 대한 안내를 보십시오.

- 앱 플랫폼 선택하기
- 코딩 시작하기

Windows 개발 셋업 스크립트

당신의 디바이스에서 다른 기능이나 패키지를 설치, 혹은 커스터마이징을 원하신다면 시스템 세팅 스크립트를 보세요.

- 개발자 셋업 스크립트

더 많은 도구를 알아보고 싶으세요?

모든 도구 및 그 다운로드는 이곳에서 찾을 수 있습니다.

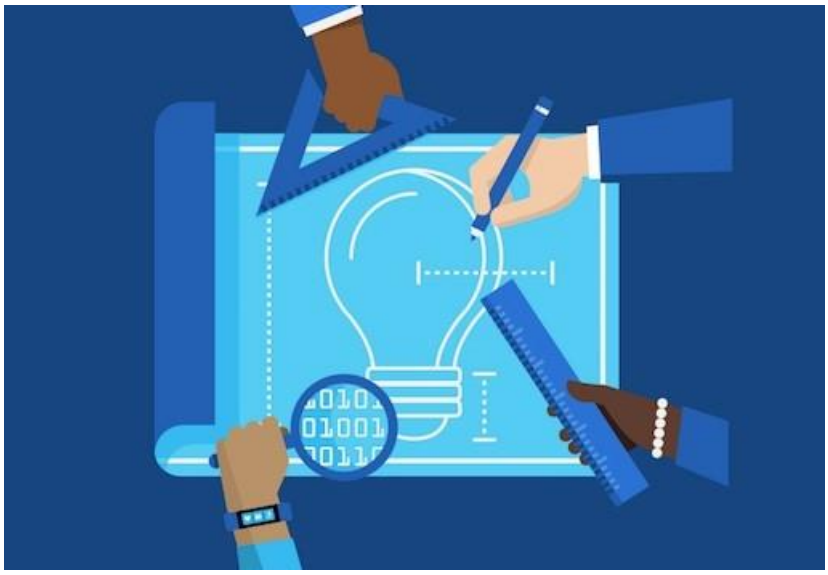
같이 보기

- UWP 개발자를 위한 코드 샘플
- UWP 앱이란?
- Windows 가입하기

장치에서 개발할 수 있도록 하기

11/2/2020

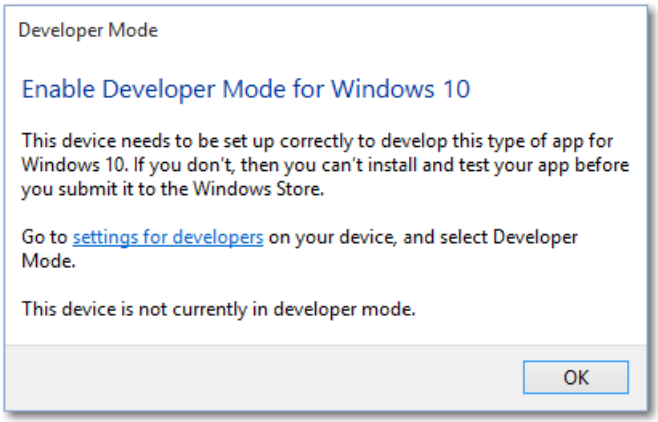
개발자 모드 활성화, 앱 사이드 로드하기, 다른 개발자 모드 사용하기



> **IMPORTANT** 당신의 PC 에서 앱을 만드는 것이 아니라면 개발자 모드를 활성화시킬 필요는 없습니다. 컴퓨터의 문제를 해결하려고 한다면, Windows 도움말을 확인해 보세요. 개발이 처음이라면 필요한 도구를 받아 보세요.

게임, 웹 검색, 전자 메일 또는 Office 앱과 같은 일상적인 작업에 컴퓨터를 사용하는 경우 개발자 모드를 활성화할 필요가 없으며 실제로 활성화해서는 안 됩니다. 이 페이지의 나머지 정보는 여러분에게 중요하지 않을 것이고, 여러분은 안전하게 여러분이 하던 일을 다시 할 수 있습니다. 이 한글화 문서를 보는 경우 그럴 리는 없겠지만..

그러나 컴퓨터에서 Visual Studio 를 사용하여 소프트웨어를 처음 작성하는 경우 개발 PC 와 코드를 테스트하는 데 사용할 장치에서 개발자 모드를 활성화해야 합니다. 개발자 모드가 활성화되지 않은 상태에서 UWP 프로젝트를 열면 개발자 설정 페이지가 열리거나 이 대화 상자가 Visual Studio 에 나타납니다.



여기에서는, settings for developers 를 눌러 개발자 설정 페이지를 열어 주세요.

> NOTE
>
> 언제든지 개발자용 페이지로 이동하여 개발자 모드를 활성화하거나 비활성화할 수 있습니다. 작업 표시줄의 Cortana 검색 상자에 "개발자 설정"을 입력하면 됩니다.

개발자 설정

개발자 모드를 활성화하거나 다른 설정을 하려면,

1. 개발자 설정 대화 상자에서, 필요한 수준의 접근 권한을 선택하세요.
2. 주의 사항을 읽고 나서, '예'를 클릭해 주세요.

> NOTE
>
> 개발자 모드를 활성화하려면 관리자 권한이 필요합니다. 장치가 조직 소유인 경우, 이 옵션을 사용할 수 없을 수 있습니다.

개발자 모드 기능

개발자 모드는 개발자 라이센스에 대한 윈도우즈 8.1 요구 사항을 대체합니다. 개발자 모드 설정은 사이드로드 외에도 디버깅 및 추가 배포 옵션을 활성화합니다. 여기에는 이 장치를 배포할 수 있는 SSH 서비스 시작이 포함됩니다. 이 서비스를 중지하려면 개발자 모드를 비활성화해야 합니다.

데스크톱에서 개발자 모드를 활성화하면, 다음을 포함하는 패키지가 설치됩니다.

- Windows 장치 포털. 장치 포털 옵션이 켜진 경우에만 장치 포털이 활성화되며 방화벽 규칙이 구성됩니다.
- 앱 원격 설치를 허용하는 SSH 서비스에 대한 방화벽 규칙을 설치하고 구성합니다. 장치 검색을 활성화하면 SSH 서버가 켜집니다.

위 기능에 대한 자세한 설명이 보고 싶거나 설치 과정에 어려움이 발생한 경우 개발자 모드 기능 및 디버깅 쪽을 확인하십시오.

같이 보기

- Windows 가입하기
- 개발자 모드 기능 및 디버깅

개발자 모드 기능 및 디버깅

2/18/2021

앱에 개발자 모드를 설치하는 기본 사항에 관심이 있는 경우, 장치에서 개발할 수 있도록 하기에 설명된 지침을 따르십시오. 이 자료에서는 개발자 모드의 고급 기능, 이전 버전의 Windows 10 의 개발자 모드 및 개발자 모드 설치의 디버깅 실패를 다룹니다.

개발자 모드의 다른 기능

각 장치 제품군에 대해 추가 개발자 기능을 사용할 수 있습니다. 이러한 기능은 장치에서 개발자 모드를 사용하도록 설정한 경우에만 사용할 수 있으며 OS 버전에 따라 다를 수 있습니다.

다음은 Winodws 10 의 개발자 모드 기능입니다.

장치 포털

자세한 설명은 Windows 장치 포털 개요를 참고하세요.

장치별 설정 안내는 다음을 보시면 됩니다.

- 데스크톱 장치 포털
- HoloLens 장치 포털
- IoT 장치 포털
- 모바일 장치 포털
- Xbox 장치 포털

개발자 모드 또는 장치 포털을 사용하는 데 문제가 발생하면 알려진 문제 포럼을 참조하여 이러한 문제에 대한 해결 방법을 찾거나, 아래의 '개발자 모드 설치 실패' 부분을 보고 추가 세부 정보와 개발자 모드 패키지 차단 해제를 위해 어떤 WSUS KB 를 허용해야 하는지 알아보십시오.

앱 사이드로드

> **IMPORTANT** 최신 Windows 10 업데이트에서는 사이드로드가 기본적으로 설정되어 있으므로 이 설정은 표시되지 않습니다. 이전 버전의 Windows 10 인 경우 기본 설정에서는 Microsoft Store 에서만 앱을 실행할 수 있으며 Microsoft 가 아닌 소스에서 앱을 설치하려면 사이드 로딩을 사용하도록 설정해야 합니다.

앱 사이드로드 설정은 일반적으로 Microsoft Store 를 거치지 않고 관리되는 기기에 사용자 지정 앱을 설치해야 하는 회사나 학교 또는 Microsoft 가 아닌 다른 소스에서 앱을 실행해야 하는 모든 사용자가 사용합니다. 이 경우 조직은 설정 페이지의 이미지에 앞서 표시된 것처럼 UWP 앱 설정을 실행 중지하는 정책을 시행하는 것이 일반적입니다. 조직은 또한 앱의 사이드로드에 필요한 인증서와 설치 위치도 제공합니다. 자세한 내용은 TechNet 의 Windows 10 사이드로드 앱 및 Microsoft Intune 기본 문서 참조하십시오.

장치 계열에 따른 정보

- 데스크톱 계열: 패키지로 생성된 Windows PowerShell 스크립트("AddAppDevPackage.ps1")를 실행하여 애플리케이션 패키지(.appx) 및 애플리케이션 실행에 필요한 인증서를 설치할 수 있습니다. 자세한 내용은 UWP 앱 패키징을 참조하십시오.
- 모바일 계열: 필요한 인증서가 이미 설치되어 있는 경우 파일을 눌러 이메일 또는 SD 카드를 통해 사용자에게 전송된 .appx 를 설치할 수 있습니다.

사이드로드 앱은 신뢰 가능한 인증서가 없는 앱은 설치할 수 없으므로 개발자 모드보다 안전한 옵션입니다.

> **NOTE**
>
> 사이드로드를 하는 경우에도 신뢰할 수 있는 출처의 앱만 설치해야 합니다. Microsoft Store 에서 인증받지 않은 사이드로드 앱을 설치하는 경우, 당신은 앱의 사이드로드에 필요한 모든 권한을 얻었으며 앱 설치 및 실행으로 인한 모든 손상에 대해 전적으로 사용자에게 책임이 있다는 데 동의하는 것입니다. 개인 정보 처리 방침의 Windows -> Microsoft Store 섹션을 참조하세요.

SSH

장치에서 장치 검색 켜기를 설정하면 SSH 서비스가 활성화됩니다. 장치가 UWP 응용 프로그램의 원격 배포 대상인 경우에 사용됩니다. 서비스 이름은 'SSH 서버 브로커' 및 'SSH 서버 프록시'입니다.

> **NOTE**
>
> 이것은 Microsoft 의 OpenSSH 구현이 아닙니다.

SSH 서비스를 이용하기 위해 장치 검색을 활성화하여 핀 페어링을 허용할 수 있습니다. 다른 SSH 서비스를 실행하려면 다른 포트에 이 서비스를 설정하거나 개발자 모드 SSH 서비스를 해제할 수 있습니다. SSH 서비스를 끄려면 장치 검색을 끄십시오.

SSH 로그인은 "DevToolsUser" 계정을 통해 수행되며, 이 계정을 통해 인증에 대한 암호를 수락합니다. 이 암호는 장치 검색 "페어" 버튼을 누른 후 장치에 표시되는 PIN 이며 PIN 이 표시되는 동안에만 유효합니다. 또한 SFTP 하위 시스템은 Visual Studio 에서 느슨한 파일 배포가 설치된 DevelopmentFiles 폴더의 수동 관리를 위해 사용할 수 있습니다.

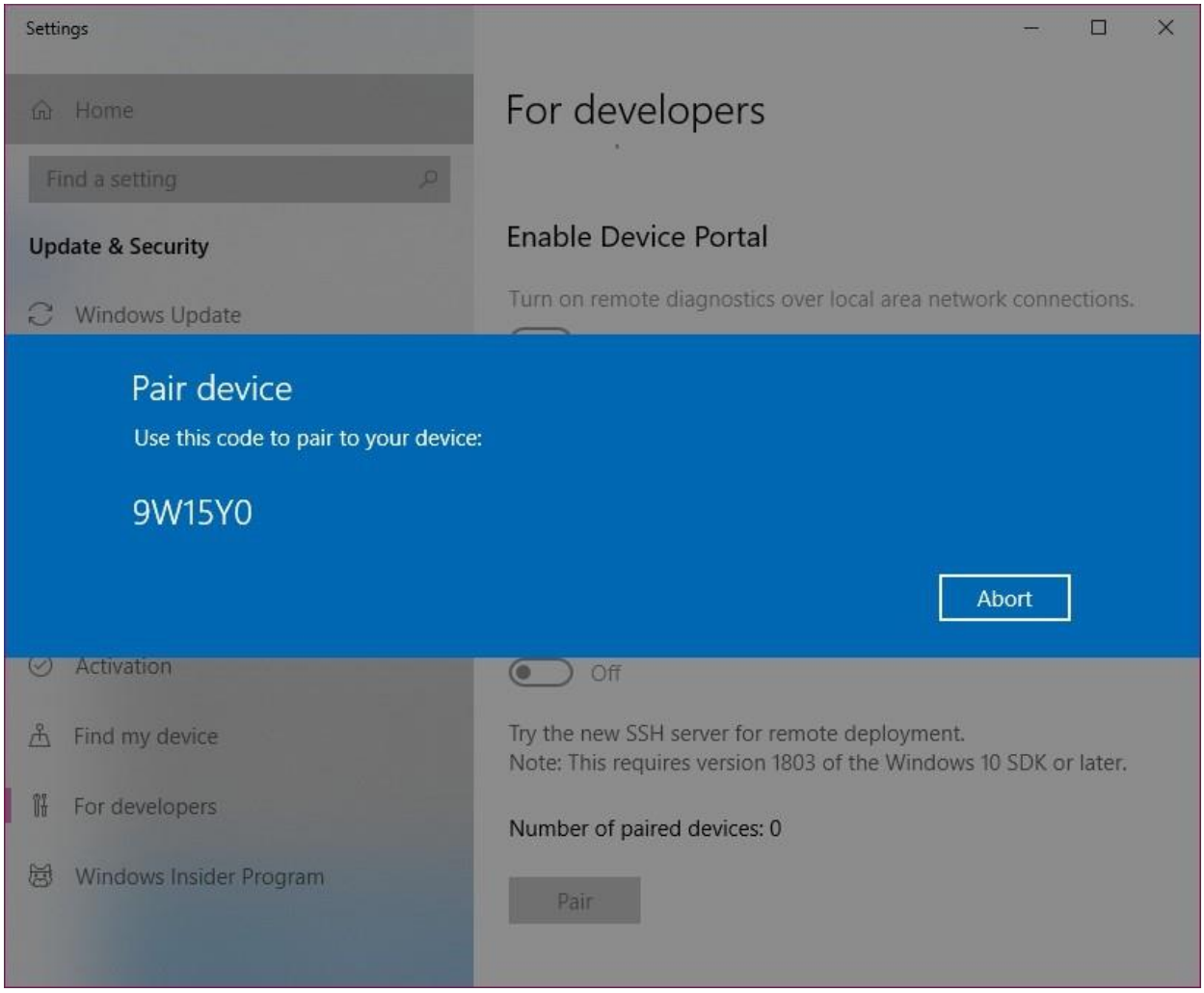
SSH 사용에 대한 주의 사항

Windows 에서 사용되는 기존 SSH 서버는 아직 프로토콜 규정을 준수하지 않으므로 SFTP 또는 SSH 클라이언트를 사용하려면 특별한 구성이 필요할 수 있습니다. 특히 SFTP 하위 시스템은 버전 3 이하에서 실행되므로 모든 연결 클라이언트가 이전 서버를 사용하도록 구성되어야 합니다. 이전 장치의 SSH 서버는 공개 키 인증에 ssh-ds 를 사용하며, OpenSSH 는 이를 더 이상 사용하지 않습니다. 이러한 장치에 연결하려면 SSH 클라이언트를 수동으로 구성하여 ssh-dss 를 수락해야 합니다.

장치 검색

장치 검색을 사용하도록 설정하면 mDNS 를 통해 네트워크의 다른 장치에 장치를 표시할 수 있습니다. 이 기능을 사용하면 장치 검색이 활성화되면 "페어" 버튼을 눌러 이 장치와 페어링하기 위한 SSH PIN 을 가져올 수도

있습니다. 장치를 대상으로 한 첫 번째 Visual Studio 배포를 완료하려면 이 PIN 프롬프트가 화면에 표시되어야 합니다.



장치를 배포 대상으로 만들려는 경우에만 장치 검색을 활성화해야 합니다. 예를 들어 장치 포털을 사용하여 테스트하기 위해 스마트폰에 앱을 배포하는 경우 개발 PC 가 아닌 스마트폰에서 장치 검색을 사용하도록 설정해야 합니다.

Windows 탐색기, 원격 데스크톱, PoerShell 을 위한 최적화(데스크톱)

데스크톱 장치 제품군에서 개발자 설정 페이지에는 개발 작업에 대해 PC 를 최적화하는 데 사용할 수 있는 설정에 대한 바로 가기가 있습니다. 각 설정에 대해 확인란을 선택하고 적용을 클릭하거나 설정 표시 링크를 클릭하여 해당 옵션에 대한 설정 페이지를 열 수 있습니다.

Notes

이전 버전의 Windows 10 Mobile 에서는 개발자 설정 메뉴에 크래시 덤프 옵션이 있었습니다. USB 를 통해서만 사용할 수 있는 것이 아니라 원격으로 사용할 수 있도록 장치 포털로 이동되었습니다.

Windows 10 PC 에서 Windows 10 장치로 앱을 배포하는 데 사용할 수 있는 도구는 여러 가지가 있습니다. 두 장치는 모두 유선 또는 무선 연결을 통해 네트워크의 동일한 서브넷에 연결되거나 USB 로 연결되어야 합니다. 나열된 두 가지 방법 모두 앱 패키지만 설치합니다.

- Windows 10 애플리케이션 배포(WinAppDeployCmd) 도구를 사용합니다. WinAppDeployCmd 도구에 대해 자세히 알아보십시오.
- 장치 포털을 사용하여 브라우저에서 Windows 10, 버전 1511 이상을 실행하는 모바일 장치로 배포할 수 있습니다. 장치 포털의 앱 페이지를 사용하여 앱 패키지(.appx)를 업로드하고 장치에 설치합니다.

개발자 모드 패키지 설치에 실패한 경우

네트워크 또는 관리 문제로 인해 개발자 모드가 제대로 설치되지 않는 경우가 있습니다. 개발자 모드 패키지는 브라우저의 장치 포털 또는 장치 검색을 사용하여 이 PC 에 원격으로 배포하려면 필요하지만 로컬 개발에는 배포되지 않습니다. 이러한 문제가 발생하더라도 Visual Studio 를 사용하거나 이 장치에서 다른 장치로 앱을 로컬로 배포할 수 있습니다.
이러한 문제에 대한 해결 방법 등을 알아보려면 알려진 이슈 포럼을 참조하세요.

> NOTE
> 개발자 모드가 올바르게 설치되지 않은 경우 피드백 요청을 제출하는 것이 좋습니다. 피드백 허브 앱에서 새 피드백 추가를 선택하고 개발자 플랫폼 범주 및 개발자 모드 하위 범주를 선택합니다. 피드백을 제출하면 Microsoft 에서 발생한 문제를 해결하는 데 도움이 됩니다.

패키지를 찾지 못함

"개발자 모드 패키지를 Windows Update 에서 찾을 수 없습니다. 에러 코드 0x80004005 자세히 보기" 이 오류는

네트워크 연결 문제, 엔터프라이즈 설정 또는 패키지가 누락되어 발생할 수 있습니다.

해결 방법:

1. 컴퓨터가 인터넷에 연결되어 있어야 합니다.
2. 도메인에 가입된 컴퓨터를 사용하는 경우 네트워크 관리자에게 문의하십시오. 모든 주문형 기능과 마찬가지로 개발자 모드 패키지는 WSUS 2.1 에서 기본적으로 차단됩니다. 현재 및 이전 릴리스에서 개발자 모드 패키지를 차단 해제하려면 WSUS 에서 다음 KB 를 허용해야 합니다: 4016509, 3180030, 3197985
3. 설정 > 업데이트 및 보안 > Windows Update 에서 Windows 업데이트를 확인합니다.
4. 설정 > 시스템 > 앱 & 기능 > 옵션 기능 관리 > 기능 추가에 Windows Developer Mode 패키지가 있는지 확인합니다. 이 패키지가 없으면 컴퓨터에 맞는 올바른 패키지를 찾을 수 없습니다.

위의 단계를 수행한 후 개발자 모드를 비활성화했다가 다시 활성화하여 수정 사항을 확인합니다.

패키지를 설치하지 못함

"개발자 모드 패키지 설치에 실패했습니다. 에러 코드 0x80004005 자세히 보기"

이 오류는 Windows 빌드와 개발자 모드 패키지 간의 비호환성으로 인해 발생할 수 있습니다.

해결 방법:

1. 설정 > 업데이트 및 보안 > Windows Update 에서 Windows 업데이트를 확인합니다.
2. 컴퓨터를 다시 부팅하여 모든 업데이트가 적용되었는지 확인합니다.

그룹 정책 또는 레지스트리 키를 사용하여 장치 사용

대부분의 개발자의 경우 설정 앱을 사용하여 장치에서 디버깅을 사용하도록 설정하려고 합니다. 자동 테스트와 같은 특정 시나리오에서는 Windows 10 데스크톱 장치를 개발할 수 있도록 다른 방법을 사용할 수 있습니다. 이러한 단계에서는 SSH 서버를 사용하거나 장치를 원격 배포 및 디버깅 대상으로 지정할 수 없습니다.

Windows 10 Home 이 없는 경우 gpedit.msc 를 사용하여 장치를 사용하도록 그룹 정책을 설정할 수 있습니다. Windows 10 Home 이 있는 경우 regedit 또는 PowerShell 명령을 사용하여 장치를 사용하도록 레지스트리 키를 직접 설정해야 합니다.

gpedit 을 이용하여 디바이스 활성화

1. Gpedit.msc 를 실행합니다.
2. 로컬 컴퓨터 정책 > 컴퓨터 구성 > 관리 템플릿 > Windows 구성 요소 > 앱 패키지 배포
3. 사이드 로딩을 활성화하려면 다음과 같이 정책을 수정합니다.

- 신뢰할 수 있는 모든 앱 설치 허용

  또는 개발자 모드를 활성화하려면 다음과 같이 정책을 수정합니다.

- 신뢰할 수 있는 모든 앱 설치 허용
- 통합 개발 환경(IDE)에서 UWP 앱을 개발하고 설치 허용

4. 장치를 재부팅합니다.

Regedit 을 이용하여 장치 활성화

1. regedit 을 실행합니다.

2. 사이드로딩을 활성화하려면 다음 DWORD 의 값을 1 로 설정합니다.
   HKLM₩SOFTWARE₩Microsoft₩Windows₩CurrentVersion₩AppModelUnlock₩AllowAllTruestedApps
   또는 개발자 모드를 활성화하려면 다음 DWORD 의 값을 1 로 설정합니다.

   HKLM₩SOFTWARE₩Microsoft₩Windows₩CurrentVersion₩AppModelUnlock₩AllowDevelopmentWithoutDevLicense

PowerShell 을 이용하여 장치 활성화

1. 개발자 권한으로 PowrShell 을 실행합니다.
2. 사이드로딩을 활성화하려면 다음 명령을 입력합니다.

```
PS C:₩WINDOWS₩system32> reg add
"HKEY_LOCAL_MACHINE₩SOFTWARE₩Microsoft₩Windows₩CurrentVersion₩AppModelUnlock" /t REG_DWORD /f /v
"AllowAllTrustedApps" /d "1"
```

또는 개발자 모드를 활성화하려면 다음 명령을 입력합니다.

```
PS C:₩WINDOWS₩system32> reg add
"HKEY_LOCAL_MACHINE₩SOFTWARE₩Microsoft₩Windows₩CurrentVersion₩AppModelUnlock" /t REG_DWORD /f /v "AllowDevelopmentWithoutDevLicense"
/d "1"
```

장치를 Windows 8.1 에서 Windows 10 으로 업그레이드하기

Windows 8.1 장치에서 앱을 만들거나 빼낼 때는 개발자 라이선스를 설치해야 합니다. 장치를 Windows 8.1 에서 Windows 10 으로 업그레이드해도 이 정보는 그대로 유지됩니다. 다음 명령을 실행하여 업그레이드된 Windows 10 장치에서 이 정보를 제거합니다. Windows 8.1 에서 Windows 10, 버전 1511 이상으로 직접 업그레이드하는 경우에는 이 단계가 필요하지 않습니다.

개발자 라이선스 등록 해제

1. 관리자 권한으로 PowerShell 을 실행합니다.
2. 다음 명령을 입력합니다.

Unregister-windowsdeveloperlicense

이 작업을 마친 후에는 이 항목에 설명된 대로 장치에서 계속 개발할 수 있도록 설정해야 합니다. 이렇게 하지 않으면 앱을 디버깅하거나 해당 앱에 대한 패키지를 만들 때 다음 오류를 비롯한 여러 오류가 발생할 수 있습니다.

에러 : DEP0700 : 앱 등록에 실패했습니다.

개발자 계정 생성

2/18/2021

지금 개발자 계정에 등록하면 Microsoft Store 로 앱을 가져오고 다른 Microsoft 프로그램에 참여할 수 있습니다.

지금 가입하세요!

개발자 계정 개설

Microsoft 는 개인용 계정과 회사용 계정을 제공합니다. 가입 과정의 개요를 확인하고 어떻게 되는 것인지 확인하세요.

앱 이름 넣기

개발자 계정을 개설하면 바로 이름을 등록하여 앱을 만들 수 있습니다.

Windows 앱을 만들기 위한 Visual Studio 템플릿

2/18/2021

Visual Studio 2019 는 C#이나 C++로 앱을 만들 때 도움이 되는 다양한 프로젝트 및 아이템 템플릿을 제공합니다. 이번 자료는 그러한 템플릿들에 대하여 설명하고 당신의 시나리오에 맞게 선택을 도와줄 수 있습니다.

- 프로젝트 템플릿에는 프로젝트 파일, 코드 파일 및 앱 또는 앱에서 로드하여 사용할 수 있는 구성 요소를 빌드하도록 구성된 기타 자산이 포함됩니다.
- 아이템 템플릿은 일반적으로 사용되는 코드와 개발 시간을 줄이기 위해 프로젝트에 추가할 수 있는 XAML 을 포함하는 프로젝트 파일입니다. 예를 들어 항목 템플릿을 사용하여 새 창, 페이지 또는 컨트롤을 앱에 추가할 수 있습니다

WinUI 템플릿

Windows UI 라이브러리 (WinUI)는 데스크톱(.NET, 네이티브 Win32) 이나 UWP 플랫폼 등의 Windows 앱을 위한 최신식 네이티브 유저 인터페이스 플랫폼입니다. WinUI 3(현재 개발자 프리뷰)는 WinUI 의 최근 주 버전이며, WinUI 를 Windows 앱을 위한, 완전한 UX 프레임워크로 변환시킵니다.

WinUI 3 에는 WinUI 기반 인터페이스를 가지는 앱을 위한 프로젝트/아이템 템플릿을 제공하는 Visual Studio 2019 용 VSIX 패키지가 포함되어 있습니다. WinUI 3 VSIX 패키지와 제공되는 프로젝트 템플릿에 대하여 더 알아보시려면 WinUI 3 Preview 4 설치를 보십시오.

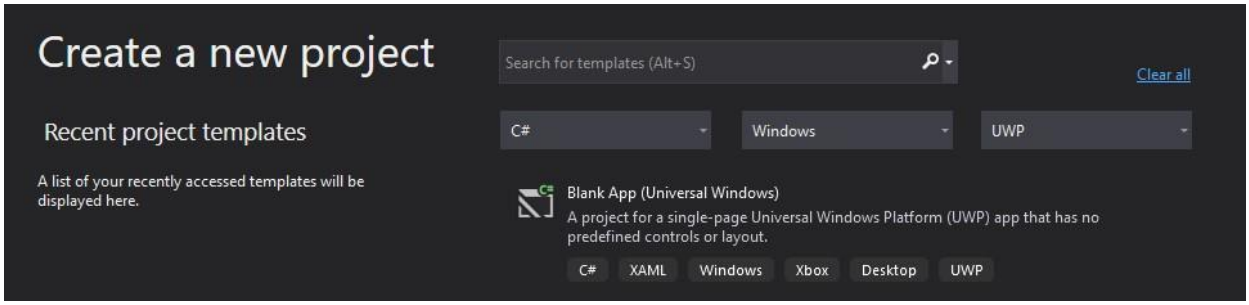
> **IMPORTANT**
>
> 관련 Visual Studio 템플릿을 포함한 WinUI 3 는 현재 개발자 커뮤니티의 피드백을 수집하고 초기 평가를 위한 개발자 미리보기로 제공됩니다. 지금은 생산성 앱에 사용해서는 안 됩니다.

UWP 템플릿

Visual Studio 는 C# 또는 C++로 UWP 앱을 만들기 위한 다양한 프로젝트 템플릿을 제공합니다. 이러한 프로젝트 템플릿을 사용하려면 Visual Studio 를 설치할 때 범용 Windows 플랫폼 개발 워크로드를 포함해야 합니다. C++ 프로젝트 템플릿의 경우 범용 Windows 플랫폼 개발 워크로드에 대한 C++(v142) 범용 Windows 플랫폼 도구(옵션)도 포함해야 합니다.

C# 및 UWP 프로젝트 템플릿

UWP C# 프로젝트 템플릿에 접근하려면 새 프로젝트에서 언어를 C#, 플랫폼을 Windows, 프로젝트 타입을 UWP 로 필터를 설정합니다.



C# UWP 앱을 위하여 다음 프로젝트 템플릿들을 사용할 수 있습니다.

템플릿	설명
빈 앱 (범용 Windows)	UWP 앱을 생성합니다. 생성된 프로젝트는 Windows.UI.Xaml.Controls.Page 클래스에서 파생된 기본 페이지를 가집니다. 그것으로 UI 를 만들기 시작할 수 있습니다.
단위 테스트 앱 (범용 Windows)	UWP 앱을 위한 C# 단위 테스트 프로젝트를 생성합니다. 추가적인 정보는 단위 테스트 C# 코드를 참조하세요.

C# UWP 앱의 구성 요소를 만들기 위해 다음 프로젝트 템플릿들을 사용할 수 있습니다.

템플릿	설명
클래스 라이브러리 (범용 Windows)	관리되는 코드로 작성된 다른 UWP 응용프로그램에서 사용할 수 있는 관리 클래스 라이브러리(DLL)를 C#에 만듭니다.
Windows 런타임 구성 요소 (범용 Windows)	앱이 작성된 프로그래밍 언어에 관계없이 모든 UWP 앱에서 사용할 수 있는 Windows Runtime 구성 요소를 C#에 생성합니다.
선택적 코드 패키지 (범용 Windows)	앱에서 로드할 수 있는 실행 파일 C# 코드가 있는 선택적 패키지를 생성합니다. 자세한 내용은 실행 코드가 있는 선택적 패키지를 참조하십시오.

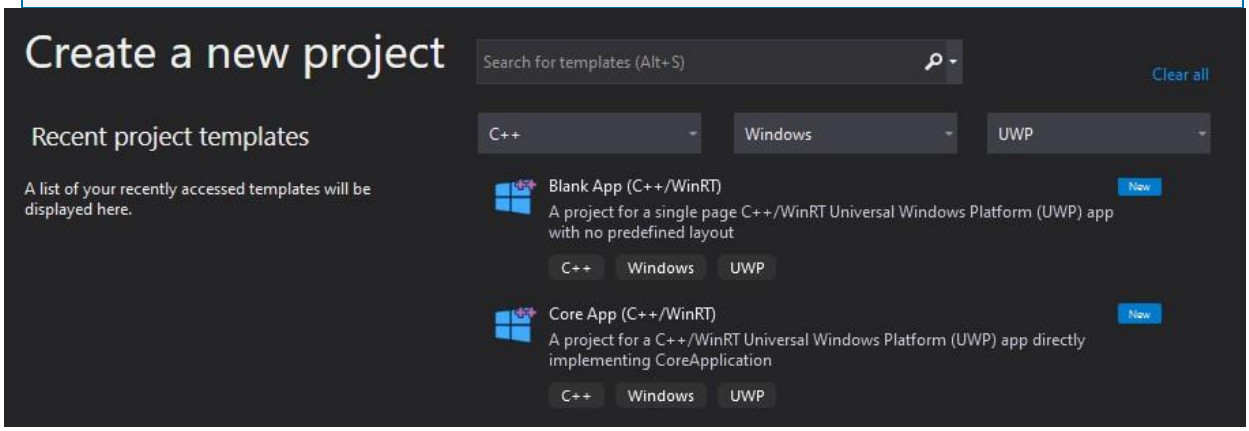
C++ 및 UWP 프로젝트 템플릿

C++ UWP 앱을 만들기 위해서 사용할 수 있는 2 가지 다른 기술이 있습니다.

- 권장되는 기술은 C++/WinRT 입니다. 이것은 전적으로 헤더 파일에 구현된 C++ 언어 프로젝션으로, 최신 WinRT API 에 대한 퍼스트 클래스 액세스를 제공하도록 설계되었습니다.
- 더 이전의 기술인 C++/CX 확장 집합을 사용할 수도 있습니다. C++/CX 는 지금도 지원되지만, C++/WinRT 를 더 권장합니다.

UWP C++ 프로젝트 템플릿에 접근하려면 새 프로젝트에서 언어를 C++, 플랫폼을 Windows, 프로젝트 타입을 UWP 로 필터를 설정합니다.

> **NOTE**
> 기본적으로 Visual Studio 에서 UWP 개발 워크로드는 C++/CX 에만 액세스할 수 있습니다. C++/WinRT 프로젝트 템플릿을 사용하려면 C++/WinRT VSIX 패키지를 설치해야 합니다.



C++ UWP 앱을 만들기 위해 다음 프로젝트 템플릿들을 사용할 수 있습니다.

템플릿	설명
빈 앱 (C++/WinRT)	XAML UI 를 가지는 C++/WinRT UWP 앱을 생성합니다. 생성된 프로젝트는 Windows.UI.Xaml.Controls.Page 클래스에서 파생된 기본 페이지를 가집니다. 그것으로 UI 를 만들기 시작할 수 있습니다.
코어 앱 (C++/WinRT)	CoreApplication 을 사용하여 XAML 사용자 인터페이스 대신 다양한 UI 프레임워크와 통합하는 C++/Win UWP 앱을 생성합니다. 이 프로젝트 템플릿을 사용하여 DirectX 를 사용하는 간단한 게임을 만드는 방법을 보여주는 자세한 내용은 DirectX 를 사용하여 간단한 UWP 게임 만들기를 참조하십시오.
빈 앱 (범용 Windows - C++/CX)	XAML UI 를 가지는 C++/WinRT UWP 앱을 생성합니다. 생성된 프로젝트는 Windows.UI.Xaml.Controls.Page 클래스에서 파생된 기본 페이지를 가집니다. 그것으로 UI 를 만들기 시작할 수 있습니다. (주: 원본 자료에 이렇게 써 있었음)
DirectX 11 and XAML 앱 (범용 Windows - C++/CX)	DirectX 11 과 SwapCahinPanel 을 사용하여 XAML UI 컨트롤을 사용할 수 있는 UWP 앱을 생성합니다. 추가적인 정보는 DirectX 게임 프로젝트 템플릿을 참조하십시오.
DirectX 11 앱 (범용 Windows - C++/CX)	DirectX 11 을 사용하는 UWP 앱을 생성합니다. 추가적인 정보는 DirectX 게임 프로젝트 템플릿을 참조하십시오.
DirectX 12 앱 (범용 Windows - C++/CX)	DirectX 12 를 사용하는 UWP 앱을 생성합니다. 추가적인 정보는 DirectX 게임 프로젝트 템플릿을 참조하십시오.
단위 테스트 앱 (범용 Windows - C++/CX)	C++/CX UWP 앱 단위 테스트 프로젝트를 만듭니다. 추가적인 정보는 C++ DLL 테스트하는 방법을 참조하십시오.

C++ UWP 앱의 구성 요소를 만들기 위해 다음 프로젝트 템플릿들을 사용할 수 있습니다.

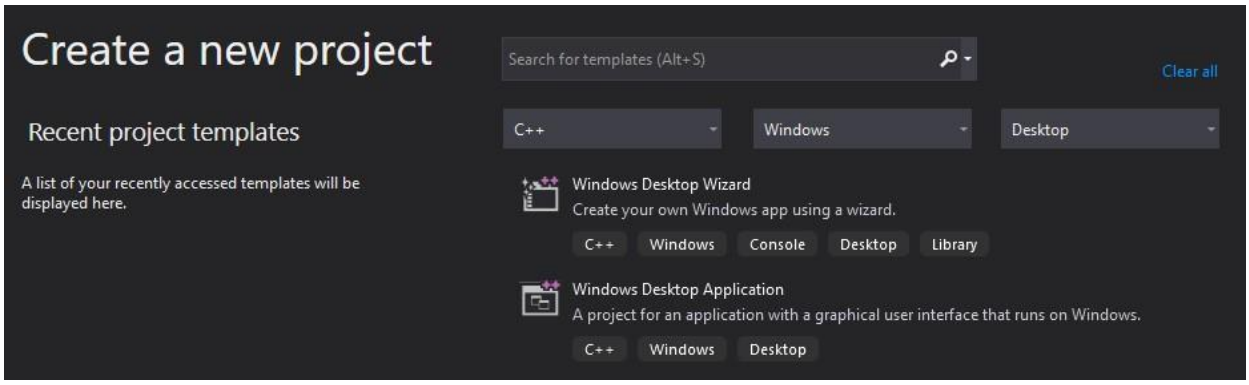
템플릿	설명
Windows 런타임 구성 요소 (C++/WinRT)	사용 언어에 관계없이 UWP 에서 사용할 수 있는 C++/WinRT Windows 런타임 구성 요소를 만듭니다.
Windows 런타임 구성 요소 (범용 Windows)	사용 언어에 관계없이 UWP 에서 사용할 수 있는 C++/CX Windows 런타임 구성 요소를 만듭니다.
DLL (범용 Windows)	UWP 에서 사용할 수 있는 C++/CX 동적 링킹 라이브러리를 만드는 프로젝트입니다. 추가적인 정보는 DLLs(C++/CX)를 참조하십시오.
정적 라이브러리 (범용 Windows)	UWP 에서 사용할 수 있는 C++/CX 정적 라이브러리(LIB)를 만드는 프로젝트입니다. 추가적인 정보는 정적 라이브러리(C++/CX)를 참조하십시오.

C++/Win32 템플릿

Visual Studio 는 기본 C++ 및 Win32 API 에 직접 액세스할 수 있는 데스크톱 Windows 앱을 만들기 위한 다양한 프로젝트 템플릿을 제공합니다. 이러한 프로젝트 템플릿을 사용하려면 Visual Studio 를 설치할 때 C++ 워크로드가 포함된 데스크톱 개발을 포함해야 합니다. 이 워크로드에는 데스크톱 애플리케이션, 콘솔 애플리케이션 및 라이브러리를 구축하기 위한 프로젝트 템플릿이 포함됩니다.

데스크톱 앱을 위한 프로젝트 템플릿

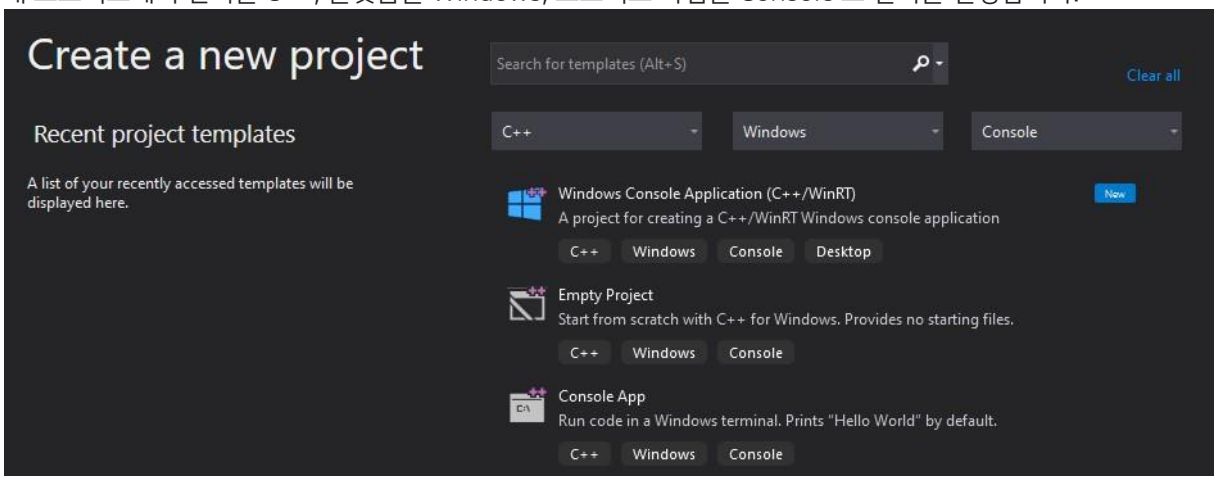
새 프로젝트에서 언어를 C++, 플랫폼을 Windows, 프로젝트 타입을 데스크톱으로 필터를 설정합니다.



템플릿	설명
Windows 데스크톱 앱	기본 Windows 데스크톱 앱을 C++로 만듭니다. 추가적인 정보는 가이드: 전통적 Windows 데스크톱 앱 만들기를 참고하십시오.
Windows 데스크톱 마법사	클래식 Windows 데스크톱 앱, 콘솔 앱, 동적 링크 라이브러리(DLL) 또는 정적 라이브러리 중 하나를 만드는 데 사용할 수 있는 단계별 마법사를 제공합니다. 자세한 내용은 Windows 데스크톱 마법사 및 가이드: 전통적 Windows 데스크톱 응용 프로그램 만들기를 참조하십시오.
Windows 앱 패키징 프로젝트	MSIX 패키지로 데스크톱 앱을 빌드하는 데 사용할 수 있는 프로젝트를 만듭니다. 이를 통해 최신 배포 환경, 패키지 확장을 통해 Windows 10 기능과 통합할 수 있는 기능 등이 제공됩니다. 자세한 내용은 Windows 응용 프로그램 패키징 프로젝트를 참조하십시오.

콘솔 앱 프로젝트 템플릿

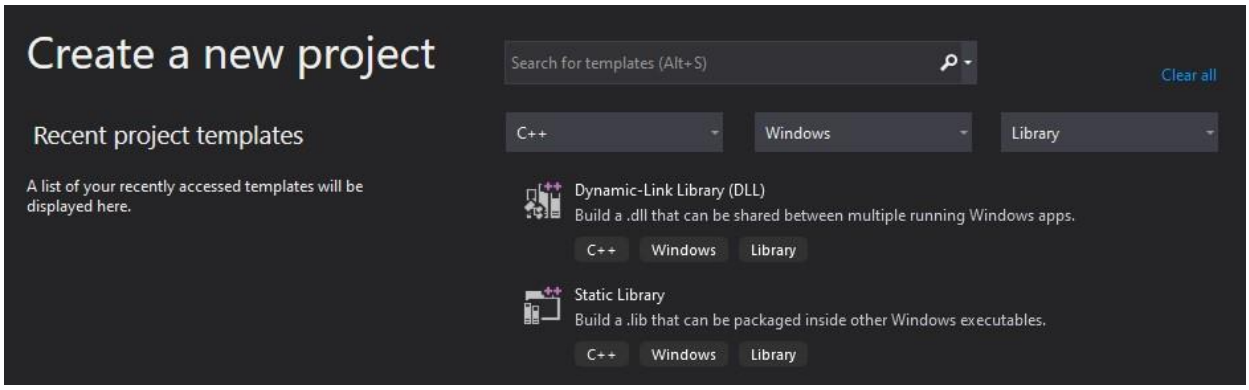
새 프로젝트에서 언어를 C++, 플랫폼을 Windows, 프로젝트 타입을 Console 로 필터를 설정합니다.



템플릿	설명
Windows 콘솔 앱 (C++/WinRT)	UI 없는 C++/WinRT 콘솔 앱을 생성합니다. 추가적인 정보는 C++/WinRT 빠른 시작을 참조하십시오. 이 프로젝트 템플릿은 C++/WinRT VSIX 를 필요로 합니다.
콘솔 앱	UI 없는 콘솔 앱을 생성합니다. 추가적인 정보는 가이드: 표준 C++ 프로그램 만들기를 참조하십시오.
빈 프로젝트	앱, 라이브러리, DLL 을 만들기 위한 빈 프로젝트입니다. 필요한 코드나 리소스를 직접 추가해야 합니다.

라이브러리를 위한 프로젝트 템플릿

새 프로젝트에서 언어를 C++, 플랫폼을 Windows, 프로젝트 타입을 라이브러리로 필터를 설정합니다.



템플릿	DESCRIPT ION
동적 링킹 라이브러리 (DLL)	동적 링킹 라이브러리를 만드는 프로젝트입니다. 추가적인 정보는 가이드: DLL 만들고 사용하기를 참조하십시오.
정적 라이브러리	정적 라이브러리(LIB)를 만드는 프로젝트입니다. 추가적인 정보는 가이드: 라이브러리 만들고 사용하기를 참조하십시오.

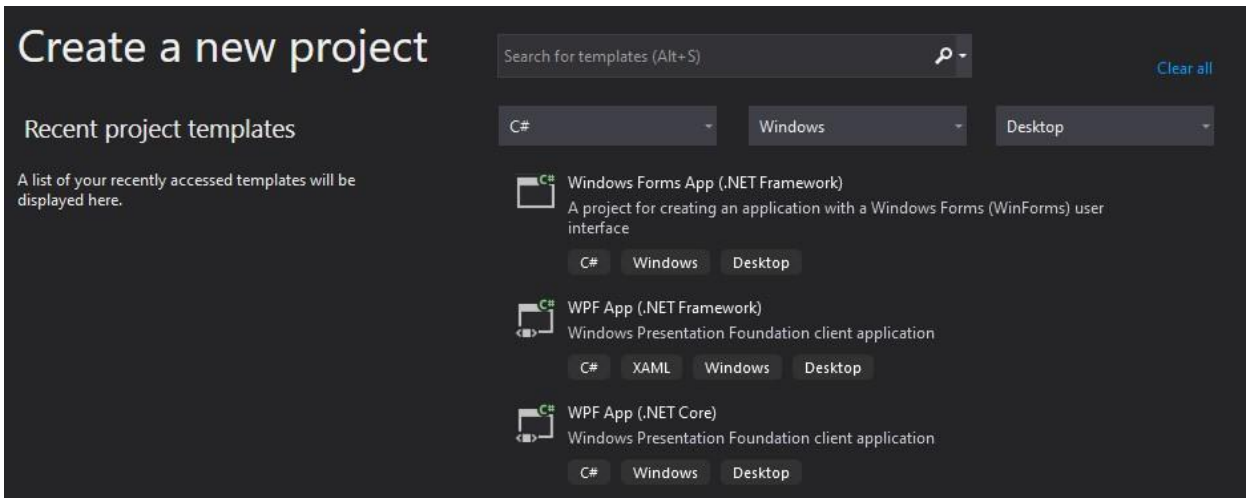
네이티브 C++와 Win32 를 위한 아이템 템플릿

C++ 프로젝트 템플릿은 새 파일 및 리소스 추가와 같은 작업을 할 수 있도록 많은 아이템 템플릿을 포함합니다. 전체 목록은 Visual C++ 사용 및 새 아이템 템플릿 추가를 참조하십시오.

.NET 템플릿

Visual Studio 는 .NET 및 C#을 사용하는 데스크톱 Windows 앱을 빌드하기 위한 다양한 프로젝트 템플릿을 제공합니다. 이러한 프로젝트 템플릿을 사용하려면 Visual Studio 를 설치할 때 .NET 데스크톱 개발 워크로드를 포함해야 합니다.

새 프로젝트에서 언어 C#, 플랫폼 Windows, 프로젝트 타입 데스크톱으로 필터를 설정합니다.



C#과 .NET 을 사용하는 앱을 만들 때 다음 프로젝트 템플릿을 이용할 수 있습니다.

템플릿	설명
WPF 앱 (.NET Core)	.NET Core 를 대상으로 하는 WPF 앱을 만듭니다. WPF 앱 만들기에서 사용법이 안내됩니다.
WPF 앱 (.NET Framework)	.NET Framework 를 대상으로 하는 WPF 앱을 만듭니다. 튜토리얼: 첫 WPF 앱 만들기에서 사용법이 안내됩니다.
Windows Forms 앱 (.NET Core)	.NET Core 대상의 Windows Forms 앱을 만듭니다.
Windows Forms 앱 (.NET Framework)	.NET Framework 대상의 Windows Forms 앱을 만듭니다. Visual Studio 에서 C#으로 Windows Forms 앱 만들기에서 사용법이 안내됩니다.
Windows 애플리케이션 패키징 프로젝트	WPF 또는 Windows Forms 앱을 MSIX 패키지로 빌드하는 데 사용할 수 있는 프로젝트를 만듭니다. 이를 통해 최신 배포 환경, 패키지 확장을 통해 Windows 10 기능과 통합할 수 있는 기능 등이 제공됩니다. 자세한 내용은 Windows 응용 프로그램 패키징 프로젝트를 참조하십시오.

데스크톱 앱 현대화하기

2/18/2021

Windows 10 및 UWP(Universal Windows Platform)는 데스크톱 앱에서 최신 환경을 제공하는 데 사용할 수 있는 다양한 기능을 제공합니다. 이러한 기능 중 대부분은 다른 플랫폼에 맞게 애플리케이션을 다시 작성할 필요 없이 자신만의 속도로 데스크톱 애플리케이션에 적용할 수 있는 모듈형 구성 요소로 제공됩니다. Windows 10 및 UWP 에서 적용할 부분을 선택하여 기존 데스크톱 앱을 향상시킬 수 있습니다.

이 자료에서는 현재 데스크톱 앱에서 사용할 수 있는 Windows 10 및 UWP 기능에 대해 설명합니다. 이 문서에 설명된 많은 기능을 사용하기 위해 기존 앱을 현대화하는 방법을 설명하는 자습서는 여기를 참조하십시오.

> **NOTE**
> 데스크톱 앱을 Windows 10 으로 마이그레이션하는 데 도움이 필요하십니까? Desktop App Assure 서비스는 Windows 10 에 애플리케이션을 포팅하는 개발자에게 비용 없이 직접 지원을 제공합니다. 이 프로그램은 모든 ISV 및 적격 기업에서 사용할 수 있습니다. 자격 및 프로그램 자체에 대한 자세한 내용은 여기를 참조하십시오. 지금 시작하려면 요청을 제출하세요.

Windows UI 라이브러리

Windows UI 라이브러리는 Windows 10 앱에 대한 컨트롤 및 기타 사용자 인터페이스 요소를 제공하는 NuGet 패키지 집합입니다. WinUI 는 Windows 10 의 하위 버전을 대상으로 하는 UWP 앱을 위해 WinRT XAML 컨트롤의 새로운 버전과 업데이트된 버전을 제공하는 툴킷으로 시작했습니다. WinUI 의 범위가 확장되어 이제는 UWP, .NET 및 기본 Win32 에서 Windows 10 앱을 위한 최신 기본 사용자 인터페이스(UI) 플랫폼이 되었습니다.

다음과 같은 방법으로 WinUI 를 데스크톱 앱에 사용할 수 있습니다.

- XAML Islands 를 이용하여 기존 WPF, Windows Forms, C++/Win32 앱을 업데이트하고 WinUI 2.x 컨트롤로 호스트합니다.
- WinUi 3.0 (현재 프리뷰)를 사용하여 완전히 WinUI 기반 UI 를 가지며 관리된 C#/.NET Core and native C++/Win32 데스크톱 앱을 제작합니다.

Windows UI (WinUI) 라이브러리

MSIX 패키지

MSIX 는 UWP, WPF, Windows Forms, Win32 앱을 포함한 모든 Windows 앱에 범용 패키지 환경을 제공하는 최신 Windows 앱 패키지 포맷입니다. MSIX 는 MSI, .appx, App-V 및 ClickOnce 설치 기술의 최고 측면을 통합하여 현대적이고 안정적인 패키징 환경을 제공합니다.

데스크톱 Windows 애플리케이션을 MSIX 패키지로 패키징하면 강력한 설치 및 업데이트 환경, 유연한 기능 시스템을 갖춘 관리 보안 모델, 마이크로소프트 Store 지원, 엔터프라이즈 관리 및 다양한 사용자 지정 배포 모델에 액세스할 수 있습니다.

더 많은 정보는 MSIX 문서의 데스크톱 앱 패키지로 만들기를 참고하십시오.

.NET Core 3

.NET Core 3 는 .NET Core 의 최신 주 릴리스입니다. 이번 릴리스의 주 업데이트는 Windows Forms 및 WPF 앱을 포함한 Windows 데스크톱 앱 지원입니다. .NET Core 3 에서 새로운 Windows 데스크톱 앱과 기존 Windows 데스크톱 앱을 실행하고 .NET Core 가 제공하는 모든 혜택을 누릴 수 있습니다. XAML Islands 에서 호스팅되는 WinRT XAML 컨트롤은 .NET Core 3 를 대상으로 하는 Windows Forms 및 WPF 앱에서도 사용할 수 있습니다.

더 많은 정보는 .NET Core 3.0 의 새로운 기능을 참고하십시오.

Windows 런타임 API

WPF, Windows Forms 또는 C++ Win32 데스크톱 앱에서 직접 많은 Windows Runtime API 를 호출하여 Windows 10 사용자를 위한 최신 환경을 통합할 수 있습니다. 예를 들어 Windows Runtime API 를 호출하여 데스크톱 앱에 토스트 알림을 추가할 수 있습니다.

더 많은 정보는 데스크톱 앱에서 Windows 런타임 API 호출을 참고하십시오.

WinRT XAML 컨트롤 (XAML Islands) 호스팅

Windows 10 버전 1903 부터는 WPF, Windows Forms 또는 HWND(윈도우 핸들)와 연결된 C++ Win32 앱의 UI 요소에 UWP XAML 컨트롤을 직접 추가할 수 있습니다. 따라서 Fluent Design System 을 지원하는 Windows Ink 및 컨트롤과 같은 최신 UWP 기능을 데스크톱 앱의 창 및 기타 디스플레이 표면에 완전히 통합할 수 있습니다. 이 개발자의 시나리오를 XAML Islands 라고 부르기도 합니다.

더 많은 정보는 데스크톱 앱에서 WinRT XAML 컨트롤을 참고하십시오.

데스크톱 앱에서 Visual layer 사용

이제 UWP 가 아닌 데스크톱 앱에서 Windows Runtime API 를 사용하여 WPF, Windows Forms 및 C++ Win32 앱의 모양, 느낌 및 기능을 향상하고 UWP 를 통해서만 사용할 수 있는 최신 Windows 10 UI 기능을 활용할 수 있습니다. 이 기능은 XAML Islands 를 사용하여 호스팅할 수 있는 기본 제공 WinRT XAML 컨트롤을 벗어나는 사용자 지정 환경을 생성해야 할 때 유용합니다.

더 많은 정보는 Visual layer 를 사용하여 데스크톱 앱 현대화하기를 참고하십시오.

패키지 ID 를 가진 앱에 사용할 수 있는 다른 기능들

일부 최신 Windows 10 환경은 패키지 ID 가 있는 데스크톱 앱에서만 사용할 수 있습니다. 이러한 기능에는 특정 Windows Runtime API, 패키지 확장 및 UWP 구성 요소가 포함됩니다. 자세한 내용은 패키지 ID 가 필요한 기능을 참조하십시오.

데스크톱 앱에서 ID 를 부여하는 방법은 다음과 같습니다.

- MSIX 패키지로 만들기. MSIX 는 모든 Windows 앱, WPF, Windows Forms 및 Win32 앱에 범용 패키지 환경을 제공하는 최신 앱 패키지 포맷입니다. 강력한 설치 및 업데이트 환경, 유연한 기능 시스템을 갖춘 관리형 보안 모델, Microsoft Store 지원, 엔터프라이즈 관리 및 다양한 맞춤형 배포 모델을 제공합니다. 자세한 내용은 MSIX 설명서의 데스크톱 앱을 패키지로 만들기를 참조하십시오.
- Windows 10(윈도우 10) 버전 2004 부터 데스크톱 앱 배포에 MSIX 패키지를 적용할 수 없는 경우 패키지 매니페스트만 포함된 sparse MSIX 패키지를 생성하여 패키지 ID 를 부여할 수 있습니다. 자세한 내용은 비패키지 데스크톱 앱에 ID 부여를 참조하십시오.

데스크톱 앱에 최적화된 WinRT XAML 컨트롤

데스크톱 장치 제품군만을 대상으로 하는 UWP 앱을 구축하거나 WPF, Windows Forms 또는 C++ Win32 데스크톱 앱에서 WinRT XAML 컨트롤을 사용하고자 하는 경우, 다음과 같이 새롭게 업데이트된 WinRT XAML 컨트롤은 Fluent Design System 을 통해 데스크톱에 최적화된 환경을 제공하도록 설계되었습니다. 이러한 컨트롤은 Windows 10, 버전 1809(2018.10 업데이트, v10.0.17763)에 도입되었습니다.

컨트롤	설명
MenuBar	CommandBar 에서 허용하는 것보다 더 많은 구성 또는 그룹이 필요할 수 있는 앱에 대한 명령 집합을 쉽고 빠르게 노출할 수 있는 방법을 제공합니다.
DropDownButton	더 많은 옵션을 포함하는 플라이아웃이 첨부되어 있음을 시각적 지표로 v 모양을 표시합니다.

SplitButton	별도로 호출할 수 있는 두 개의 부분으로 된 버튼을 제공합니다. 하나는 표준 버튼처럼 작동하여 즉시 작업을 실행합니다. 다른 하나는 사용자가 선택할 수 있는 추가 옵션이 포함된 플라이아웃을 호출합니다.
ToggleSplitButton	별도로 호출할 수 있는 두 개의 부분으로 된 버튼을 제공합니다. 하나는 ON/OFF 버튼과 같습니다. 다른 하나는 사용자가 선택할 수 있는 추가 옵션이 포함된 플라이아웃을 호출합니다.
CommandBarFlyout	UI 캔버스에 있는 항목의 컨텍스트에서 일반 사용자 작업을 표시할 수 있습니다.
ComboBox	이제 사용자가 컨트롤에 나열되지 않은 값을 입력할 수 있도록 콤보 박스를 편집할 수 있습니다.
TreeView	이제 데이터 바인딩, 항목 템플릿 및 드래그/드롭을 사용하도록 트리 보기를 구성할 수 있습니다.
DataGridView	데이터 컬렉션을 행과 열에 유연하게 표시할 수 있는 방법을 제공합니다. 이 컨트롤은 Windows 커뮤니티 툴킷에서 사용할 수 있습니다.

현대 데스크톱 앱의 다른 기술들

Microsoft 그래프

Microsoft Graph 는 수백만 사용자의 데이터와 상호작용하는 조직 및 소비자를 위한 앱을 구축하는 데 사용할 수 있는 API 모음입니다. Microsoft Graph 는 REST API 및 클라이언트 라이브러리를 표시하여 다음 데이터에 액세스합니다.

- Azure Active Directory
- Microsoft 365 오피스 앱: SharePoint, OneDrive, Outlook/Exchange, Microsoft Teams, OneNote, Planner, and Excel
- 기업 이동/보안 서비스: Identity Manager, Intune, Advanced Threat Analytics, Advanced Threat Protection
- Windows 10 서비스: 활동, 장치

더 많은 정보는 Microsoft 그래프 문서를 참조해 주세요.

적응형 카드

Adaptive Cards 는 여러 장치와 플랫폼에서 공통적이고 일관된 방식으로 카드 기반 UI 콘텐츠를 교환하는 데 사용할 수 있는 개방형 교차 플랫폼 프레임워크입니다.

더 많은 정보는 적응형 카드 문서를 참조해 주세요.

데스크톱 앱에서 Windows 런타임 API 호출

2/18/2021

UWP(Universal Windows Platform) API 를 사용하여 Windows 10 사용자를 위해 켜지는 데스크톱 앱에 최신 환경을 추가할 수 있습니다.

먼저 필요한 참조로 프로젝트를 설정합니다. 그런 다음 코드에서 Windows Runtime API 를 호출하여 Windows 10 환경을 데스크톱 앱에 추가합니다. Windows 10 사용자에 대해 별도로 빌드하거나 실행 중인 Windows 버전에 관계없이 동일한 이진 파일을 모든 사용자에게 배포할 수 있습니다.

일부 Windows Runtime API 는 패키지 ID 가 있는 데스크톱 앱에서만 지원됩니다. 자세한 내용은 사용 가능한 Windows 런타임 API 를 참조하십시오.

.NET 프로젝트에서 Windows 런타임 API 를 사용하도록 수정

There are several options for .NET projects:

- .NET 5 Preview 8 부터는 프로젝트 파일에 TPM(Target Framework Moniker)을 추가하여 WinRT API 에 액세스할 수 있습니다. 이 옵션은 Windows 10 버전 1809 이상을 대상으로 하는 프로젝트에서 지원됩니다.
- 이전 버전의 .NET 의 경우 NuGet 패키지 **Microsoft.Windows.SDK.Contracts** 를 설치하여 프로젝트에 필요한 모든 참조를 추가할 수 있습니다. 이 옵션은 Windows 10 버전 1803 이상을 대상으로 하는 프로젝트에서 지원됩니다.
- .NET 5 Preview 8 와 .NET 의 이전 버전을 모두 대상으로 하는 경우, 두 옵션 모두 사용하도록 프로젝트 파일을 구성할 수 있습니다.

.NET 5: Target Framework Moniker 옵션 사용

이 옵션은 .NET 5(또는 이후 버전 릴리스) 및 Windows 10 1809 이상 버전 OS 릴리스 대상 프로젝트에서만 지원됩니다. 이 시나리오에 대한 자세한 배경 정보는 이 블로그 게시물을 참조하십시오.

1. Visual Studio 에서 프로젝트를 열고 솔루션 탐색기의 프로젝트를 우클릭하여 프로젝트 파일 수정을 선택하면, 다음과 같이 생긴 것을 볼 수 있습니다.

```
<Project Sdk="Microsoft.NET.Sdk.WindowsDesktop">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net5.0</TargetFramework>
    <UseWindowsForms>true</UseWindowsForms>
  </PropertyGroup>
</Project>
```

2. <TargetFramework> 다음의 값을 다음 중 하나로 수정합니다.

   net5.0-windows10.0.17763.0 (Windows 10, version 1809 대상)
   net5.0-windows10.0.18362.0 (Windows 10, version 1903 대상)
   net5.0-windows10.0.19041.0 (Windows 10, version 2004 대상)
   이를 테면 Windows 10 버전 2004 를 대상으로 하는 경우 다음과 같이 수정합니다.

```
<TargetFramework>net5.0-windows10.0.19041.0</TargetFramework>
```

3. 저장하고 프로젝트 파일을 닫습니다.

이전 버전 .NET: Microsoft.Windows.SDK.Contracts NuGet 패키지 설치

앱이 .NET. Core 3.x, .NET 5 Preview 7(이하), .NET Framework 를 사용하는 경우 이 선택지를 따릅니다. 타겟이 Windows 10 버전 1803 이후인 경우 지원됩니다.

1. 패키지 레퍼런스가 활성화되어 있어야 합니다.
   a. Visual Studio 에서 도구 -> NuGet 패키지 관리자 -> 패키지 관리자 설정
   b. **PackageReference** 가 기본 패키지 관리 포맷으로 선택되어 있어야 합니다.
2. Visual Studio 에서 프로젝트를 열고 솔루션 탐색기에서 프로젝트 우클릭 후 NuGet 패키지 관리를 선택합니다.
3. NuGet 패키지 관리자 창에서 찾아보기 탭을 선택하고 Microsoft.Windows.SDK.Contracts 를 찾습니다.
4. 페이지를 찾았으면 NuGet 패키지 관리자 창의 오른쪽 창에서 대상 Windows 10 버전을 기준으로 설치할 패키지 버전을 선택합니다.

   10.0.19041.xxxx: (Windows 10, version 2004 대상)
   10.0.18362.xxxx: (Windows 10, version 1903 대상)
   10.0.17763.xxxx: (Windows 10, version 1809 대상)
   10.0.17134.xxxx: (Windows 10, version 1803 대상)
5. 설치를 클릭합니다.

여러 버전의 .NET 대상일 때 구성하기

.NET 5 Preview 8 과 그 이전 버전(.NET Core 3.x, .NET Framework)을 같이 대상으로 하는 경우 Target Framework Moniker 를 사용하여 .NET 5 에 대한 WinRT API 참조를 자동으로 가져오도록 프로젝트 파일을 구성하고 이전 버전에 **Microsoft.Windows.SDK.Contracts** NuGet 패키지를 사용할 수 있습니다.

1. Visual Studio 에서 프로젝트를 열고 솔루션 탐색기의 프로젝트를 우클릭하여 프로젝트 파일 수정을 선택합니다. .NET Core 3.1 을 사용하는 앱은 다음과 같이 보여집니다.

```
<Project Sdk="Microsoft.NET.Sdk.WindowsDesktop">
 <PropertyGroup>
  <OutputType>WinExe</OutputType>
  <TargetFramework>netcoreapp3.1</TargetFramework>
  <UseWindowsForms>true</UseWindowsForms>
 </PropertyGroup>
</Project>
```

2. <TargetFramework> 다음의 값을 수정합니다. 여기서 모든 대상 .NET 버전의 Target Framework Moniker 를 ;으로 구분하여 명시합니다.

- .NET 5 Preview 8 이후:

  net5.0-windows10.0.17763.0 (Windows 10 버전 1809 대상)
  net5.0-windows10.0.18362.0 (Windows 10 버전 1903 대상)
  net5.0-windows10.0.19041.0 (Windows 10 버전 2004 대상)
- .NET Core 3.x: netcoreapp3.0 또는 netcoreapp3.1
- .NET Framework: net46.

  예를 들어 .NET Core 3.1 과 .NET 5 Preview 8 대상은 다음과 같습니다.

```
<TargetFrameworks>netcoreapp3.1;net5.0-windows10.0.19041.0</TargetFrameworks>
```

3. </PropertyGroup> 다음에 어떤 버전의 .NET Core 3.x, .NET Framework 등 타겟을 위해 **Microsoft.Windows.SDK.Contracts** 를 설치하도록 PackageReference 를 추가합니다. 이것은 ItemGroup 의 안에 있어야 합니다. .NET Core3.1 에 대하여 예를 들면 다음과 같습니다.

```
<ItemGroup>
 <PackageReference Condition="'$(TargetFramework)' == 'netcoreapp3.1'"          Include="Microsoft.Windows.SDK.Contracts"
            Version="10.0.19041.0" /> </ItemGroup>
```

전부 수행했다면 프로젝트 파일은 다음과 같이 생겼을 것입니다.

```
<Project Sdk="Microsoft.NET.Sdk.WindowsDesktop">
 <PropertyGroup>
  <OutputType>WinExe</OutputType>
  <TargetFrameworks>netcoreapp3.1;net5.0-windows10.0.19041.0</TargetFrameworks>
  <UseWPF>true</UseWPF>
 </PropertyGroup>
 <ItemGroup>
  <PackageReference Condition="'$(TargetFramework)' == 'netcoreapp3.1'"
            Include="Microsoft.Windows.SDK.Contracts"
            Version="10.0.19041.0" />   </ItemGroup>
</Project>
```

4. 저장하고 프로젝트 파일을 닫습니다.

C++ Win32 프로젝트가 Windows 런타임 API 를 사용하도록 수정

C++/WinRT 를 사용합니다. C++/WinRT 는 윈도우 런타임(WinRT) API 를 위한 완전 표준 현대식 C++17 언어 프로젝션으로, 헤더 파일 기반 라이브러리로 구현되며 최신 윈도우 API 에 대한 최고 수준의 액세스를 제공하도록 설계되었습니다.

C++/WinRT 를 사용하도록 구성하려면:

- 새 프로젝트의 경우, C++/WinRT Visual Studio Extension (VSIX)를 설치하고 거기에 포함된 C++/WinRT 프로젝트 템플릿을 선택합니다.
- 기존 프로젝트의 경우, Microsoft.Windows.CppWinRT NuGet 패키지를 설치합니다.

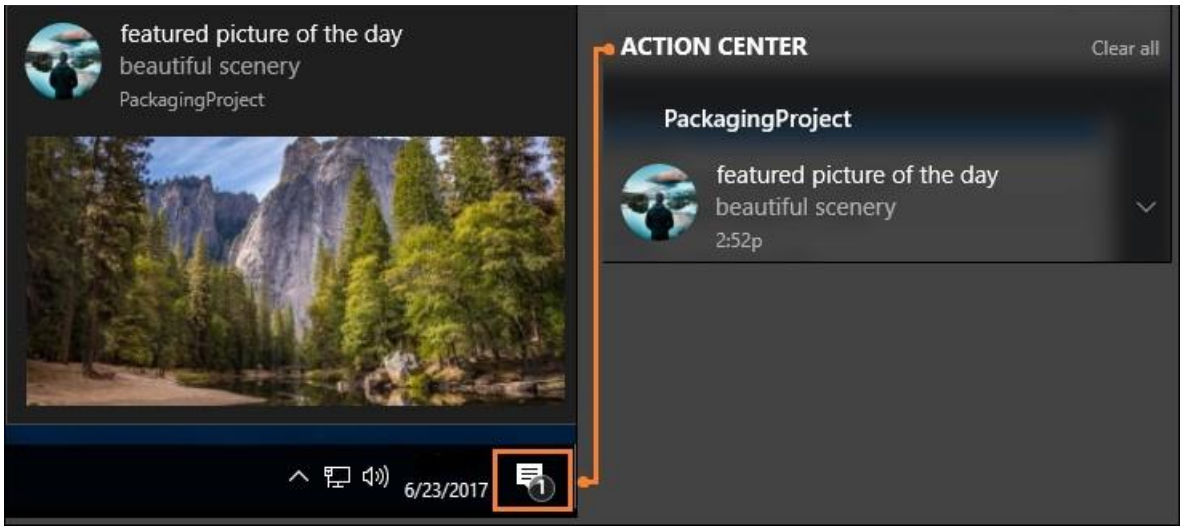
위 선택지에 대하여 더 자세한 사항은 여기를 참조하십시오.

Windows 10 experiences 추가

이제 Windows 10 에서 응용 프로그램을 실행할 때 켜지는 최신 환경을 추가할 준비가 되었습니다. 이 설계 흐름을 사용합니다.

☑ 먼저, 추가하려는 경험(UX)을 결정합니다.

선택할 수 있는 것이 많습니다. 예를 들어 수익화 API 를 사용하여 구매 주문 흐름을 단순화하거나, 다른 사용자가 게시한 새 사진과 같이 공유할 흥미로운 내용이 있을 때 응용 프로그램에 주의를 끌 수 있습니다.



사용자가 메시지를 무시하거나 무시하더라도 알림 센터에서 메시지를 다시 본 다음 메시지를 눌러 응용프로그램을 열 수 있습니다. 이렇게 하면 응용 프로그램에 대한 참여가 증가하고 응용 프로그램이 운영 체제와 긴밀하게 통합된 것처럼 보일 수 있는 추가 비용이 발생합니다. 이러한 경험을 위한 코드는 이 글의 뒷부분에서 보여드리겠습니다.

더 많은 아이디어는 UWP 문서를 보시면 많이 있습니다.

☑ 개선할지 확장할지 결정합니다.

개선 및 확장이라는 용어가 자주 등장하므로 잠시 시간을 내어 각 용어가 정확히 무엇을 의미하는지 설명하겠습니다.

개선은 데스크톱 앱에서 직접 호출할 수 있는 Windows 런타임 API 를 설명하는 데 사용되는 단어입니다(MSIX 패키지를 하도록 설정한 것과 관계없이). Windows 10 환경을 선택했으면 생성에 필요한 API 를 식별한 다음 해당 API 가 이 목록에 표시되는지 확인하십시오. 데스크톱 앱에서 직접 호출할 수 있는 API 목록입니다. API 가 이 목록에 나타나지 않으면 해당 API 와 관련된 기능이 UWP 프로세스 내에서만 실행될 수 있기 때문입니다. UWP 맵 컨트롤이나 Windows Hello 보안 프롬프트와 같은 UWP XAML 을 렌더링하는 API 를 포함하는 경우가 많습니다.

> **NOTE**
>
> UWP XAML 을 렌더링하는 API 는 일반적으로 데스크톱에서 직접 호출할 수 없지만 다른 방법을 사용할 수도 있습니다. UWP XAML 컨트롤이나 기타 사용자 지정 시각 환경을 호스팅하려면 XAML Islands(Windows 10, 버전 1903 부터 시작) 및 Visual layer(Windows 10, 버전 1803)을 사용할 수 있습니다. 이러한 기능은 패키지 또는 패키지화되지 않은 데스크톱 앱에서 사용할 수 있습니다.

데스크톱 앱을 MSIX 패키지로 패키징하도록 선택한 경우 다른 옵션은 솔루션에 UWP 프로젝트를 추가하여 애플리케이션을 확장하는 것입니다. 데스크톱 프로젝트는 여전히 응용 프로그램의 진입점이지만 UWP 프로젝트는 이 목록에 나타나지 않는 모든 API 에 대한 액세스를 제공합니다. 데스크탑 앱은 앱 서비스를 이용하여 UWP 프로세스와 통신할 수 있으며, 이를 설정하는 방법에 대한 많은 지침이 있습니다. UWP 프로젝트가 필요한 환경을 추가하려면 UWP 구성 요소로 확장을 참조하십시오.

☑ API 계약을 참조합니다.

데스크톱 앱에서 직접 API 를 호출할 수 있는 경우 브라우저를 열고 해당 API 의 참조 항목을 검색합니다. API 요약 아래에는 해당 API 의 API 계약을 설명하는 테이블이 있습니다. 이 표의 예는 다음과 같습니다.

Windows 10 requirements

Device family	Windows 10 (introduced v10.0.10240.0)
API contract	Windows.Foundation.UniversalApiContract (introduced v1)

.NET 기반 데스크톱 앱이 있는 경우 해당 API 계약에 참조를 추가한 다음 해당 파일의 Copy Local 속성을 False 로 설정합니다. C++ 기반 프로젝트가 있는 경우 이 계약이 포함된 폴더의 경로인 추가 포함 디렉터리에 추가하십시오.

☑ 경험을 추가하기 위해 API 를 호출합니다.

앞에서 살펴본 알림 창을 표시할 때 사용하는 코드입니다. 이러한 API 가 이 목록에 표시되므로 이 코드를 데스크톱 앱에 추가하고 지금 바로 실행할 수 있습니다.

```
using Windows.Foundation; using
Windows.System; using
Windows.UI.Notifications; using
Windows.Data.Xml.Dom; ...

private void ShowToast()
{
   string title = "featured picture of the day";    string content = "beautiful scenery";
   string image = "https://picsum.photos/360/180?image=104";    string logo =
"https://picsum.photos/64?image=883";

   string xmlString =    $@"<toast><visual>
     <binding template='ToastGeneric'>
     <text>{title}</text>
     <text>{content}</text>
     <image src='{image}'/>
     <image src='{logo}' placement='appLogoOverride' hint-crop='circle'/>      </binding>
    </visual></toast>";

   XmlDocument toastXml = new XmlDocument();    toastXml.LoadXml(xmlString);

   ToastNotification toast = new ToastNotification(toastXml);

   ToastNotificationManager.CreateToastNotifier().Show(toast); }
```

```
#include <sstream>
#include <winrt/Windows.Data.Xml.Dom.h>
#include <winrt/Windows.UI.Notifications.h>

using namespace winrt::Windows::Foundation; using namespace winrt::Windows::System;
using namespace winrt::Windows::UI::Notifications; using namespace winrt::Windows::Data::Xml::Dom;

void UWP::ShowToast()
{
   std::wstring const title = L"featured picture of the day";    std::wstring const content = L"beautiful scenery";
   std::wstring const image = L"https://picsum.photos/360/180?image=104";    std::wstring const logo = L"https://picsum.photos/64?image=883";

   std::wostringstream xmlString;
   xmlString << L"<toast><visual><binding template='ToastGeneric'>" <<
      L"<text>" << title << L"</text>" <<
      L"<text>" << content << L"</text>" <<
      L"<image src='" << image << L"'/>" <<
      L"<image src='" << logo << L"'" <<
      L" placement='appLogoOverride' hint-crop='circle'/>" <<
      L"</binding></visual></toast>";    XmlDocument toastXml;    toastXml.LoadXml(xmlString.str().c_str());

   ToastNotificationManager::CreateToastNotifier().Show(ToastNotification(toastXml)); }
```

```
using namespace Windows::Foundation; using namespace
Windows::System;
using namespace Windows::UI::Notifications; using namespace
Windows::Data::Xml::Dom;

void UWP::ShowToast()
{
 Platform::String ^title = "featured picture of the day";
 Platform::String ^content = "beautiful scenery";
 Platform::String ^image = "https://picsum.photos/360/180?image=104";  Platform::String ^logo =
"https://picsum.photos/64?image=883";

 Platform::String ^xmlString =
  L"<toast><visual><binding template='ToastGeneric'>" +
  L"<text>" + title + "</text>" +
  L"<text>"+ content + "</text>" +
  L"<image src='" + image + "'/>" +
  L"<image src='" + logo + "'" +
  L" placement='appLogoOverride' hint-crop='circle'/>" +
  L"</binding></visual></toast>";

 XmlDocument ^toastXml = ref new XmlDocument();  toastXml->LoadXml(xmlString);

 ToastNotificationManager::CreateToastNotifier()->Show(ref new ToastNotification(toastXml));
}
```

알림에 대하여 더 많은 정보를 알고 싶다면 적응적, 상호작용 토스트 알림을 참조하십시오.

Windows XP, Vista, 7/8 설치 기반 지원하기

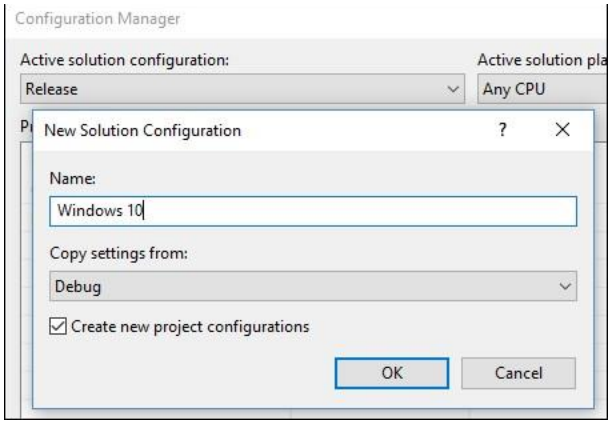
새 분기를 생성하고 별도의 코드베이스를 유지 관리할 필요 없이 Windows 10 용 응용 프로그램을 현대화할 수 있습니다.

Windows 10 사용자를 위한 별도의 이진 파일을 작성하려면 조건부 컴파일을 사용합니다. 모든 Windows 사용자에게 배포하는 이진 파일 집합을 하나 작성하려면 런타임 검사를 사용하십시오.

각 옵션에 대하여 짧게 알아보겠습니다.

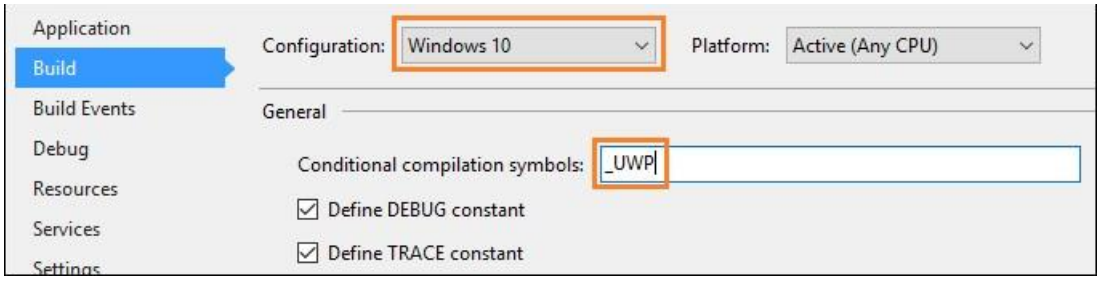
조건부 컴파일

하나의 코드베이스를 유지하고 Windows 10 사용자만을 위한 이진 파일 집합을 컴파일할 수 있습니다. 먼저 새 빌드 구성을 프로젝트에 추가합니다.

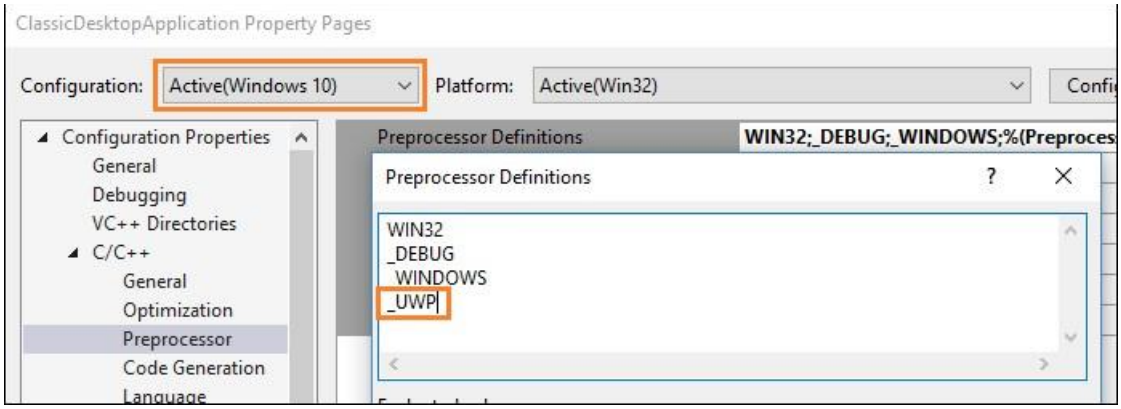


해당 빌드 구성의 경우 Windows 런타임 API 를 호출하는 코드를 식별할 상수를 생성합니다.

.NET 기반 프로젝트의 경우 이 상수는 조건부 컴파일 상수라고 합니다.



C++ 기반 프로젝트에서는 이 상수를 전처리기 정의라고 부릅니다.



UWP 코드 블록 앞에 해당 상수를 추가합니다.

```
[System.Diagnostics.Conditional("_UWP")] private void
ShowToast() { ... }
```

```
#if _UWP
void UWP::ShowToast() { ... }
#endif
```

컴파일러는 해당 상수가 활성 빌드 구성에서 정의된 경우에만 해당 코드를 빌드합니다.

런타임 체크

실행하는 Windows 버전에 관계없이 모든 Windows 사용자에 대해 하나의 이진 파일을 컴파일할 수 있습니다. 응용 프로그램은 사용자가 응용 프로그램을 Windows 10 에서 패키지 응용 프로그램으로 실행하는 경우에만 Windows 런타임 API 를 호출합니다.

런타임 체크를 추가하는 가장 쉬운 방법은 데스크톱 브리지 헬퍼 Nuget 패키지를 설치하고 **IsRunningAsUWP()** 함수를 사용하여 Windows 런타임 API 를 호출하는 모든 코드의 게이트를 해제하는 것입니다. 더 자세한 사항은 다음 블로그 포스트를 참조하십시오. 데스크톱 브리지 – 애플리케이션 컨텍스트 식별

관련 샘플

- Hello World 샘플
- 보조 타일 샘플
- 스토어 API 샘플
- UWP UpdateTask 를 구현하는 WinForms 앱
- Desktop app bridge to UWP 샘플

질문에 대한 답을 찾아보기

질문이 있으신가요? Stack Overflow 에서 Microsoft 에 질문하세요. 이 팀은 이 태그들을 지속 확인합니다. MSDN 포럼에서도 질문할 수 있습니다.

가용 Windows Runtime API

11/2/2020

데스크톱 앱은 UWP 앱으로 완전히 전환되지 않았더라도 많은 UWP API 를 사용할 수 있습니다. 이 자료에는 데스크톱 앱에서 사용할 수 있는 사용 가능한 클래스가 나열되어 있습니다.

대부분의 Windows Runtime API 는 데스크톱 앱에서 패키지 ID 를 사용해야 합니다. 대부분의 Windows Runtime API 는 패키지 ID 를 가진 앱에서 잘 작동합니다. 그러나 일부 기능 영역은 아직 완전히 테스트되지 않았거나 현재 의도한 대로 작동 중입니다.

기능 영역	세부 사항
약속, 연락, e-mail	현재 일부 API 는 지원되지 않습니다.
인증 및 사용자 ID	현재 일부 API 는 지원되지 않습니다.
백그라운드 작업	데스크톱 앱은 백그라운드 작업을 등록할 수 있지만, 완전 UWP 앱만 백그라운드 작업으로 실행할 수 있습니다. 더 많은 정보는 GitHub 의 백그라운드 작업 샘플을 확인해 주세요.
블루투스	현재 일부 API 는 지원되지 않습니다.
채팅, 전화통화	UWP 로 완전히 포팅되지 않으면 데스크톱 앱은 모바일에서 실행될 수 없습니다.
계약 활성화	완전 UWP 앱만 계약에 의해 활성화될 수 있습니다.
암호(Cryptography)	현재 일부 API 는 지원되지 않습니다.
Geolocation	현재 일부 API 는 지원되지 않습니다.
파일/폴더 선택기	ID 가 있는 앱은 파일 시스템에 접근 권한을 가지며, UWP 피커가 필요하지 않습니다.
미디어	현재 일부 API 는 지원되지 않습니다.
Point of service	현재 일부 API 는 지원되지 않습니다.
스마트카드	현재 일부 API 는 지원되지 않습니다.
PlayReady	현재 일부 API 는 지원되지 않습니다.
VPN	현재 일부 API 는 지원되지 않습니다.
월렛	UWP 로 완전히 포팅되지 않으면 데스크톱 앱은 모바일에서 실행될 수 없습니다.
XAML UI	Windows 10, 1809 이하 버전은 데스크톱 앱의 UWP XAML 콘텐츠 호스팅을 지원하지 않습니다. 버전 1903 부터 XAML Islands 로 UWP XAML 콘텐츠를 사용할 수 있습니다.

간혹 위 영역의 API 가 제대로 작동하는 것으로 보일 수 있으나, 명시적으로 지원되는 것이 아닌 경우 의도치 않은 부작용/동작이 발생할 수 있으므로 API 사용을 피하는 것이 좋습니다. 향후 릴리스에서 지원될 수 있습니다. 그 사이에는 Win32, .NET API 를 대신 사용해야 합니다.

API 목록

다음은 데스크톱 앱에서 지원되는 Windows 런타임 API 의 전체 목록입니다. 패키지 ID 가 부여된 앱에서만 지원되는 API 와 모든 데스크톱 앱에서 지원하는 API 로 나뉘어 보여집니다.

Windows Runtime API 사용 방법에 대한 예는 GitHub 의 데스크톱 브리지 샘플 및 UWP(Universal Windows Platform) 앱 샘플을 참조하십시오. 또한 Building App for Windows 블로그 게시물인 데스크톱 앱에서 Windows 10 API 호출을 참조할 수 있습니다.

패키지 ID 가 부여된 앱에서만 지원되는 API 목록

Windows.ApplicationModel.DataTransfer.DataProviderHandler
Windows.ApplicationModel.DataTransfer.DataRequest
Windows.ApplicationModel.DataTransfer.DataRequestDeferral
Windows.ApplicationModel.DataTransfer.DataRequestedEventArgs
Windows.ApplicationModel.DataTransfer.DataTransferManager
Windows.ApplicationModel.DataTransfer.SharedStorageAccessManager
Windows.ApplicationModel.DataTransfer.TargetApplicationChosenEventArgs
Windows.ApplicationModel.Resources.Core.NamedResource
Windows.ApplicationModel.Resources.Core.ResourceCandidate
Windows.ApplicationModel.Resources.Core.ResourceCandidateVectorView
Windows.ApplicationModel.Resources.Core.ResourceContext
Windows.ApplicationModel.Resources.Core.ResourceContextLanguagesVectorView
Windows.ApplicationModel.Resources.Core.ResourceManager
Windows.ApplicationModel.Resources.Core.ResourceMap
Windows.ApplicationModel.Resources.Core.ResourceMapIterator
Windows.ApplicationModel.Resources.Core.ResourceMapMapView
Windows.ApplicationModel.Resources.Core.ResourceMapMapViewIterator
Windows.ApplicationModel.Resources.Core.ResourceQualifier
Windows.ApplicationModel.Resources.Core.ResourceQualifierMapView
Windows.ApplicationModel.Resources.Core.ResourceQualifierObservableMap
Windows.ApplicationModel.Resources.Core.ResourceQualifierVectorView
Windows.ApplicationModel.Resources.ResourceLoader
Windows.Data.Pdf.PdfDocument
Windows.Data.Pdf.PdfPage
Windows.Data.Pdf.PdfPageDimensions
Windows.Data.Pdf.PdfPageRenderOptions
Windows.Data.Text.SelectableWordSegmentsTokenizingHandler
Windows.Data.Text.SemanticTextQuery
Windows.Data.Text.TextConversionGenerator
Windows.Data.Text.TextPredictionGenerator
Windows.Data.Text.TextReverseConversionGenerator
Windows.Data.Text.WordSegmentsTokenizingHandler
Windows.Data.Xml.Dom.DtdEntity
Windows.Data.Xml.Dom.DtdNotation
Windows.Data.Xml.Dom.XmlAttribute
Windows.Data.Xml.Dom.XmlCDataSection
Windows.Data.Xml.Dom.XmlComment
Windows.Data.Xml.Dom.XmlDocument
Windows.Data.Xml.Dom.XmlDocumentFragment
Windows.Data.Xml.Dom.XmlDocumentType
Windows.Data.Xml.Dom.XmlDomImplementation

Windows.Data.Xml.Dom.XmlElement
Windows.Data.Xml.Dom.XmlEntityReference
Windows.Data.Xml.Dom.XmlLoadSettings
Windows.Data.Xml.Dom.XmlNamedNodeMap
Windows.Data.Xml.Dom.XmlNodeList
Windows.Data.Xml.Dom.XmlProcessingInstruction
Windows.Data.Xml.Dom.XmlText
Windows.Data.Xml.Xsl.XsltProcessor
Windows.Devices.Input.KeyboardCapabilities
Windows.Devices.Input.MouseCapabilities
Windows.Devices.Input.MouseDevice
Windows.Devices.Input.MouseEventArgs
Windows.Devices.Input.PointerDevice
Windows.Devices.Input.TouchCapabilities
Windows.Devices.Lights.Lamp
Windows.Devices.Lights.LampAvailabilityChangedEventArgs
Windows.Devices.Perception.Provider.PerceptionStartFaceAuthenticationHandler
Windows.Devices.Perception.Provider.PerceptionStopFaceAuthenticationHandler
Windows.Devices.PointOfService.MagneticStripeReader
Windows.Devices.PointOfService.MagneticStripeReaderAamvaCardDataReceivedEventArgs
Windows.Devices.PointOfService.MagneticStripeReaderBankCardDataReceivedEventArgs
Windows.Devices.PointOfService.MagneticStripeReaderCapabilities
Windows.Devices.PointOfService.MagneticStripeReaderCardTypes
Windows.Devices.PointOfService.MagneticStripeReaderEncryptionAlgorithms
Windows.Devices.PointOfService.MagneticStripeReaderErrorOccurredEventArgs
Windows.Devices.PointOfService.MagneticStripeReaderReport
Windows.Devices.PointOfService.MagneticStripeReaderStatusUpdatedEventArgs
Windows.Devices.PointOfService.MagneticStripeReaderTrackData
Windows.Devices.PointOfService.MagneticStripeReaderVendorSpecificCardDataReceivedEvent
Args
Windows.Devices.Portable.ServiceDevice
Windows.Devices.Portable.StorageDevice
Windows.Devices.Printers.Print3DDevice
Windows.Devices.Printers.PrintSchema
Windows.Devices.SmartCards.SmartCard
Windows.Devices.SmartCards.SmartCardConnection
Windows.Devices.SmartCards.SmartCardReader
Windows.Foundation.AsyncActionCompletedHandler
Windows.Foundation.AsyncActionProgressHandler
Windows.Foundation.AsyncActionWithProgressCompletedHandler
Windows.Foundation.AsyncOperationCompletedHandler
Windows.Foundation.Collections.VectorChangedEventHandler
Windows.Foundation.DeferralCompletedHandler
Windows.Foundation.Diagnostics.FileLoggingSession
Windows.Foundation.Diagnostics.LogFileGeneratedEventArgs
Windows.Foundation.Diagnostics.LoggingActivity
Windows.Foundation.Diagnostics.LoggingChannel
Windows.Foundation.Diagnostics.LoggingChannelOptions
Windows.Foundation.Diagnostics.LoggingFields

Windows.Foundation.Diagnostics.LoggingOptions
Windows.Foundation.Diagnostics.LoggingSession
Windows.Foundation.EventHandler
Windows.Foundation.MemoryBuffer
Windows.Globalization.ApplicationLanguages
Windows.Globalization.JapanesePhoneme
Windows.Globalization.JapanesePhoneticAnalyzer
Windows.Globalization.PhoneNumberFormatting.PhoneNumberFormatter
Windows.Globalization.PhoneNumberFormatting.PhoneNumberInfo
Windows.Graphics.Imaging.BitmapBuffer
Windows.Graphics.Imaging.BitmapCodecInformation
Windows.Graphics.Imaging.BitmapDecoder
Windows.Graphics.Imaging.BitmapEncoder
Windows.Graphics.Imaging.BitmapFrame
Windows.Graphics.Imaging.BitmapProperties
Windows.Graphics.Imaging.BitmapPropertiesView
Windows.Graphics.Imaging.BitmapPropertySet
Windows.Graphics.Imaging.BitmapTransform
Windows.Graphics.Imaging.BitmapTypedValue
Windows.Graphics.Imaging.ImageStream
Windows.Graphics.Imaging.PixelDataProvider
Windows.Graphics.Imaging.SoftwareBitmap
Windows.Graphics.Printing3D.Print3DTaskRequestedEventArgs
Windows.Graphics.Printing3D.Print3DTaskSourceRequestedHandler
Windows.Graphics.Printing3D.Printing3D3MFPackage
Windows.Graphics.Printing3D.Printing3DBaseMaterial
Windows.Graphics.Printing3D.Printing3DBaseMaterialGroup
Windows.Graphics.Printing3D.Printing3DColorMaterial
Windows.Graphics.Printing3D.Printing3DColorMaterialGroup
Windows.Graphics.Printing3D.Printing3DComponent
Windows.Graphics.Printing3D.Printing3DComponentWithMatrix
Windows.Graphics.Printing3D.Printing3DCompositeMaterial
Windows.Graphics.Printing3D.Printing3DCompositeMaterialGroup
Windows.Graphics.Printing3D.Printing3DFaceReductionOptions
Windows.Graphics.Printing3D.Printing3DMaterial
Windows.Graphics.Printing3D.Printing3DMesh
Windows.Graphics.Printing3D.Printing3DMeshVerificationResult
Windows.Graphics.Printing3D.Printing3DModel
Windows.Graphics.Printing3D.Printing3DModelTexture
Windows.Graphics.Printing3D.Printing3DMultiplePropertyMaterial
Windows.Graphics.Printing3D.Printing3DMultiplePropertyMaterialGroup
Windows.Graphics.Printing3D.Printing3DTexture2CoordMaterial
Windows.Graphics.Printing3D.Printing3DTexture2CoordMaterialGroup
Windows.Graphics.Printing3D.Printing3DTextureResource
Windows.Management.Core.ApplicationDataManager
Windows.Management.Deployment.DeploymentResult
Windows.Management.Deployment.PackageManager
Windows.Management.Deployment.PackageUserInformation
Windows.Management.Deployment.PackageVolume

Windows.Management.Workplace.MdmPolicy
Windows.Management.Workplace.WorkplaceSettings
Windows.Media.AudioBuffer
Windows.Media.Capture.AdvancedCapturedPhoto
Windows.Media.Capture.AppCaptureAlternateShortcutKeys
Windows.Media.Capture.AppCaptureManager
Windows.Media.Capture.AppCaptureSettings
Windows.Media.Capture.CapturedFrame
Windows.Media.Capture.MediaCaptureFailedEventArgs
Windows.Media.Capture.MediaCaptureFailedEventHandler
Windows.Media.Capture.MediaCapturePauseResult
Windows.Media.Capture.MediaCaptureStopResult
Windows.Media.Capture.OptionalReferencePhotoCapturedEventArgs
Windows.Media.Capture.RecordLimitationExceededEventHandler
Windows.Media.ClosedCaptioning.ClosedCaptionProperties
Windows.Media.Devices.DefaultAudioCaptureDeviceChangedEventArgs
Windows.Media.Devices.DefaultAudioRenderDeviceChangedEventArgs
Windows.Media.Devices.MediaDevice
Windows.Media.DialProtocol.DialApp
Windows.Media.DialProtocol.DialAppStateDetails
Windows.Media.DialProtocol.DialDevice
Windows.Media.FaceAnalysis.DetectedFace
Windows.Media.FaceAnalysis.FaceDetector
Windows.Media.FaceAnalysis.FaceTracker
Windows.Media.MediaExtensionManager
Windows.Media.MediaProperties.H264ProfileIds
Windows.Media.MediaProperties.MediaEncodingSubtypes
Windows.Media.MediaProperties.Mpeg2ProfileIds
Windows.Media.Ocr.OcrEngine
Windows.Media.Ocr.OcrLine
Windows.Media.Ocr.OcrResult
Windows.Media.Ocr.OcrWord
Windows.Media.Playback.PlaybackMediaMarker
Windows.Media.Playback.PlaybackMediaMarkerReachedEventArgs
Windows.Media.Playback.PlaybackMediaMarkerSequence
Windows.Media.SpeechRecognition.SpeechContinuousRecognitionCompletedEventArgs
Windows.Media.SpeechRecognition.SpeechContinuousRecognitionResultGeneratedEventArgs
Windows.Media.SpeechRecognition.SpeechContinuousRecognitionSession
Windows.Media.SpeechRecognition.SpeechRecognitionCompilationResult
Windows.Media.SpeechRecognition.SpeechRecognitionGrammarFileConstraint
Windows.Media.SpeechRecognition.SpeechRecognitionHypothesis
Windows.Media.SpeechRecognition.SpeechRecognitionHypothesisGeneratedEventArgs
Windows.Media.SpeechRecognition.SpeechRecognitionListConstraint
Windows.Media.SpeechRecognition.SpeechRecognitionQualityDegradingEventArgs
Windows.Media.SpeechRecognition.SpeechRecognitionResult
Windows.Media.SpeechRecognition.SpeechRecognitionSemanticInterpretation
Windows.Media.SpeechRecognition.SpeechRecognitionTopicConstraint
Windows.Media.SpeechRecognition.SpeechRecognitionVoiceCommandDefinitionConstraint
Windows.Media.SpeechRecognition.SpeechRecognizer

Windows.Media.SpeechRecognition.SpeechRecognizerStateChangedEventArgs
Windows.Media.SpeechRecognition.SpeechRecognizerTimeouts
Windows.Media.SpeechRecognition.SpeechRecognizerUIOptions
Windows.Media.SpeechSynthesis.SpeechSynthesisStream
Windows.Media.SpeechSynthesis.SpeechSynthesizer
Windows.Media.SpeechSynthesis.VoiceInformation
Windows.Networking.PushNotifications.PushNotificationChannel
Windows.Networking.PushNotifications.PushNotificationChannelManager
Windows.Networking.PushNotifications.PushNotificationReceivedEventArgs
Windows.Networking.PushNotifications.RawNotification
Windows.Networking.Sockets.DatagramSocketMessageReceivedEventArgs
Windows.Networking.Sockets.MessageWebSocketMessageReceivedEventArgs
Windows.Services.Maps.Guidance.GuidanceAudioNotificationRequestedEventArgs
Windows.Services.Maps.Guidance.GuidanceLaneInfo
Windows.Services.Maps.Guidance.GuidanceManeuver
Windows.Services.Maps.Guidance.GuidanceMapMatchedCoordinate
Windows.Services.Maps.Guidance.GuidanceNavigator
Windows.Services.Maps.Guidance.GuidanceReroutedEventArgs
Windows.Services.Maps.Guidance.GuidanceRoadSegment
Windows.Services.Maps.Guidance.GuidanceRoadSignpost
Windows.Services.Maps.Guidance.GuidanceRoute
Windows.Services.Maps.Guidance.GuidanceTelemetryCollector
Windows.Services.Maps.Guidance.GuidanceUpdatedEventArgs
Windows.Services.Maps.LocalSearch.LocalCategories
Windows.Services.Maps.LocalSearch.LocalLocation
Windows.Services.Maps.LocalSearch.LocalLocationFinder
Windows.Services.Maps.LocalSearch.LocalLocationFinderResult
Windows.Services.Maps.LocalSearch.LocalLocationHoursOfOperationItem
Windows.Services.Maps.LocalSearch.LocalLocationRatingInfo
Windows.Services.Maps.MapAddress
Windows.Services.Maps.MapLocation
Windows.Services.Maps.MapLocationFinder
Windows.Services.Maps.MapLocationFinderResult
Windows.Services.Maps.MapManager
Windows.Services.Maps.MapRoute
Windows.Services.Maps.MapRouteDrivingOptions
Windows.Services.Maps.MapRouteFinder
Windows.Services.Maps.MapRouteFinderResult
Windows.Services.Maps.MapRouteLeg
Windows.Services.Maps.MapRouteManeuver
Windows.Services.Maps.MapService
Windows.Services.Store.StoreAcquireLicenseResult
Windows.Services.Store.StoreAppLicense
Windows.Services.Store.StoreAvailability
Windows.Services.Store.StoreCollectionData
Windows.Services.Store.StoreConsumableResult
Windows.Services.Store.StoreContext
Windows.Services.Store.StoreImage
Windows.Services.Store.StoreLicense

Windows.Services.Store.StorePackageLicense
Windows.Services.Store.StorePackageUpdate
Windows.Services.Store.StorePackageUpdateResult
Windows.Services.Store.StorePrice
Windows.Services.Store.StoreProduct
Windows.Services.Store.StoreProductPagedQueryResult
Windows.Services.Store.StoreProductQueryResult
Windows.Services.Store.StoreProductResult
Windows.Services.Store.StorePurchaseProperties
Windows.Services.Store.StorePurchaseResult
Windows.Services.Store.StoreRequestHelper
Windows.Services.Store.StoreSendRequestResult
Windows.Services.Store.StoreSku
Windows.Services.Store.StoreVideo
Windows.Storage.ApplicationDataSetVersionHandler
Windows.Storage.CachedFileManager
Windows.Storage.DownloadsFolder
Windows.Storage.FileIO
Windows.Storage.FileProperties.BasicProperties
Windows.Storage.FileProperties.DocumentProperties
Windows.Storage.FileProperties.ImageProperties
Windows.Storage.FileProperties.MusicProperties
Windows.Storage.FileProperties.StorageItemContentProperties
Windows.Storage.FileProperties.StorageItemThumbnail
Windows.Storage.FileProperties.VideoProperties
Windows.Storage.KnownFolders
Windows.Storage.PathIO
Windows.Storage.StorageFile
Windows.Storage.StorageFolder
Windows.Storage.StorageLibrary
Windows.Storage.StorageProvider
Windows.Storage.StorageStreamTransaction
Windows.Storage.StreamedFileDataRequest
Windows.Storage.StreamedFileDataRequestedHandler
Windows.Storage.Streams.Buffer
Windows.Storage.Streams.DataReader
Windows.Storage.Streams.DataReaderLoadOperation
Windows.Storage.Streams.DataWriter
Windows.Storage.Streams.DataWriterStoreOperation
Windows.Storage.Streams.FileInputStream
Windows.Storage.Streams.FileOutputStream
Windows.Storage.Streams.FileRandomAccessStream
Windows.Storage.Streams.InMemoryRandomAccessStream
Windows.Storage.Streams.InputStreamOverStream
Windows.Storage.Streams.OutputStreamOverStream
Windows.Storage.Streams.RandomAccessStream
Windows.Storage.Streams.RandomAccessStreamOverStream
Windows.Storage.Streams.RandomAccessStreamReference
Windows.Storage.SystemAudioProperties

Windows.Storage.SystemGPSProperties
Windows.Storage.SystemImageProperties
Windows.Storage.SystemMediaProperties
Windows.Storage.SystemMusicProperties
Windows.Storage.SystemPhotoProperties
Windows.Storage.SystemProperties
Windows.Storage.SystemVideoProperties
Windows.System.Diagnostics.ProcessCpuUsage
Windows.System.Diagnostics.ProcessCpuUsageReport
Windows.System.Diagnostics.ProcessDiagnosticInfo
Windows.System.Diagnostics.ProcessDiskUsage
Windows.System.Diagnostics.ProcessDiskUsageReport
Windows.System.Diagnostics.ProcessMemoryUsage
Windows.System.Diagnostics.ProcessMemoryUsageReport
Windows.System.Profile.AnalyticsInfo
Windows.System.Profile.AnalyticsVersionInfo
Windows.System.Threading.Core.PreallocatedWorkItem
Windows.System.Threading.Core.SignalHandler
Windows.System.Threading.Core.SignalNotifier
Windows.System.Threading.ThreadPool
Windows.System.Threading.ThreadPoolTimer
Windows.System.Threading.TimerDestroyedHandler
Windows.System.Threading.TimerElapsedHandler
Windows.System.Threading.WorkItemHandler
Windows.System.TimeZoneSettings
Windows.UI.Notifications.BadgeNotification
Windows.UI.Notifications.BadgeUpdateManager
Windows.UI.Notifications.BadgeUpdater
Windows.UI.Notifications.ScheduledTileNotification
Windows.UI.Notifications.ScheduledToastNotification
Windows.UI.Notifications.TileNotification
Windows.UI.Notifications.TileUpdateManager
Windows.UI.Notifications.TileUpdater
Windows.UI.Notifications.ToastNotificationHistory
Windows.UI.StartScreen.JumpList
Windows.UI.StartScreen.JumpListItem

모든 데스크톱 앱에서 지원되는 API

DualApiPartition 을 사용하는 API 는 패키지 ID 및 패키지 ID 가 없는 애플리케이션을 포함한 모든 데스크톱 애플리케이션에서 지원됩니다. 데스크톱 응용 프로그램에서 호출하려는 API 가 지원되는지 여부를 확인하려면 WinRT 네임스페이스에 대한 참조 문서에 관심 있는 클래스 또는 멤버에 대한 참조 문서를 찾으십시오. 그런 다음 참조 문서의 **Attributes** 섹션에 DualApiPartition 특성이 포함되어 있는지 확인합니다.

다음 단계

질문이 있으신가요? Stack Overflow 에서 Microsoft 에 질문하세요. 이 팀은 이 태그들을 지속 확인합니다. MSDN 포럼에서도 질문할 수 있습니다.

데스크톱 앱에서 WinRT XAML 컨트롤 호스팅(XAML Islands)

2/18/2021

Windows 10, 1903 버전부터는 XAML Islands 라는 기능을 사용하여 WinRT XAML 컨트롤을 UWP 가 아닌 데스크톱 응용 프로그램에서 호스팅할 수 있습니다. 이 기능을 사용하면 WinRT XAML 컨트롤을 통해서만 사용할 수 있는 최신 Windows 10 UI 기능을 통해 기존 WPF, Windows Forms 및 C++ Win32 응용 프로그램의 모양, 느낌 및 기능을 향상시킬 수 있습니다. 즉, Windows Ink 와 같은 UWP 기능과 기존 WPF, Windows Forms 및 C++ Win32 응용 프로그램에서 Fluent Design System 을 지원하는 컨트롤을 사용할 수 있습니다.

다음을 포함하여 Windows.UI.Xaml.UIElement 에서 파생되는 모든 WinRT XAML 컨트롤을 호스팅할 수 있습니다.

- Windows SDK 또는 WinUI 2.x 라이브러리에서 제공하는 모든 퍼스트 파티 WinRT XAML 컨트롤
- 사용자 지정 WinRT XAML 컨트롤(예: 함께 작동하는 여러 WinRT XAML 컨트롤로 구성된 사용자 컨트롤) 응용 프로그램으로 컴파일하려면 사용자 정의 컨트롤에 대한 소스 코드가 있어야 합니다.

기본적으로 XAML Islands 는 UWP XAML 호스팅 API 를 사용하여 생성됩니다. 이 API 는 Windows 10 버전 1903 SDK 에 도입된 여러 Windows Runtime 클래스와 COM 인터페이스로 구성되어 있습니다. 또한 UWP XAML 호스팅 API 를 내부적으로 사용하고 WPF 및 Windows Forms 앱에 보다 편리한 개발 환경을 제공하는 Windows 커뮤니티 툴킷에서 XAML Island .NET 컨트롤 세트를 제공합니다.

XAML Islands 를 사용하는 방법은 응용 프로그램 유형 및 호스팅하려는 WinRT XAML 컨트롤 유형에 따라 달라집니다.

> **NOTE**
> XAML Islands 에 대한 피드백이 있는 경우, Microsoft.Toolkit.Win32 리포지토리에 새 이슈를 만들고 의견을 남겨주십시오. 개인적으로 피드백을 제출하고 싶으시면 XamlIslandsFeedback@microsoft.com 으로 보내주셔도 됩니다. 당신의 통찰력과 시나리오는 저희에게 매우 중요합니다.

요구 사항

XAML Islands 은 다음을 만족해야 사용할 수 있습니다.

- Windows 10, 버전 1903 이상
- 응용 프로그램이 MSIX 패키지에 포함되어 있지 않은 경우 시스템에 Visual C++ Runtime 이 설치되어 있어야 합니다.

WPF 과 Windows Forms 앱

> **NOTE**
>
> XAML Islands 를 사용하여 WPF 및 Windows Forms 앱에서 WinRT XAML 컨트롤을 호스팅하는 것은 현재 .NET Core 3.x 를 대상으로 하는 앱에만 지원됩니다. 현재 .NET 5 나 .NET Framework 대상 앱에는 지원되지 않습니다.

WPF 및 Windows Forms 응용 프로그램은 Windows 커뮤니티 툴킷에서 사용할 수 있는 XAML Island .NET 컨트롤을 사용하는 것이 좋습니다. 이러한 컨트롤은 해당하는 WinRT XAML 컨트롤의 속성, 방법 및 이벤트를 모방하거나 액세스할 수 있는 개체 모델을 제공합니다. 또한 키보드 탐색 및 레이아웃 변경과 같은 동작도 처리합니다.

XAML Island 컨트롤은 wrapped, host 로 두 세트가 있습니다.

Wrapped 컨트롤

WPF 및 Windows Forms 응용 프로그램은 특정 WinRT XAML 컨트롤의 인터페이스와 기능을 래핑하는 XAML Island 컨트롤을 사용할 수 있습니다. 이러한 컨트롤을 WPF 또는 Windows Forms 프로젝트의 설계 표면에 직접 추가한 다음 디자이너의 다른 WPF 또는 Windows Forms 컨트롤처럼 사용할 수 있습니다.

The following wrapped WinRT XAML controls are currently available in the Windows Community Toolkit.

컨트롤	최소 지원 OS	설명 DESCRIPTION
InkCanvas InkToolbar	Windows 10, version 1903	Windows Forms 또는 WPF 데스크톱 응용 프로그램에서 Windows Ink 기반 사용자 상호 작용을 위한 표면 및 관련 도구 모음을 제공합니다.
MediaPlayerElement	Windows 10, version 1903	Windows Forms 또는 WPF 데스크톱 응용 프로그램에서 비디오와 같은 미디어 콘텐츠를 스트리밍하고 렌더링하는 보기를 포함합니다.
MapControl	Windows 10, version 1903	Windows Forms 또는 WPF 데스크톱 응용 프로그램에 심볼 맵 또는 사진 맵을 표시할 수 있습니다.

Wrapped WinRT XAML 컨트롤을 사용하는 방법을 보여주는 자세한 내용은 WPF 앱에서 표준 WinRT XAML 컨트롤 호스팅을 참조하십시오.

Host 컨트롤

사용자 지정 컨트롤 및 사용 가능한 wrapped 컨트롤이 적용되지 않는 다른 시나리오의 경우 WPF 및 Windows Forms 응용 프로그램은 Windows 커뮤니티 툴킷에서 사용할 수 있는 WindowsXamlHost 컨트롤을 사용할 수도 있습니다.

컨트롤	최소 지원 OS	설명
WindowsXamlHost	Windows 10, version 1903	Windows SDK 또는 WinUI 2.x 라이브러리에서 제공하는 모든 퍼스트 파티 WinRT XAML 컨트롤을 포함하여 Windows.UI.Xaml.UIElement 의 파생 컨트롤을 모두 호스팅할 수 있습니다.

WindowsXamlHost 컨트롤 사용법을 보여주는 가이드는 WPF 앱에서 표준 WinRT XAML 컨트롤 호스팅과 커스텀 WinRT XAML 컨트롤 호스팅 부분을 참고하세요.

프로젝트에서 XAML Island .NET 컨트롤 사용하도록 구성하기

XAML Island .NET 컨트롤에는 Windows 10, 1903 이상 버전이 필요합니다. 이러한 컨트롤을 사용하려면 아래에 나열된 NuGet 패키지 중 하나를 설치하십시오. 이러한 패키지는 XAML Island wrapped 컨트롤 및 host 컨트롤을 사용하는 데 필요한 모든 기능을 제공하며 필요한 다른 관련 NuGet 패키지도 포함합니다.

컨트롤 유형	NUGET 패키지	관련 글
Wrapped 컨트롤	다음 패키지의 6.0.0 이상 버전 WPF: Microsoft.Toolkit.Wpf.UI.Controls Windows Forms: Microsoft.Toolkit.Forms.UI.Controls	WPF 앱에서 표준 WinRT XAML 컨트롤 호스팅
Host 컨트롤	다음 패키지의 6.0.0 이상 버전 WPF: Microsoft.Toolkit.Wpf.UI.XamlHost Windows Forms: Microsoft.Toolkit.Forms.UI.XamlHost	WPF 앱에서 표준 WinRT XAML 컨트롤 호스팅 WPF 앱에서 커스텀 WinRT XAML 컨트롤 호스팅

다음 세부사항에 유의하십시오.

- Host 컨트롤 패키지도 wrapped 컨트롤 패키지에 포함됩니다. 두 가지 컨트롤 세트를 모두 사용하려면 wrapped 컨트롤 패키지를 설치하면 됩니다.
- 사용자 정의 WinRT XAML 컨트롤을 호스트하는 경우 사용자 정의 컨트롤을 참조하기 위한 몇 가지 추가 단계도 수행해야 합니다. 자세한 내용은 XAML Islands 를 사용하여 WPF 앱에서 커스텀 WinRT XAML 컨트롤 호스팅을 참조하십시오.

웹 뷰 컨트롤

Windows 커뮤니티 툴킷은 WPF 및 Windows Forms 응용 프로그램에서 웹 콘텐츠를 호스팅하기 위한 다음과 같은 .NET 컨트롤도 제공합니다. 이러한 컨트롤은 XAML Island 컨트롤과 유사한 데스크톱 앱 현대화 시나리오에서 자주 사용되며 XAML Island 제어와 동일한 리포지토리에서 유지됩니다.

컨트롤	최소 지원 OS	설명
WebView	Windows 10, version 1803	Microsoft Edge 렌더링 엔진을 사용하여 웹 콘텐츠를 표시합니다.
WebViewCompatible	Windows 7	추가 OS 버전과 호환되는 WebView 버전을 제공합니다. 이 컨트롤은 Microsoft Edge 렌더링 엔진을 사용하여 Windows 10 버전 1803 이상에서 웹 콘텐츠를 표시하고 Internet Explorer 렌더링 엔진을 사용하여 이전 버전의 Windows 10, Windows 8.x 및 Windows 7 에서 웹 콘텐츠를 표시합니다.

이 컨트롤들을 사용하려면 다음 NuGet 패키지 중 하나를 설치하십시오.

- WPF: Microsoft.Toolkit.Wpf.UI.Controls.WebView
- Windows Forms: Microsoft.Toolkit.Forms.UI.Controls.WebView

C++ Win32 앱

The XAML Island .NET 컨트롤은 C++ Win32 앱에서 지원되지 않습니다. 이 앱들은 그 대신 Windows 10 SDK(1903 버전 이상)에서 제공하는 UWP XAML 호스팅 API 를 사용해야 합니다.

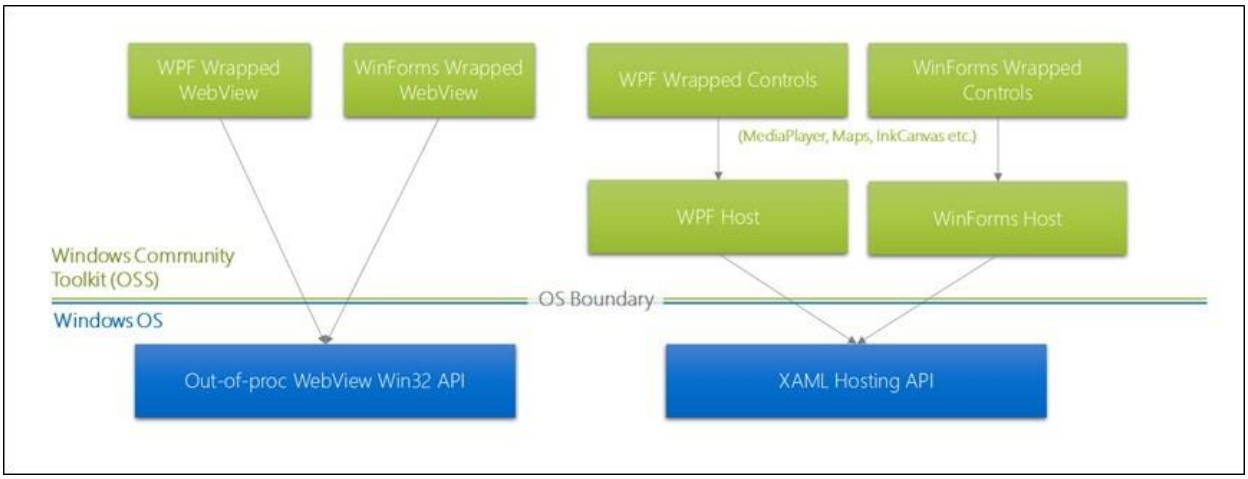
UWP XAML 호스팅 API 는 C++ Win32 응용 프로그램이 Windows.UI.Xaml.UIElement 에서 파생된 WinRT XAML 컨트롤을 호스팅하는 데 사용할 수 있는 여러 Windows Runtime 클래스와 COM 인터페이스로 구성됩니다. 연결된 Windows 핸들(HWND)이 있는 응용 프로그램의 모든 UI 요소에 WinRT XAML 컨트롤을 호스팅할 수 있습니다.

- C++ Win32 앱에서 UWP XAML 호스팅을 사용하기
- C++ Win32 앱에서 표준 WinRT XAML 컨트롤 호스팅
- C++ Win32 앱에서 커스텀 WinRT XAML 컨트롤 호스팅

> **NOTE**
>
> Windows 커뮤니티 툴킷의 wrapped, host 컨트롤은 내부적으로 UWP XAML 호스팅 API 를 사용하며 키보드 탐색 및 레이아웃 변경을 포함하여 UWP XAML 호스팅 API 를 직접 사용할 때 사용자가 직접 처리해야 하는 모든 동작을 구현합니다. WPF, Windows Forms 앱의 경우 API 사용에 대한 많은 구현 세부 정보를 추상화하므로 UWP XAML 호스팅 API 대신 이러한 컨트롤을 사용하는 것이 좋습니다.

XAML Islands 의 구조

UWP XAML 호스팅 API 위에 다양한 유형의 XAML Islands 컨트롤이 어떻게 구성되는지 간단히 살펴보겠습니다.



그림의 하단의 API 는 Windows SDK 와 함께 제공됩니다. Wrapped, host 컨트롤은 Windows 커뮤니티 툴킷의 NuGet 패키지를 통해 사용할 수 있습니다.

한계점과 대처법

다음은 XAML Islands 를 사용하는 데스크톱 앱의 특정 UWP 개발 시나리오에 대한 한계점과 그 해결 방법을 설명합니다.

대처법을 사용할 수 있는 경우

☒✔ WinUI 2.x 라이브러리의 컨트롤을 호스팅하는 것은 현재 릴리스에서 조건부로 지원됩니다. 만약 당신의 앱이 MSIX 패키지를 사용하여 배포된다면 WinUI 컨트롤은 Microsoft.UI.Xaml NuGet 패키지의 (사전)릴리스 버전에서 호스팅할 수 있습니다. 그렇지 않은 경우, 사전 릴리스 버전의 패키지를 설치한 경우에만 WinUI 컨트롤을 호스팅할 수 있습니다. WinUI 3.0 라이브러리의 컨트롤 호스팅은 이후 릴리스에서 제공될 예정입니다.
☒✔ XAML Island 에서 XAML 콘텐츠 트리의 루트 요소에 액세스하고 해당 콘텐츠가 호스팅되는 컨텍스트에 대한 관련 정보를 가져오려면 CoreWindow, ApplicationView 및 Window 클래스를 사용하지 마십시오. 대신 XamlRoot 클래스를 사용하십시오. 자세한 내용은 아래쪽을 참조하십시오.

☒✔ WPF, Windows Forms 또는 C++ Win32 앱에서 공유를 지원하려면 앱에서 IDataTransferManagerInterop 인터페이스를 사용하여 DataTransferManager 객체를 얻어 공유 작업을 특정 창에서 시작하도록 해야 합니다. WPF 앱에서 이 인터페이스를 사용하는 방법을 보여주는 샘플은 ShareSource 샘플을 참조하십시오.

☒✔ XAML Islands 에서 호스팅된 컨트롤과 함께 x:Bind 를 사용하는 것은 지원되지 않습니다. .NET 표준 라이브러리에서 데이터 모델을 선언해야 합니다.

미지원

☒ .NET Framework 를 대상으로 하는 WPF, Windows Forms 앱에 XAML Islands 를 사용할 수 없습니다. .NET Core 3.x. 대상 앱에서만 지원됩니다.

☒ XAML Islands 의 UWP XAML 콘텐츠는 Windows 의 어두움-밝음 테마 변동에 대응하지 않습니다. 고대비 변동에는 런타임에 대응합니다.

☒ **WebView** 컨트롤을 커스텀 사용자 컨트롤(thread, on-thread, off-thread, 프로세스 외부)에 추가할 수 없습니다.

☒ MediaPlayer 컨트롤과 MediaPlayerElement host 컨트롤은 전체화면에서 지원되지 않습니다.

☒ 필기 뷰를 이용한 문자 입력을 지원하지 않습니다. 더 자세한 정보는 여기를 읽어 보시면 됩니다.

☒ **@Places** 와 **@People** 콘텐츠 링크를 사용하는 텍스트 컨트롤은 지원되지 않습니다. 더 자세한 정보는 여기를 읽어 보시면 됩니다.

☒ XAML Islands 는 TextBox, RichEditBox, AutoSuggestBox 와 같은 문자 입력 컨트롤을 포함하는 ContentDialog 에 대한 호스팅을 지원하지 않습니다. 이렇게 하면 입력 컨트롤이 키 입력에 제대로 응답하지 않을 것입니다. XAML Islands 로 비슷한 기능을 이용하려면 입력 컨트롤이 포함된 Popup 을 호스트하는 것이 좋습니다.

☒ XAML Islands 는 현재 SVG 파일을 Windows.UI.Xaml.Controls.Image 또는 Windows.UI.Xaml.Media.Imaging.SvgImageSource 객체를 호스트하여 표시할 수 없습니다. 대신 표시하려는 이미지를 래스터 기반 포맷인 JPG, PNG 등으로 변환시키십시오.

XAML Islands 의 창 호스트 컨텍스트

데스크톱 앱에서 XAML Islands 를 호스트할 때 여러 트리의 XAML 콘텐츠가 동시에 동일한 스레드에서 실행될 수 있습니다. XAML Island 에서 XAML 콘텐츠 트리의 루트 요소에 액세스하고 해당 콘텐츠가 호스팅되는 컨텍스트에 대한 관련 정보를 가져오려면 Xamlroot 클래스를 사용합니다. CoreWindow, ApplicationView 및 Window 클래스는 XAML Islands 에 대한 올바른 정보를 제공하지 않습니다. CoreWindow 및 Window 객체는 스레드에 존재하며 앱에서 액세스할 수 있지만 의미 있는 경계 또는 가시성을 반환하지 않습니다(항상 보이지 않고 크기가 1x1). 자세한 내용은 호스트 윈도우 설정을 참조하십시오.
예를 들어 XAML Island 에서 호스팅되는 WinRT XAML 컨트롤을 포함하는 창의 경계 직사각형을 얻으려면 컨트롤의 XamlRoot.Size 속성을 사용합니다. XAML Island 에서 호스팅할 수 있는 모든 WinRT XAML 컨트롤이 Windows.UI.Xaml.UIElement 의 파생 객체이므로, 컨트롤의 XamlRoot 속성을 이용하여 **XamlRoot** 객체에 접근할 수 있습니다.

```
Size windowSize = myUWPControl.XamlRoot.Size;
```

경계 직사각형을 얻을 때 CoreWindows.Bounds 를 사용하면 안 됩니다.

```
// WinRT XAML 컨트롤에서 이렇게 하면 잘못된 정보를 리턴할 수 있습니다.
Rect windowSize = CoreWindow.GetForCurrentThread().Bounds;
```

XAML Islands 이용 시 피해야 하는 창 관련 API 와 대신 권장되는 XamlRoot 용법을 정리한 표는 이곳에서 보실 수 있습니다.

이 인터페이스를 WPF 앱에서 사용하는 샘플은 여기(ShareSource)를 참고하십시오.

추가 자료

XAML Islands 에 대하여 더 많은 배경 정보와 튜토리얼을 보려면 다음 자료를 참고하십시오.

- WPF 현대화하기 튜토리얼: 이 튜토리얼에서는 Windows Community Toolkit 의 줄바꿈된 컨트롤과 호스트 컨트롤을 사용하여 기존 WPF LOB(Line-of-Business) 애플리케이션에 WinRT XAML 컨트롤을 추가하는 단계별 지침을 제공합니다. 이 튜토리얼에는 WPF 응용 프로그램의 전체 코드와 프로세스의 각 단계에 대한 세부 지침이 포함되어 있습니다.
- XAML Islands 코드 샘플: 이 리포지토리에는 Windows Forms, WPF, 그리고 C++/Win32 에서 각각 어떻게 XAML Islands 를 사용하는지 보여주는 샘플이 있습니다.
- XAML Islands v1 – 업데이트와 로드맵: 이 블로그 포스트는 XAML Islands 에 관한 많은 공통 질문과 자체한 개발 로드맵을 제공합니다.

XAML Islands 를 이용하여 WPF 앱에 표준 WinRT XAML 컨트롤 호스팅

2/18/2021

이 자료에서는 XAML Islands 를 사용하여 .NET Core 3.1 을 대상으로 하는 WPF 앱에서 표준 WinRT XAML 컨트롤(Windows SDK 제공 퍼스트 파티 WinRT XAML 컨트롤)을 호스팅하는 두 가지 방법을 보여줍니다.

- Windows 커뮤니티 툴킷에서 wrapped 컨트롤을 사용하여 UWP InkCanvas 와 InkToolbar 컨트롤을 호스팅하는 방법을 보여줍니다. 이러한 컨트롤들은 유용한 WinRT XAML 컨트롤의 인터페이스와 기능을 몇 개 포함합니다. WPF 또는 Windows Forms 프로젝트의 설계 표면에 직접 넣어 디자이너의 다른 WPF 나 Windows Forms 컨트롤을 사용하는 것처럼 사용할 수 있습니다.
- 또한 Windows 커뮤니티 툴킷에서 WindowsXamlHost 컨트롤을 사용하여 UWP CalendarView 컨트롤을 호스팅하는 방법도 보여줍니다. 일부 WinRT XAML 컨트롤만 wrapped 컨트롤로 사용할 수 있으므로, WindowsXamlHost 를 사용하여 다른 표준 WinRT XAML 컨트롤을 호스팅할 수 있습니다.

이 자료는 WPF 앱에 대하여 WinRT XAML 컨트롤을 호스팅하는 방법을 보여주는 것이지만, Windows Forms 앱에 대해서도 과정은 비슷합니다.

> NOTE
>
> XAML Islands 를 이용해 WinRT XAML 컨트롤을 WPF, Windows Forms 에 호스팅하는 것은 현재 .NET Core 3.x 대상의 앱만 지원됩니다. .NET 5 나 .NET Framework 에 대해서는 아직 지원되지 않습니다.

필요한 요소

WinRT XAML 컨트롤을 WPF 및 Windows Forms 앱에 호스팅하기 위해서는 솔루션에 다음 요소가 필요합니다. 이 자료는 그것들을 준비하는 과정도 안내합니다.

- **프로젝트/소스 코드.** WinRT XAML 컨트롤을 호스트하기 위해 WindowsXamlHost 컨트롤을 사용하는 것은 현재 .NET Core 3.x 대상의 앱에서만 지원됩니다.
- **루트 애플리케이션 클래스가 XamlApplication 파생형으로 정의된 UWP 앱 프로젝트.** 당신의 WPF 및 Windows Forms 프로젝트는 Windows 커뮤니티 툴킷에서 제공되는 Microsoft.Toolkit.Win32.-UI.XamlHost.XamlApplication 의 인스턴스에 접근하여 커스텀 UWP XAML 컨트롤을 발견하고 로드할 수 있어야 합니다. 권장되는 방법은 이 객체를 다른 UWP 앱 프로젝트에서 정의하고 당신의 WPF 및 Windows Forms 앱의 솔루션에 포함시키는 것입니다.

> NOTE
> **XamlApplication** 객체는 퍼스트파티 WinRT XAML 컨트롤을 호스팅하는 데 반드시 필요한 것은 아니지만, 커스텀 WinRT XAML 컨트롤 호스팅을 포함하여 전 범위의 XAML Island 시나리오를 지원하기 위해서 필요합니다. 그렇기에 본 자료는 XAML Islands 를 사용하는 모든 솔루션에 **XamlApplication** 객체를 정의하는 것을 권장합니다.
>
> NOTE
> 솔루션은 **XamlApplication** 객체를 정의하는 프로젝트를 한 개만 포함할 수 있습니다. **XamlApplication** 객체를 정의하는 프로젝트는 반드시 XAML Island 로 WinRT XAML 컨트롤을 호스팅하기 위해서 사용된 모든 다른 라이브러리와 프로젝트에 대한 참조를 포함해야 합니다.

WPF 프로젝트 생성

시작하기 전에 다음 안내를 따라 WPF 프로젝트를 만들어 XAML Islands 를 호스팅하도록 구성합니다. WPF 프로젝트를 이미 만들어 두었다면, 프로젝트에 다음 단계와 코드 예시를 적용할 수 있습니다.

1. Visual Studio 2019 에서 새 WPF 앱 프로젝트(.NET Core)를 생성합니다. 먼저 .NET Core 3.1 SDK 가 설치되어 있어야 합니다.
2. 패키지 참조가 활성화되어 있어야 합니다.
   a. Visual Studio 에서 **도구 -> NuGet 패키지 관리자 -> 패키지 관리자 설정**을 클릭합니다.

b. PackageReference 가 **기본 패키지 관리 포맷**으로 선택되어야 합니다.

3. **솔루션 탐색기**의 WPF 프로젝트를 우클릭하고 **NuGet 패키지 관리**를 선택합니다.

4. 찾아보기를 선택 후 Microsoft.Toolkit.Wpf.UI.Controls 패키지를 찾아 최신 안정 버전을 설치합니다. 이 패키지는 WPF 를 위해 필요한 wrapped WinRT XAML 컨트롤을 모두 제공합니다(InkCanvas, InkToolbar, WindowsXamlHost 포함).

> **NOTE**
>
> Windows Forms 앱은 반드시 Microsoft.Toolkit.Forms.UI.Controls 패키지를 사용해야 합니다.

5. 당신의 솔루션이 x86 이나 x64 와 같은 특정 플랫폼을 대상으로 하도록 구성합니다. 대부분의 XAML Islands 시나리오는 모든 CPU 를 대상으로 하는 것을 지원하지는 않습니다.

   a. 솔루션 탐색기에서 솔루션 노드를 우클릭하고 속성 -> 구성 속성 -> 구성 관리자를 선택합니다.

   b. 솔루션 플랫폼 활성화 아래의 New 를 선택합니다.

   c. 새 솔루션 플랫폼 대화상자에서 x64 또는 x86 를 선택 후 확인을 누릅니다.

   d. 열었던 대화상자를 닫습니다.

UWP 앱 프로젝트에서 XamlApplication 클래스를 정의하기

이제, 솔루션에 UWP 앱 프로젝트를 추가하고 그 프로젝트의 기본 App 클래스를 Windows 커뮤니티 툴킷에서 제공하는 Microsoft.Toolkit.Win32.UI.XamlHost.XamlApplication 을 상속하도록 수정합니다. 이 클래스는 앱이 런타임에 응용 프로그램의 현재 디렉토리에서 커스텀 UWP XAML 컨트롤의 메타데이터를 어셈블리로 발견하고 로드할 수 있게 하는 IXamlMetadataProvider 인터페이스를 지원합니다. 이 클래스는 또한 현재 스레드에서 UWP XAML 프레임워크를 초기화합니다.

> **NOTE**
> 이 단계가 퍼스트파티 WinRT XAML 컨트롤 호스팅에는 필요하지 않지만, 앱에서 커스텀 WinRT XAML 컨트롤 호스팅을 포함하여 전 범위의 XAML Island 시나리오를 지원하기 위해서는 **XamlApplication** 객체가 필요합니다. 그렇기에 본 자료는 XAML Islands 를 사용하는 모든 솔루션에 **XamlApplication** 객체를 정의하는 것을 권장합니다.

1. **솔루션 탐색기**에서 솔루션 노드를 우클릭하고 **새로 만들기->새 프로젝트**를 선택합니다.

2. 빈 앱(범용 Windows) 프로젝트를 추가합니다. 대상 버전과 최소 버전이 모두 Windows 10 1903(빌드 18362) 이상 버전으로 설정되어야 합니다.

3. UWP 앱 프로젝트에서 Microsoft.Toolkit.Win32.UI.XamlApplication NuGet 패키지의 최신 안정 버전을 설치합니다.

4. **App.xaml** 파일을 열고 다음 XAML 이 있는 파일에서 **MyUWPApp** 을 당신의 UWP 앱 프로젝트의 네임스페이스로 수정합니다.

```
<xaml:XamlApplication    x:Class="MyUWPApp.App"
   xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"    xmlns:xaml="using:Microsoft.Toolkit.Win32.UI.XamlHost"
xmlns:local="using:MyUWPApp">
</xaml:XamlApplication>
```

5. **App.xaml.cs** 파일을 열고 다음 내용의 **MyUWPApp** 을 당신의 UWP 앱 프로젝트의 네임스페이스로 수정합니다.

```
namespace MyUWPApp
{
   public sealed partial class App : Microsoft.Toolkit.Win32.UI.XamlHost.XamlApplication {
      public App() {
            this.Initialize();
      }
   }
}
```

6. UWP 앱 프로젝트에서 **MainPage.xaml** 파일을 제거합니다.

7. UWP 앱 프로젝트를 빌드합니다.

UWP 프로젝트에 참조를 추가하기

1. WPF 프로젝트 파일에 호환 프레임워크 버전을 명시합니다.

   a. **솔루션 탐색기**에서 WPF 프로젝트 노드를 더블클릭하여 프로젝트 파일을 에디터로 엽니다.

   b. 첫 **PropertyGroup** 내부에 다음을 추가합니다. **19041** 부분을 UWP 프로젝트의 대상/최소 OS 빌드에 맞춥니다.

```
<AssetTargetFallback>uap10.0.19041</AssetTargetFallback>
```

   완료하셨다면 **PropertyGroup** 은 다음과 같은 모습이 될 것입니다.

```
<PropertyGroup>
  <OutputType>WinExe</OutputType>
  <TargetFramework>netcoreapp3.1</TargetFramework>
  <UseWPF>true</UseWPF>
  <Platforms>AnyCPU;x64</Platforms>
  <AssetTargetFallback>uap10.0.19041</AssetTargetFallback> </PropertyGroup>
```

2. **솔루션 탐색기**에서 WPF 프로젝트 아래에 있는 **의존성** 노드를 우클릭하고, UWP 앱 프로젝트에 참조를 추가합니다.

WPF 앱의 시작점에 XamlApplication 객체를 생성하도록 하기

그 다음, WPF 앱의 시작점에 UWP 프로젝트에서 (**XamlApplication** 을 상속하도록)정의한 **App** 클래스의 인스턴스를 생성하도록 코드를 추가합니다.

1. WPF 프로젝트에서 프로젝트 노드를 우클릭하고 **추가->새 항목**을 선택하고, **클래스**를 선택합니다. 클래스 이름을 **Program** 으로 하고 **추가**를 선택합니다.

2. 생성된 **Program** 클래스의 코드를 다음과 같이 바꾸고 저장합니다. **MyUWPApp** 을 당신의 UWP 앱 프로젝트의 네임스페이스로 바꾸고 **MyWPFApp** 을 당신의 WPF 앱 프로젝트의 네임스페이스로 바꿉니다.

```
public class Program
{
  [System.STAThreadAttribute()]    public static void Main()
  {
    using (new MyUWPApp.App())
    {
      MyWPFApp.App app = new MyWPFApp.App();
      app.InitializeComponent();
       app.Run();
    }
  }
}
```

3. 프로젝트 노드를 우클릭하고 **속성**을 선택합니다.

4. **애플리케이션** 탭에서 **스타트업 객체** 드롭다운을 클릭하고 방금 추가한 Program 클래스의 허용된 이름을 선택합니다.

   > **NOTE**
   > 기본적으로 WPF 프로젝트는 수정되지 않도록 의도된 **Main** 이라는 시작점 함수를 정의합니다. 이 단계를 수행하면 그 시작점을 Program 클래스의 Main 함수로 변경하여, 시작하는 시점에 원하는 작업을 하도록 코드를 추가할 수 있습니다.

5. 변동 내용을 저장합니다.

Wrapped 컨트롤을 사용하여 InkCanvas 와 InkToolbar 를 호스팅

프로젝트가 UWP XAML Islands 를 사용하도록 구성하였으므로, 이제 wrapped WinRT XAML 컨트롤인 InkCanvas 와 InkToolbar 를 추가할 준비가 되었습니다.

1. WPF 프로젝트에서 **MainWindow.xaml** 파일을 엽니다.

2. XAML 파일의 상단부의 **Window** 에 다음 속성을 추가합니다. 이것은 InkCanvas 와 InkToolbar 컨트롤의 XAML 네임스페이스를 참조하도록 합니다.

```
xmlns:Controls="clrnamespace:Microsoft.Toolkit.Wpf.UI.Controls;assembly=Microsoft.Toolkit.Wpf.UI.Controls"
```

   이 속성을 추가하였다면 **Window** 는 다음과 같은 모양이 됩니다.

```
<Window xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x=http://schemas.microsoft.com/winfx/2006/xaml
xmlns:d=http://schemas.microsoft.com/expression/blend/2008
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"xmlns:local="clr-namespace:WpfApp"
xmlns:Controls="clr-
namespace:Microsoft.Toolkit.Wpf.UI.Controls;assembly=Microsoft.Toolkit.Wpf.UI.Controls"
x:Class="WpfApp.MainWindow"
mc:Ignorable="d"
Title="MainWindow" Height="800" Width="800">
```

3. **MainWindow.xaml** 파일에 **Grid** 를 다음과 같이 수정합니다. 다음 XAML 은 **Grid** 에 InkCanvas 와 InkToolbar 를 추가합니다(접두사는 앞에서 네임스페이스로 정의한 **Controls** 키워드).

```
<Grid Margin="10,50,10,10">
  <Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="Auto"/>
  </Grid.RowDefinitions>
  <Controls:InkToolbar x:Name="myInkToolbar" TargetInkCanvas="{x:Reference myInkCanvas}" Grid.Row="0" Width="300"
    Height="50"  Margin="10,10,10,10"  HorizontalAlignment="Left"  VerticalAlignment="Top"  />        <Controls:InkCanvas  x:Name="myInkCanvas"
Grid.Row="1" HorizontalAlignment="Left" Width="600" Height="400"
    Margin="10,10,10,10" VerticalAlignment="Top" /> </Grid>
```

> **NOTE**
> 이것들을 비롯한 다른 컨트롤은 **도구 상자**의 **Windows 커뮤니티 툴킷**에서 디자이너로 드래그하여 추가할 수도 있습니다.

4. **MainWindow.xaml** 파일을 저장합니다.

   Surface 와 같은 디지털 펜 지원 장치가 있고 실제 컴퓨터에서 이 랩을 실행하는 경우, 이제 앱을 빌드하고 실행하면 그 펜으로 화면에 그릴 수 있습니다. 그러나 해당 장치가 없고 마우스로 입력하려 하면 그려지지 않습니다. 이는 **InkCanvas** 툴이 기본적으로 디지털 펜에만 사용 가능하기 때문입니다. 그러나 이 동작은 변경할 수 있습니다.

5. **MainWindow.xaml.cs** 파일을 엽니다.

6. 다음 네임스페이스 using 선언을 파일 최상단에 추가합니다.

```
using Microsoft.Toolkit.Win32.UI.Controls.Interop.WinRT;
```

7. MainWindow() 생성자를 찾아, 다음 코드를 InitializeComponent() 메소드 다음에 추가하고 코드 파일을 저장합니다.

```
this.myInkCanvas.InkPresenter.InputDeviceTypes = CoreInputDeviceTypes.Mouse | CoreInputDeviceTypes.Pen;
```

   InkPresenter 객체를 이용하여 기본 잉크 기록 UX 를 조정할 수 있습니다. 이 코드는 InputDeviceTypes 속성을 바꾸어 펜 입력에 추가로 마우스 입력도 유효하게 만듭니다.

8. F5 를 다시 눌러 앱을 다시 빌드하고 디버거로 앱을 실행합니다. 마우스가 연결된 컴퓨터를 사용한다면, 잉크 캔버스 공간에 마우스로 그릴 수 있게 된 것을 확인하십시오.

Host 컨트롤을 사용하여 CalendarView 를 호스트

InkCanvas 와 InkToolbar wrapped WinRT XAML 컨트롤을 앱에 추가하였고, 이제 WindowsXamlHost 컨트롤을 사용하여 CalendarView 를 추가할 준비가 되었습니다.

> **NOTE**
> WindowsXamlHost 컨트롤을 Microsoft.Toolkit.Wpf.UI.XamlHost 패키지에서 제공됩니다. 이 패키지는 앞서 설치한 Microsoft.Toolkit.Wpf.UI.Controls 패키지에 포함되어 있습니다.

1. **솔루션 탐색기**에서 **MainWindow.xaml** 파일을 엽니다.

2. 파일 상단의 Window 에 다음 속성을 추가합니다. 이것은 WindowsXamlHost 컨트롤의 XAML 네임스페이스를 참조하도록 합니다.

```
xmlns:xamlhost="clrnamespace:Microsoft.Toolkit.Wpf.UI.XamlHost;assembly=Microsoft.Toolkit.Wpf.UI.XamlHost"
```

   이 속성을 추가한 후의 Window 는 다음과 같은 모양이 됩니다.

```
<Window xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"        xmlns:local="clr-namespace:WpfApp"
xmlns:Controls="clr-
namespace:Microsoft.Toolkit.Wpf.UI.Controls;assembly=Microsoft.Toolkit.Wpf.UI.Controls"        xmlns:xamlhost="clr-
namespace:Microsoft.Toolkit.Wpf.UI.XamlHost;assembly=Microsoft.Toolkit.Wpf.UI.XamlHost"        x:Class="WpfApp.MainWindow"
mc:Ignorable="d"
        Title="MainWindow" Height="800" Width="800">
```

3. **MainWindow.xaml** 파일에서 **Grid** 부분을 다음 XAML 로 수정합니다. 이 XAML 은 그리드에 행을 추가하고 WindowsXamlHost 객체를 마지막 행에 추가합니다. CalendarView 컨트롤을 호스트하기 위해, 이 XAML 은 **InitialTypeName** 속성을 컨트롤의 허용된 이름으로 설정합니다. 또한 호스트된 컨트롤이 화면에 나오면 호출되는 **ChildChanged** 이벤트 핸들러를 정의합니다.

```
<Grid Margin="10,50,10,10">
   <Grid.RowDefinitions>
      <RowDefinition Height="Auto"/>
      <RowDefinition Height="Auto"/>
      <RowDefinition Height="Auto"/>
   </Grid.RowDefinitions>
   <Controls:InkToolbar x:Name="myInkToolbar" TargetInkCanvas="{x:Reference myInkCanvas}" Grid.Row="0" Width="300"
      Height="50"   Margin="10,10,10,10"   HorizontalAlignment="Left"   VerticalAlignment="Top"   />            <Controls:InkCanvas   x:Name="myInkCanvas"
Grid.Row="1" HorizontalAlignment="Left" Width="600" Height="400"
      Margin="10,10,10,10" VerticalAlignment="Top" />
   <xamlhost:WindowsXamlHost x:Name="myCalendar"
InitialTypeName="Windows.UI.Xaml.Controls.CalendarView" Grid.Row="2"
         Margin="10,10,10,10" Width="600" Height="300" ChildChanged="MyCalendar_ChildChanged"  /> </Grid>
```

4. **MainWindow.xaml** 파일을 저장하고 **MainWindow.xaml.cs** 파일을 엽니다.

5. 다음 네임스페이스 선언문을 파일 최상단에 추가합니다.

```
using Microsoft.Toolkit.Wpf.UI.XamlHost;
```

6. 다음 **ChildChanged** 이벤트 핸들러 함수를 **MainWindow** 클래스에 추가하고 코드를 저장합니다. 호스트 컨트롤이 화면에 나오면, 이 이벤트 핸들러가 실행되어 달력 컨트롤의 **SelectedDatesChanged** 이벤트 핸들러를 생성합니다.

```
private void MyCalendar_ChildChanged(object sender, EventArgs e) {
    WindowsXamlHost windowsXamlHost = (WindowsXamlHost)sender;

    Windows.UI.Xaml.Controls.CalendarView calendarView =
        (Windows.UI.Xaml.Controls.CalendarView)windowsXamlHost.Child;

    if (calendarView != null)
    {
        calendarView.SelectedDatesChanged += (obj, args) =>
        {
            if (args.AddedDates.Count > 0)
            {
                MessageBox.Show("The user selected a new date: " +
args.AddedDates[0].DateTime.ToString());          }
        };
    }
}
```

7. F5 를 다시 눌러 다시 빌드하고 디버거로 앱을 실행합니다. 달력 컨트롤이 이제 창 하단에 보여지는 것을 확인할 수 있습니다.

앱을 패키지로 만들기

원하는 경우 WPF 앱을 MSIX 패키지로 만들어서 배포할 수 있습니다. MSIX 는 Windows 의 현대적 앱 패키징 기술이며 MSI, .appx, App-V, ClickOnce 설치 기술의 결합을 기반으로 만들어졌습니다.

다음 안내는 솔루션의 모든 요소를 Visual Studio 의 Windows 애플리케이션 패키징 프로젝트 기능으로 MSIX 패키지로 묶는 방법을 보여줍니다. 다음 과정은 WPF 앱을 MSIX 패키지로 만들고 싶은 경우에만 수행하시면 됩니다.

> **NOTE**
> MSIX 패키지로 묶지 않을 것이라면, 해당 앱을 실행하는 컴퓨터는 Visual C++ 런타임이 설치되어 있어야 합니다.

1. 솔루션에 새로 Windows 애플리케이션 패키징 프로젝트를 추가합니다. 프로젝트를 생성하고, UWP 프로젝트에서 선택한 대상/최소 버전과 같도록 해당 값을 선택합니다.

2. 패키징 프로젝트에서 **애플리케이션** 노드를 우클릭하여 **참조 추가**를 선택합니다. 프로젝트 리스트에서 WPF 프로젝트를 선택하고 확인을 누릅니다.

> **NOTE**
> Microsoft 스토어에 앱을 출시하고 싶다면, UWP 프로젝트의 참조도 추가해야 합니다.

3. x86 또는 x64 와 같이 특정 플랫폼을 대상으로 하도록 솔루션을 구성합니다. Windows 애플리케이션 패키징 프로젝트를 사용하여 WPF 앱을 MSIX 패키지로 빌드하려면 이렇게 해야 합니다.

   a. **솔루션 탐색기**에서 솔루션 노드를 우클릭하고 **속성->속성 구성->구성 관리자**를 선택합니다.

   b. **활성 솔루션 플랫폼** 아래에서 x64 또는 x86 을 선택합니다.

   c. WPF 프로젝트의 행, **플랫폼** 열에 **새로 만들기**를 선택합니다.

   d. 새 솔루션 플랫폼 대화상자에서 x64 또는 x86 중 방금 선택한 것을 선택하고 확인을 클릭합니다.

   e. 대화상자들을 닫습니다.

4. 패키징 프로젝트를 빌드하고 실행합니다. WPF 가 실행되고 UWP 컨트롤이 정한 대로 나오는 것을 확인합니다.

관련 주제

데스크톱 앱에서 UWP XAML 컨트롤을 호스팅하기(XAML Islands)
XAML Islands 코드 샘플
InkCanvas
InkToolbar
WindowsXamlHost

XAML Islands 를 이용하여 WPF 앱에 커스텀 WinRT XAML 컨트롤 호스팅

2/18/2021

이 자료는 Windows 커뮤니티 툴킷의 WindowsXamlHost 컨트롤을 사용하여 커스텀 WinRT XAML 컨트롤을 .NET Core 3.1 대상의 WPF 앱에서 호스팅하는 방법으로 보여줍니다. 커스텀 컨트롤은 Windows SDK 의 퍼스트파티 컨트롤을 다수 포함하며, WinRT XAML 컨트롤 중 하나에 있는 속성이 WPF 앱의 문자열에 바인딩되어 있습니다. 이 자료에서는 WinUI 라이브러리에서 컨트롤을 호스트하는 방법도 설명합니다.

이 자료는 WPF 앱에 대한 방법을 보여주나, Windows Forms 앱에 대해서도 비슷한 과정을 거칩니다. WinRT XAML 컨트롤을 WPF 및 Windows Forms 앱에 호스팅하는 것에 대한 개요는 이 자료의 해당 문단을 참고해 주세요.

> NOTE
> XAML Islands 를 이용해 WinRT XAML 컨트롤을 WPF, Windows Forms 에 호스팅하는 것은 현재 .NET Core 3.x 대상의 앱만 지원됩니다. .NET 5 나 .NET Framework 에 대해서는 아직 지원되지 않습니다.

필요한 요소

WPF(또는 Windows Forms) 앱에서 WinRT XAML 컨트롤을 호스트하려면 솔루션에 다음 요소가 포함되어 있어야 합니다. 아래에서 그것들을 생성하는 방법을 보여줍니다.

- **앱의 프로젝트와 소스 코드.** 커스텀 컨트롤을 호스팅하고자 WindowsXamlHost 컨트롤을 사용하는 것은 .NET Core 3.x 대상의 앱에만 지원됩니다.
- **커스텀 WinRT XAML 컨트롤.** 호스팅하려는 컨트롤의 컴파일할 소스코드가 필요합니다. 일반적으로 커스텀 컨트롤은 UWP 클래스 라이브러리 프로젝트로 정의되어 동일 솔루션에 있는 WPF(또는 Windows Forms) 프로젝트를 참조하도록 합니다.
- **XamlApplication 을 상속하는 루트 애플리케이션 클래스를 정의하는 UWP 앱 프로젝트.** WPF 또는 Windows Forms 프로젝트는 Windows 커뮤니티 툴킷에서 제공되는 Microsoft.Toolkit.Win32.UI.-XamlHost.XamlApplication 클래스에 접근하여 커스텀 UWP XAML 컨트롤을 발견하고 로드할 수 있어야 합니다. 권장되는 방법은 솔루션에서 UWP 앱 프로젝트를 새로 만들어 해당 객체를 정의하는 것입니다.

> NOTE
> 솔루션은 **XamlApplication** 객체를 정의하는 프로젝트를 한 개만 포함할 수 있습니다. 앱의 모든 커스텀 WinRT XAML 컨트롤은 동일한 **XamlApplication** 객체를 공유합니다. **XamlApplication** 객체를 정의하는 프로젝트는 반드시 XAML Islands 에서 컨트롤을 호스팅하기 위한 모든 WinRT 라이브러리 및 프로젝트에 대한 참조를 포함해야 합니다.

WPF 프로젝트

시작하기 전에 다음 안내를 따라 WPF 프로젝트를 만들고 XAML Islands 를 호스팅하도록 구성합니다. 이미 있는 WPF 프로젝트에 대하여 다음을 적용해도 됩니다.

> NOTE
> .NET Framework 대상의 기존 프로젝트는 .NET Core 3.1 로 마이그레이션해야 합니다. 더 많은 정보는 이 블로그 시리즈를 참고하십시오.

1. .NET Core 3.1 SDK 의 최신 버전이 설치되어 있어야 합니다.
2. Visual Studio 2019 에서 WPF 앱(.NET Core) 프로젝트를 생성합니다

3. 패키지 참조가 활성화되어 있어야 합니다.

   a. Visual Studio 에서 **도구->NuGet 패키지 관리자->패키지 관리자 설정**에 들어갑니다.

   b. **기본 패키지 관리 포맷**으로 PackageReference 가 선택되어야 합니다.

4. **솔루션 탐색기**의 WPF 프로젝트를 우클릭하고 **NuGet 패키지 관리**를 선택합니다.

5. 찾아보기 탭에서 Microsoft.Toolkit.Wpf.UI.XamlHost 패키지를 찾아 최신 안정 버전을 설치합니다. 이 패키지는 **WindowsXamlHost** 컨트롤을 이용하여 WinRT XAML 컨트롤을 호스팅하기 위해 필요한 모든 것 외에 추가로 관련된 NuGet 패키지도 포함합니다.

> NOTE
> Windows Forms 앱은 Microsoft.Toolkit.Forms.UI.XamlHost 패키지를 사용해야 합니다.

6. 솔루션이 x86 또는 x64 와 같이 특정 플랫폼을 대상으로 하도록 구성합니다. 커스텀 WinRT XAML 컨트롤은 모든 CPU 를 대상으로 하는 프로젝트에서는 지원되지 않습니다.

   a. 솔루션 탐색기에서 솔루션 노드를 우클릭하여 속성->구성 속성->구성 관리자를 선택합니다.

   b. **활성 솔루션 플랫폼** 아래의 **새로 만들기**를 선택합니다.

   c. 새 솔루션 플랫폼 대화상자에서 x64 또는 x86 을 선택하고 확인을 누릅니다.

   d. 대화 상자를 종료합니다.

UWP 앱 프로젝트에서 XamlApplication 클래스 정의하기

솔루션에 UWP 앱 프로젝트를 추가하고 기본 App 클래스가 Windows 커뮤니티 툴킷에서 제공하는 Microsoft.Toolkit.Win32.UI.XamlHost.XamlApplication 클래스를 상속하도록 수정합니다. 이 클래스는 앱이 런타임에 응용 프로그램의 현재 디렉토리에서 커스텀 UWP XAML 컨트롤의 메타데이터를 어셈블리로 발견하고 로드할 수 있게 하는 IXamlMetadataProvider 인터페이스를 지원합니다. 이 클래스는 또한 현재 스레드에서 UWP XAML 프레임워크를 초기화합니다.

1. **솔루션 탐색기**에서 솔루션 노드를 우클릭하고 **새로 만들기->프로젝트**를 선택합니다.

2. **빈 프로젝트(범용 Windows)**를 추가합니다. 대상 버전 및 최소 버전이 모두 Windows 10 1903(빌드 18362)이상 버전이어야 합니다.

3. UWP 앱 프로젝트에서 Microsoft.Toolkit.Win32.UI.XamlApplication NuGet 패키지의 최신 안정 버전을 설치합니다.

4. **App.xaml** 파일을 열고 다음 XAML 을 찾아 **MyUWPApp** 을 당신의 UWP 앱 프로젝트의 네임스페이스로 수정합니다.

```
<xaml:XamlApplication    x:Class="MyUWPApp.App"
   xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"    xmlns:xaml="using:Microsoft.Toolkit.Win32.UI.XamlHost"
xmlns:local="using:MyUWPApp">
</xaml:XamlApplication>
```

5. **App.xaml.cs** 파일을 열어 내용을 다음 코드와 같이 수정합니다. **MyUWPApp** 은 방금 고친 것처럼 UWP 앱 프로젝트 네임스페이스로 바꿉니다.

```
namespace MyUWPApp {
  public sealed partial class App : Microsoft.Toolkit.Win32.UI.XamlHost.XamlApplication
  {
    public App()
    {
      this.Initialize();
    }
  }
}
```

6. UWP 앱 프로젝트에서 **MainPage.xaml** 파일을 삭제합니다.

7. UWP 앱 프로젝트를 정리하고 빌드합니다.

WPF 프로젝트에 UWP 프로젝트 참조 추가하기

1. WPF 프로젝트 파일의 호환 프레임워크 버전을 명시합니다.

   a. **솔루션 탐색기**에서 WPF 프로젝트 노드를 더블클릭하고 프로젝트 파일을 에디터로 엽니다.

   b. 첫 **PropertyGroup** 내부에 다음 줄을 추가합니다. **19041** 부분은 UWP 에서 설정한 대상/최소 OS 빌드 버전과 같게 맞춥니다.

```
<AssetTargetFallback>uap10.0.19041</AssetTargetFallback>
```

   이것을 했다면 PropertyGroup 부분은 이런 모양이 됩니다.

```
<PropertyGroup>
  <OutputType>WinExe</OutputType>
  <TargetFramework>netcoreapp3.1</TargetFramework>
  <UseWPF>true</UseWPF>
  <Platforms>AnyCPU;x64</Platforms>
  <AssetTargetFallback>uap10.0.19041</AssetTargetFallback> </PropertyGroup>
```

2. **솔루션 탐색기**에서 **의존성 노드**를 우클릭하고 UWP 앱 프로젝트에 대한 참조를 추가합니다.

WPF 앱의 시작점에 XamlApplication 객체를 생성하도록 하기

그 다음, WPF 앱의 시작점에 UWP 프로젝트에서 (**XamlApplication** 을 상속하도록) 정의한 **App** 클래스의 인스턴스를 생성하도록 코드를 추가합니다.

1. WPF 프로젝트에서 프로젝트 노드를 우클릭하고 **추가->새 항목**을 선택하고, **클래스**를 선택합니다. 클래스 이름을 **Program** 으로 하고 **추가**를 선택합니다.

2. 생성된 **Program** 클래스의 코드를 다음과 같이 바꾸고 저장합니다. **MyUWPApp** 을 당신의 UWP 앱 프로젝트의 네임스페이스로 바꾸고 **MyWPFApp** 을 당신의 WPF 앱 프로젝트의 네임스페이스로 바꿉니다.

```
public class Program
{
  [System.STAThreadAttribute()]    public static void Main()
  {
    using (new MyUWPApp.App())
    {
      MyWPFApp.App app = new MyWPFApp.App();
        app.InitializeComponent();
        app.Run();
    }
  }
}
```

3. 프로젝트 노드를 우클릭하고 **속성**을 선택합니다.

4. **애플리케이션 탭**에서 **스타트업 객체** 드롭다운을 클릭하고 방금 추가한 Program 클래스의 허용된 이름을 선택합니다.

   > **NOTE**
   > 기본적으로 WPF 프로젝트는 수정되지 않도록 의도된 Main 이라는 시작점 함수를 정의합니다. 이 단계를 수행하면 그 시작점을 Program 클래스의 Main 함수로 변경하여, 시작하는 시점에 원하는 작업을 하도록 코드를 추가할 수 있습니다.

5. 변동 내용을 저장합니다.

커스텀 WinRT XAML 컨트롤 만들기

WPF 앱에서 커스텀 WinRT XAML 컨트롤을 호스팅하기 위해서는 앱과 같이 컴파일할 컨트롤의 소스 코드가 있어야 합니다. 일반적으로 커스텀 컨트롤은 간편한 휴대성(portability)을 위해 UWP 클래스 라이브러리 프로젝트에 정의되어 있습니다.

이 자료에서는 간단한 커스텀 컨트롤을 새 클래스 라이브러리 프로젝트로 정의할 것입니다. 방금 생성한 UWP 앱 프로젝트에서 커스텀 컨트롤을 정의할 수도 있습니다. 그러나 이러한 단계는 일반적으로 휴대성을 위해 사용자 지정 제어가 구현되는 방식이기 때문에 설명을 위한 별도의 클래스 라이브러리 프로젝트에서 이러한 작업을 수행합니다.

이미 커스텀 컨트롤을 가지고 있다면, 다음 컨트롤 대신 그것을 사용하셔도 됩니다. 그러나 이 단계에 표시된 대로 컨트롤이 포함된 프로젝트를 구성하는 단계가 남습니다.

1. **솔루션 탐색기**에서 솔루션 노드를 우클릭하고 **추가->새 프로젝트**를 선택합니다.

2. 솔루션에 **클래스 라이브러리(범용 Windows)**를 추가합니다. 대상 버전과 최소 버전은 UWP 프로젝트에서 정한 것과 같게 합니다.

3. 프로젝트 파일을 우클릭하여 **프로젝트 언로드**를 선택합니다. 다시 프로젝트 파일을 우클릭하고 **수정**을 선택합니다.

4. </Project> 이전에 다음 XML 을 추가해서 몇 가지 속성을 비활성화하고 프로젝트 파일을 저장합니다. 이 속성들은 WPF(또는 Windows Forms) 앱에서 커스텀 컨트롤을 호스팅하기 위해서는 활성화되어 있어야 합니다.

```
<PropertyGroup>
  <EnableTypeInfoReflection>false</EnableTypeInfoReflection>
<EnableXBindDiagnostics>false</EnableXBindDiagnostics> </PropertyGroup>
```

5. 프로젝트 파일을 우클릭하고 **프로젝트 리로드**를 선택합니다.

6. 기본 **Class1.cs** 파일을 삭제하고 새로 **사용자 컨트롤** 항목을 프로젝트에 추가합니다.

7. 사용자 컨트롤 XAML 파일에서 **StackPanel** 을 기본 **Grid** 의 내부에 추가합니다. 다음 예시는 **Textblock** 컨트롤을 추가하고 그것의 **Text** 속성을 **XamlIslandMessage** 필드에 바인드합니다.

```
<StackPanel Background="LightCoral">
    <TextBlock>This is a simple custom WinRT XAML control</TextBlock>
    <Rectangle Fill="Blue" Height="100" Width="100"/>
    <TextBlock Text="{x:Bind XamlIslandMessage}" FontSize="50"></TextBlock> </StackPanel>
```

8. 사용자 컨트롤의 코드 비하인드 파일에 **XamlIslandMessage** 필드를 아래와 같이 추가합니다.

```
public sealed partial class MyUserControl : UserControl
{
    public string XamlIslandMessage { get; set; }

 public MyUserControl()

    {  this.InitializeComponent();    }

}
```

9. UWP 클래스 라이브러리 프로젝트를 빌드합니다.

10. WPF 프로젝트에서 **의존성** 노드를 우클릭하고 UWP 클래스 라이브러리 프로젝트의 참조를 추가합니다.

11. 앞서 구성한 UWP 앱 프로젝트에서 **참조** 노드를 우클릭하고 UWP 클래스 라이브러리 프로젝트 참조를 추가합니다.

12. 솔루션 전체를 다시 빌드여 프로젝트가 성공적으로 빌드되도록 합니다.

WPF 앱에 커스텀 WinRT XAML 컨트롤 호스팅

1. 솔루션 탐색기에서 WPF 프로젝트를 펼쳐 **MainWindow.xaml** 파일이나 커스텀 컨트롤을 호스트하고 싶은 창을 엽니다.

2. 그 XAML 파일에서 다음 네임스페이스 선언을 **Window** 내부에 추가합니다.

```
xmlns:xaml="clr-
namespace:Microsoft.Toolkit.Wpf.UI.XamlHost;assembly=Microsoft.Toolkit.Wpf.UI.XamlHost"
```

3. 같은 파일에서 다음 컨트롤을 **Grid** 내부에 추가합니다. **InitialTypeName** 속성을 UWP 클래스 라이브러리 프로젝트의 허용된 이름으로 바꿔 줍니다.

```
<xaml:WindowsXamlHost InitialTypeName="UWPClassLibrary.MyUserControl" ChildChanged="WindowsXamlHost_ChildChanged" />
```

   코드 비하인드 파일을 열어 Window 클래스에 다음 코드를 추가합니다. 이 코드는 ChildChanged 이벤트 핸들러가 UWP 커스텀 컨트롤의 XamlIslandMessage 필드의 값을 WPF 앱의 WPFMessage 필드에 할당하도록 합니다. UWPClassLibrary.MyUserControl 을 UWP 클래스 라이브러리 프로젝트의 허용된 이름으로 수정합니다.

```
private void WindowsXamlHost_ChildChanged(object sender, EventArgs e) {
  // Hook up x:Bind source.    global::Microsoft.Toolkit.Wpf.UI.XamlHost.WindowsXamlHost windowsXamlHost =       sender as
global::Microsoft.Toolkit.Wpf.UI.XamlHost.WindowsXamlHost;    global::UWPClassLibrary.MyUserControl userControl =
      windowsXamlHost.GetUwpInternalObject() as global::UWPClassLibrary.MyUserControl;

  if (userControl != null)
  {
    userControl.XamlIslandMessage = this.WPFMessage;
  } }

public string WPFMessage
{   get   {
      return "Binding from WPF to UWP XAML";
  }
}
```

4. 앱을 빌드하고 실행하여 UWP 사용자 컨트롤이 의도한 대로 보여지는지 확인합니다.

WinUI 2.x 라이브러리의 컨트롤을 커스텀 컨트롤에 추가하기

전통적으로 WinRT XAML 컨트롤은 Windows 10 OS 의 일부로 출시되었으며 개발자들이 Windows SDK 를 통해 사용할 수 있게 되었습니다. WinUI 라이브러리는 Windows SDK 에서 WinRT XAML 컨트롤의 업데이트된 버전이 Windows SDK 릴리스에 연결되지 않은 NuGet 패키지로 배포되는 다른 접근 방식입니다. 이 라이브러리에는 Windows SDK 와 기본 UWP 플랫폼에 속하지 않는 새 컨트롤도 포함되어 있습니다. 자세한 내용은 WinUI 라이브러리 로드맵을 참조하십시오.

다음에는 WinUI 2.x 라이브러리의 WinRT XAML 컨트롤을 사용자 컨트롤에 어떻게 추가하는 방법을 보여줍니다.

> **NOTE**
> 현재 XAML Islands 는 WinUI 2.x 라이브러리의 컨트롤을 호스팅하는 것은 지원하지만, WinUI 3 라이브러리에 대한 지원은 이후 릴리스에서 제공될 예정입니다.

1. UWP 앱 프로젝트에서 Microsoft.UI.Xaml NuGet 패키지의 최신 릴리스 또는 사전 릴리스 버전을 설치합니다.

> **NOTE**
> 데스크톱 앱이 MSIX 패키지로 패키지된 경우, Microsoft.UI.Xaml 의 사전 릴리스 버전과 일반 릴리스 버전을 모두 사용할 수 있습니다. 그렇지 않다면, 사전 릴리스 버전을 설치해야 합니다.

2. 해당 프로젝트의 App.xaml 파일에서 **xaml:XamlApplication** 내부에 다음을 추가합니다.

```
<Application.Resources>
  <XamlControlsResources xmlns="using:Microsoft.UI.Xaml.Controls" /> </Application.Resources>
```

이후, 이 파일은 다음과 같이 생긴 내용을 가집니다.

```
<xaml:XamlApplication    x:Class="MyUWPApp.App"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"    xmlns:xaml="using:Microsoft.Toolkit.Win32.UI.XamlHost"
xmlns:local="using:MyUWPApp">
  <Application.Resources>
    <XamlControlsResources xmlns="using:Microsoft.UI.Xaml.Controls" />    </Application.Resources>
</xaml:XamlApplication>
```

3. UWP 클래스 라이브러리 프로젝트에서, Microsoft.UI.Xaml NuGet 패키지의 최신 버전(UWP 앱 프로젝트에서 설치한 것과 같은 버전)을 설치합니다.

4. 해당 프로젝트의 사용자 컨트롤 XAML 파일을 열어 **UserControl** 내부에 다음 네임스페이스 선언을 추가합니다.

```
xmlns:winui="using:Microsoft.UI.Xaml.Controls"
```

5. 같은 파일에 **<winui:RatingControl/>**을 **StackPanel** 내부에 추가합니다. 이것은 WinUI 라이브러리의 RatingControl 클래스의 인스턴스를 추가하게 합니다. 추가 후, **StackPanel** 부분은 다음과 같이 보여집니다.

```
<StackPanel Background="LightCoral">
  <TextBlock>This is a simple custom WinRT XAML control</TextBlock>
  <Rectangle Fill="Blue" Height="100" Width="100"/>
  <TextBlock Text="{x:Bind XamlIslandMessage}" FontSize="50"></TextBlock>
  <winui:RatingControl />
</StackPanel>
```

6. 앱을 빌드하고 실행하여 새 평가 컨트롤이 의도한 대로 보여지는지 확인합니다.

앱 패키징

원하는 경우 WPF 앱을 MSIX 패키지로 만들어서 배포할 수 있습니다. MSIX 는 Windows 의 현대적 앱 패키징 기술이며 MSI, .appx, App-V, ClickOnce 설치 기술의 결합을 기반으로 만들어졌습니다.

다음 안내는 솔루션의 모든 요소를 Visual Studio 의 Windows 애플리케이션 패키징 프로젝트 기능으로 MSIX 패키지로 묶는 방법을 보여줍니다. 다음 과정은 WPF 앱을 MSIX 패키지로 만들고 싶은 경우에만 수행하시면 됩니다.

> **NOTE**
> MSIX 패키지로 묶지 않을 것이라면, 해당 앱을 실행하는 컴퓨터는 Visual C++ 런타임이 설치되어 있어야 합니다.

1. 솔루션에 새로 Windows 애플리케이션 패키징 프로젝트를 추가합니다. 프로젝트를 생성하고, UWP 프로젝트에서 선택한 대상/최소 버전과 같도록 해당 값을 선택합니다.

2. 패키징 프로젝트에서 **애플리케이션** 노드를 우클릭하여 **참조 추가**를 선택합니다. 프로젝트 리스트에서 WPF 프로젝트를 선택하고 확인을 누릅니다.

> **NOTE**
> Microsoft 스토어에 앱을 출시하고 싶다면, UWP 프로젝트의 참조도 추가해야 합니다.

3. x86 또는 x64 와 같이 특정 플랫폼을 대상으로 하도록 솔루션을 구성합니다. Windows 애플리케이션 패키징 프로젝트를 사용하여 WPF 앱을 MSIX 패키지로 빌드하려면 이렇게 해야 합니다.

   a. **솔루션 탐색기**에서 솔루션 노드를 우클릭하고 **속성->구성 속성->구성 관리자**를 선택합니다.

   b. **활성 솔루션 플랫폼** 아래에서 x64 또는 x86 을 선택합니다.

c. WPF 프로젝트의 행, **플랫폼** 열에 **새로 만들기**를 선택합니다.

d. 새 솔루션 플랫폼 대화상자에서 x64 또는 x86 중 방금 선택한 것을 선택하고 확인을 클릭합니다.

e. 대화상자들을 닫습니다.

4. 패키징 프로젝트를 빌드하고 실행합니다. WPF 가 실행되고 UWP 컨트롤이 정한 대로 나오는 것을 확인합니다.

관련 주제

- 데스크톱 앱에서 UWP XAML 컨트롤을 호스팅하기(XAML Islands)
- XAML Islands code samples
- WindowsXamlHost

C++ Win32 앱에서 UWP XAML 호스팅 API 사용하기

11/2/2020

Windows 10 버전 1903 부터, UWP 가 아닌 데스크톱 앱(C++ Win32, WPF, Windows Forms 포함)에서 윈도 핸들(HWND)와 연관된 모든 UI 요소를 호스팅하기 위해 UWP 컨트롤 UWP XAML 호스팅 API 를 사용할 수 있게 되었습니다. 이 API 는 UWP 가 아닌 데스크톱 앱이 UWP 컨트롤을 통해서만 사용할 수 있는 최신 Windows 10 UI 기능을 사용할 수 있게 해 줍니다. 이를 테면 UWP 가 아닌 데스크톱 앱에서 Fluent Design System 을 사용하는 UWP 컨트롤을 사용하여 Windows Ink 를 지원하도록 이 API 를 사용할 수 있습니다.

UWP XAML 호스팅 API 는 개발자들이 UWP 가 아닌 데스크톱 앱에 Fluent UI 를 가져올 수 있도록 하기 위해 우리가 제공하는 더 광범위한 컨트롤 세트를 위한 기반을 제공합니다. 이 기능을 XAML Islands 라고 합니다. 이 기능에 대한 개요는 데스크톱 앱(XAML Islands)에서 UWP XAML 컨트롤 호스팅하기를 참조하십시오.

> **NOTE**
> XAML Islands 에 대하여 피드백이 있다면 Microsoft.Toolkit.Win32 리포지토리에서 이슈를 생성하고 코멘트를 남겨 주십시오. 다른 사람이 보지 않기를 원하신다면 XamlIslandsFeedback@microsoft.com 으로 보내주십시오. 당신의 통찰력과 시나리오가 저희에게 매우 중요합니다.

나의 데스크톱 앱에 UWP XAML 호스팅 API 가 좋은 선택인가?

UWP XAML API 는 데스크톱 앱에서 UWP 컨트롤을 호스팅할 수 있는 저수준 인프라를 제공합니다. 어떤 종류의 데스크톱 앱은 이 목표를 위해 더 편리한 다른 API 를 선택할 수 있습니다.

- C++ Win32 데스크톱 앱에서 UWP 컨트롤을 호스팅하려 한다면, UWP XAML 호스팅 API를 사용해야 합니다. 이 종류의 앱에 대해서는 다른 방법이 없습니다.
- WPF 및 Windows Forms 앱에서는 UWP 호스팅 API 대신 Windows 커뮤니티 툴킷에서 제공되는 XAML Island .NET 컨트롤 사용이 권장됩니다. 이러한 컨트롤들은 내부적으로 UWP XAML 호스팅 API 를 직접 사용할 때 구현해야 할 부분, 이를 테면 키보드 내비게이션이나 레이아웃 변경 등을 자동으로 처리합니다.

C++ Win32 앱에서만 UWP XAML 호스팅을 사용하는 것을 추천하므로, 이 자료는 우선적으로 C++ Win32 앱에 대하여 안내 및 예제를 보여줍니다. 단, 원하신다면 WPF 나 Windows Forms 앱에서 사용할 수도 있습니다. 이 자료에서는 Windows 커뮤니티 툴킷에서 WPF 및 Windows Forms 에 대한 host 컨트롤에 대한 관련 소스 코드를 가리키므로 UWP XAML 호스팅 API 가 이러한 컨트롤에서 어떻게 사용되는지 확인할 수 있습니다.

XAML 호스팅 API 를 사용하는 방법 배우기

XAML 호스팅 API 를 사용하는 단계별 안내 및 코드 예제는 다음 자료를 참고하십시오.

표준 UWP 컨트롤 호스팅하기

커스텀 UWP 컨트롤 호스팅하기

응용 시나리오

샘플

UWP XAML 호스팅 API 를 사용하는 방법은 앱 유형, 디자인 및 다른 요소에 따라 달라집니다. 이 API 를 사용하는 방법을 완전한 앱을 통해 보고 싶다면, 이 자료는 다음 샘플에 대하여 그러한 방법을 볼 수 있습니다.

C++ Win32

다음 샘플은 C++ Win32 앱에서 UWP XAML 호스팅 API 를 사용하는 방법을 보여줍니다.

- 단순 XAML Island 샘플. 이 샘플은 UWP 컨트롤을 (MSIX)패키지되지 않은 C++ Win32 앱에 대하여 기본적인 구현을 보여줍니다.
- XAML Island 커스텀 컨트롤 샘플. 커스텀 UWP 컨트롤을 패키지된 C++ Win32 앱에서 호스팅한 것에 대한 완전한 구현 샘플을 보여줍니다. 이와 동시에 키보드 입력 및 포커스 내비게이션과 같은 다른 객체를 다루는 것도 보여줍니다.

WPF 및 Windows Forms

UWP 호스팅 API 를 WPF 및 Windows Forms 앱에서 Windows 커뮤니티 툴킷에서 제공하는 WindowsXamlHost 컨트롤의 참조 샘플입니다. 소스 코드는 아래에 링크가 있습니다.

- 컨트롤의 WPF 버전은 여기에 있습니다. WPF 버전은 System.Windows.Interop.HwndHost 의 파생입니다.
- Windows Forms 버전은 여기에 있습니다. Windows Forms 버전은 System.Windows.Forms.Control 의 파생입니다.

> **NOTE**
> WPF 와 Windows Forms 앱에 대해서는 UWP XAML 호스팅 API 를 직접 사용하는 대신 Windows 커뮤니티 툴킷 제공 XAML Island .NET 컨트롤을 사용하는 것을 권장합니다. 위의 링크는 단순히 보여주는 목적입니다.

API 아키텍처

UWP XAML 호스팅 API 는 다음 메인 Windows 런타임 타입과 COM 인터페이스를 포함합니다.

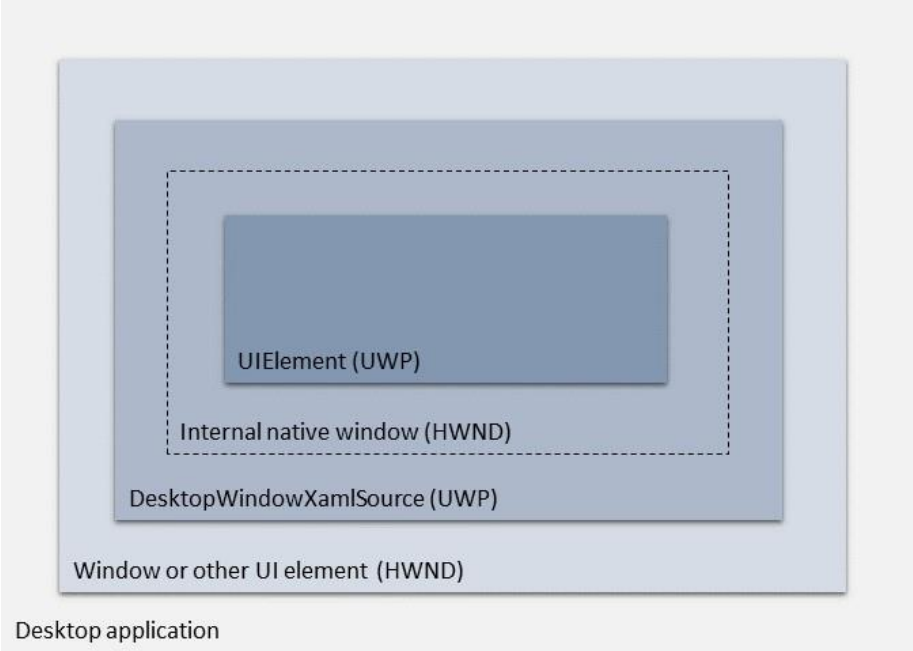
인터페이스 / 타입	설명
WindowsXamlManager	이 클래스는 UWP XAML 프레임워크를 나타냅니다. 이 클래스는 UWP XAML 프레임워크를 초기화하는 정적 메소드 InitializeForCurrentThread 를 제공합니다.
DesktopWindowXamlSource	이 클래스는 앱에 호스팅되는 UWP XAML 내용물(객체)을 나타냅니다. 이 클래스에서 가장 중요한 멤버는 Content 로, 호스팅하려는 대상인 Windows.UI.Xaml.UIElement 객체에 이 속성의 값을 할당합니다. 이 클래스에는 키보드 포커스 탐색을 XAML 아일랜드로 라우팅하기 위한 다른 멤버도 있습니다.
IDesktopWindowXamlSourceNative	이 COM 인터페이스는 AttachToWindow 메소드를 제공합니다. 이것은 앱에 있는 XAML Island 를 부모 UI 에 연결합니다. 모든 DesktopWindowXamlSource 객체는 이 인터페이스를 구현(상속)합니다. 이 인터페이스는 windows.ui.xaml.hosting.desktopwindowxamlsource.h 에 선언되어 있습니다.
IDesktopWindowXamlSourceNative2	이 COM 인터페이스는 **PreTranslateMessage** 메소드를 제공합니다. 이것은 UWP XAML 프레임워크가 특정 Windows 메시지를 바르게 처리하도록 만들어줍니다. 모든 **DesktopWindowXamlSource** 객체는 이 인터페이스를 구현합니다. 이 인터페이스는 windows.ui.xaml.hosting.desktopwindowxamlsource.h 에 선언되어 있습니다.

다음 그림은 데스크톱 앱에 호스팅된 XAML Island 의 계층 구조를 나타냅니다.

- 기본 계층은 XAML Island 를 호스팅하고자 하는 앱의 UI 원소입니다. 이 UI 에는 창 핸들(HWND)이 필요합니다. XAML Island 를 호스팅할 수 있는 UI 의 예로는 C++ Win32 앱의 window, WPF 앱의 System.Windows.Interop.HwndHost, Windows Forms 앱의 System.Windows.Forms.Control 가 있습니다.
- 다음 계층은 **DesktopWindowXamlSource** 객체입니다. 이 객체는 XAML Island 를 호스팅을 위한 인프라를 제공합니다. 당신의 코드에서는 이 객체를 생성하여 부모 UI 에 연결해야 합니다.

- **DesktopWindowXamlSource** 를 생성하면, 이 객체는 UWP 컨트롤을 호스팅하기 위해 자동으로 순수 자식 창을 생성합니다. 이 자식 window 는 코드에서 신경 쓰지 않아도 되지만, 필요한 경우 핸들(HWND)에 접근할 수 있습니다.

- 마지막으로 최상위 계층은 호스팅하려는 UWP 컨트롤 자체입니다. Windows SDK 에서 제공되거나 사용자가 정의한, Windows.UI.Xaml.UIElement 에서 파생되는 모든 UWP 객체가 이에 속합니다.



> **NOTE**
> 데스크톱 앱에서 XAML Islands 를 호스팅하는 경우 한 스레드에서 동시에 가동되는 여러 계층의 XAML 콘텐츠를 가지게 될 수 있습니다. XAML Island 의 콘텐츠의 계층의 루트 원소에 접근하고 호스팅된 곳의 컨텍스트 관련 정보를 얻으려면 XamlRoot 클래스를 사용하십시오. CoreWindow, ApplicationView, Window API 는 XAML Islands 에게 올바른 정보를 제공하지 못합니다. 더 많은 정보는 여기에서 **XAML Islands 의 창 호스트 컨텍스트**를 참조하십시오.

문제 해결

UWP 앱에서 UWP XAML 호스팅 API 를 사용할 수 없음

이슈	해결 방법
앱에서 “DesktopWindowXamlSource/WindowsXamlManager 를 활성화할 수 없습니다. 이 타입은 UWP 앱에서 사용할 수 없습니다” 와 같은 메시지와 함께 COMException 예외를 받은 경우	이 오류는 UWP 앱에서 UWP XAML 호스팅 API 를 사용하려 했다는 뜻입니다(특히 DesktopWindowXamlSource 또는 WindowsXamlManager 타입 객체를 생성하려 할 때). UWP XAML 호스팅 API 는 WPF, Windows Forms, and C++ Win32 와 같이 UWP 데스크톱 앱이 아닌 곳에서 사용하기 위한 것입니다.

WindowsXamlManager 또는 DesktopWindowXamlSource 타입 사용 에러

이슈	해결 방법
"WindowsXamlManager 및 DesktopWindow-XamlSource 는 Windows 10.0.18226.0 이상 버전에서만 지원됩니다. 애플리케이션 매니페스트나 패키지 매니페스트 파일을 확인하여 MaxTestedVersion 속성을 갱신해 주십시오”와 같은 메시지와 함께 예외를 받은 경우	이 오류는 UWP XAML 호스팅 API 에서 **WindowsXamlManager** 또는 **DesktopWindowXaml-Source** 타입을 사용하려 하지만 OS 에서 Windows 10, version 1903 이상을 사용할지 판별할 수 없다는 뜻입니다. UWP XAML 호스팅 API 는 처음에 Windows 10 이전 버전에서 프리뷰로 도입되었지만, Windows 10 버전 1903 이후로부터만 지원됩니다. 이 이슈를 해결하려면 MSIX 패키지를 만들어 패키지로 실행하거나, Microsoft.Toolkit.Win32.UI.SDK NuGet 패키지를 프로젝트에 설치하십시오.

다른 스레드에 창을 연결할 수 없음

이슈	해결 방법
"전달된 HWND 가 다른 스레드에서 만들어져서 AttachToWindow 메소드가 실패했습니다."와 같은 메시지와 함께 COMException 예외를 받은 경우	이 오류는 앱에서 **IDesktopWindowXamlSourceNative::AttachToWindow** 메소드를 호출하여 다른 스레드에서 생성된 창의 핸들(HWND)을 전달했다는 뜻입니다. 메소드를 호출한 것과 같은 스레드에서 생성된 창의 핸들만 전달할 수 있습니다.

다른 상위 창을 가진 창을 연결할 수 없음

이슈	해결 방법
"전달된 HWND 가 이전에 같은 스레드에서 이전에 AttachToWindow 에 전달된 HWND 와 다른 상위 수준 창에 포함되어 있어 AttachToWindow 메소드가 실패했습니다." 와 같은 메시지와 함께 COMException 예외를 받은 경우	이 오류는 앱에서 **IDesktopWindowXamlSourceNative::AttachToWindow** 메소드를 호출하여 동일 스레드에서 이전에 이 함수에 전달한 핸들과 다른 상위 창을 가진 창의 핸들을 전달했다는 뜻입니다. 앱의 특정 스레드에서 **AttachToWindow** 를 호출한 후에, 스레드의 다른 모든 DesktopWindowXamlSource 객체는 처음 전달된 것과 최상위 창이 같은 창에만 연결될 수 있습니다. 모든 **DesktopWindowXamlSource** 객체가 특정 스레드에서 닫힌 경우, 다음 **DesktopWindowXamlSource** 는 다시 어떤 창이든 연결될 수 있습니다. 이 이슈를 해결하려면, 같은 최상위 창 아래의 모든 **DesktopWindowXamlSource** 객체를 닫거나 이 **DesktopWindowXamlSource** 를 위한 새 스레드를 생성합니다.

Related topics

- 데스크톱 앱에서 UWP XAML 컨트롤 호스팅하기 (XAML Islands)
- C++ Win32 앱에서 표준 UWP 컨트롤 호스팅하기
- C++ Win32 앱에서 커스텀 UWP 컨트롤 호스팅하기
- 응용 시나리오 for XAML Islands in C++ Win32 apps
- XAML Islands 코드 샘플
- C++ Win32 XAML Islands 샘플

C++ Win32 앱에서 표준 WinRT XAML 컨트롤 호스팅

2/18/2021

이 자료에서는 새로 만든 C++ Win32 앱에 UWP XAML 호스팅 API 를 사용하여 표준 WinRT XAML 컨트롤, 즉 Windows SDK 에서 제공되는 컨트롤을 호스팅하는 방법을 보여줍니다. 코드는 간단한 XAML Island 예시를 기반으로 하며, 이 섹션에서는 코드에서 가장 중요한 부분을 몇 가지 다룹니다. 만약 C++ Win32 앱 프로젝트가 이미 있다면, 단계를 따라 코드 예시를 적용할 수 있습니다.

> NOTE
> 이 자료에 등장하는 시나리오는 XAML 마크업을 직접 수정하는 것에 대한 것이 아닙니다. 이 시나리오에서 컨트롤의 모습과 동작을 수정하는 것은 코드를 이용한 것에 한정됩니다. XAML 마크업을 직접 수정하는 것에 대한 안내는 C++ Win32 앱에서 커스텀 WinRT XAML 컨트롤 호스팅을 보시면 됩니다.

데스크톱 앱 프로젝트 생성

1. Windows 10 버전 1903 이상 SDK 의 Visual Studio 2019 에서 새 **Windows 데스크톱 앱** 프로젝트를 생성하고 **MyDesktopWin32App** 으로 명명하고, **C++, Windows, 데스크톱** 프로젝트로 필터를 정합니다.

2. 솔루션 탐색기에서 솔루션 노드를 우클릭하고 솔루션 재타겟을 클릭하고, 10.0.18362.0 이후 SDK 릴리스를 선택하고 확인을 누릅니다.

3. Microsoft.Windows.CppWinRT NuGet 패키지를 설치하여 프로젝트가 C++/WinRT 를 지원할 수 있도록 합니다.

   a. **솔루션 탐색기**에서 프로젝트를 우클릭하고 **NuGet 패키지 관리**를 선택합니다.

   b. 찾아보기 탭에서 Microsoft.Windows.CppWinRT 패키지를 찾아, 최신 버전으로 설치합니다.

> NOTE
> 새 프로젝트인 경우 C++/WinRT Visual Studio 확장(VSIX)를 대신 설치하고 거기에 포함된 C++/WinRT 프로젝트 템플릿을 사용해도 됩니다. 더 자세한 내용은 여기를 참조하십시오.

4. **NuGet 패키지 관리자** 창의 찾아보기 탭에서 Microsoft.Toolkit.Win32.UI.SDK NuGet 패키지를 찾아 최신 안정 버전을 설치합니다. 이 패키지는 앱에서 XAML Islands 로 작업할 수 있게 하는 몇몇 빌드 및 런타임 애셋을 제공합니다

5. 애플리케이션 매니페스트에서 **maxVersionTested** 값을 Windows 10 버전 1903 이상과 호환되도록 변경합니다.

   a. 애플리케이션 매니페스트가 현재 없는 경우 프로젝트에 XML 파일을 추가하고 **app.manifest** 로 명명합니다.

   b. 애플리케이션 매니페스트에서 **compatibility** 요소를 추가하고 다음과 같이 내부를 구성합니다. **maxVersionTested** 의 **Id** 속성을 대상 Windows 10 버전으로(10.0.18362 이상) 변경합니다.

```
<?xml version="1.0" encoding="UTF-8"?>
<assembly xmlns="urn:schemas-microsoft-com:asm.v1" manifestVersion="1.0">
  <compatibility xmlns="urn:schemas-microsoft-com:compatibility.v1">      <application>
      <!-- Windows 10 -->
      <maxversiontested Id="10.0.18362.0"/>
      <supportedOS Id="{8e0f7a12-bfb3-4fe8-b9a5-48fd50a15a9a}" />
    </application>
  </compatibility>
</assembly>
```

6. Windows 런타임 메타데이터에 대한 참조를 추가합니다.

   a. 솔루션 탐색기에서 프로젝트 참조 노드를 우클릭하고 참조 추가를 선택합니다.
   b. 페이지 하단의 찾아보기 버튼을 클릭하고 SDK 설치 경로에서 UnionMetadata 폴더를 찾습니다. 기본 경로는 **C:₩Program Files(x86)₩Windows Kits₩10₩UnionMetadata** 입니다.
   c. 그리고 대상 Windows 버전 과 같은 이름의 폴더를 열어 **Windows.winmd** 파일을 선택합니다.
   d. **확인**을 선택하고 **참조 추가** 대화상자를 종료합니다.

XAML 호스팅 API 를 사용하여 WinRT XAML 컨트롤 호스팅

XAML 호스팅 API 를 사용하는 과정은 기본적으로 다음 단계를 거칩니다.

1. 호스팅될 Windows.UI.Xaml.UIElement 객체가 앱에서 생성되기 전에 현 스레드의 UWP XAML 프레임워크를 초기화합니다. DesktopWindowXamlSource 객체를 언제 생성할지에 따라 방법이 여러 가지 있습니다.

- 앱에서 호스팅될 **Windows.UI.Xaml.UIElement** 보다 자신을 호스팅할 **DesktopWindowXaml-Source** 객체를 먼저 생성하게 하려 한다면 이 프레임워크는 **DesktopWindowXamlSource** 객체를 생성할 때 초기화됩니다. 이 시나리오에서는 프레임워크를 초기화하는 코드를 직접 추가할 필요가 없습니다.
- 앱에서 **Windows.UI.Xaml.UIElement** 객체를 더 먼저 생성하려 한다면 앱은 **Windows.UI.Xaml.UIElement** 가 생성되기 전에 WindowsXamlManager.InitializeFor-CurrentThread 정적 메소드를 호출하여 명시적으로 UWP XAML 프레임워크를 초기화해야 합니다. 일반적으로 **DesktopWindowXamlSource** 를 호스팅하는 부모 UI 요소가 생성되면 앱은 이 메서드를 호출해야 합니다.

> NOTE
> 이 메소드는 UWP XAML 프레임워크에 대한 참조를 포함하는 WindowsXamlManager 객체를 리턴합니다. 스레드에서 WindowsXamlManager 객체를 원하는 만큼 생성할 수 으나, 각 객체가 UWP XAML 프레임워크의 참조를 가지고 있기 때문에 XAML 리소스가 최종적으로 해제되도록 객체를 처리해야 합니다.

2. DesktopWindowXamlSource 객체를 생성하고 창 핸들을 가진 부모 UI 에 연결합니다. 다음 과정을 따라하시면 됩니다.

   A. **DesktopWindowXamlSource** 객체를 생성하고 **IDesktopWindowXamlSourceNative** 또는 **IDesktopWindowXamlSourceNative2** COM 인터페이스로 캐스팅합니다.

> NOTE
> 이 인터페이스들은 Windows SDK 의 헤더 파일인 **windows.ui.xaml.hosting.desktopwindowxamlsource.h** 에 선언되어 있고, %programfiles(x86)%₩Windows Kits₩10₩Include₩<빌드 번호>₩um 에 있습니다.

   B. IDesktopWindowXamlSourceNative 또는 IDesktopWindowXamlSourceNative2 의 **AttachToWindow** 메소드를 호출하여 부모 UI 의 창 핸들을 전달합니다.

   C. **DesktopWindowXamlSource** 에 포함된 자식 창의 초기 크기를 설정합니다. 기본 크기는 0 이므로 따로 설정하지 않으면 보이지 않습니다. 내부 자식 창에 접근하려면 **IDesktopWindowXamlSourceNative** 또는 **IDesktopWindowXamlSourceNative2** 인터페이스의 **WindowHandle** 속성을 사용합니다.

3. 마지막으로 호스팅하려는 **Windows.UI.Xaml.UIElement** 를 **DesktopWindowXamlSource** 객체의 Content 속성에 할당합니다.

다음 단계 및 코드 예시는 위의 과정을 구현하는 것입니다.

1. 소스 파일 폴더에서 기본 MyDesktopWin32App.cpp 파일을 엽니다. 내용을 모두 지우고 다음 using 및 include 문을 추가합니다. 표준 C++와 UWP 헤더/네임스페이스에 추가로 특정 XAML Islands 의 항목에 대한 것이 포함됩니다.

```
#include <windows.h>
#include <stdlib.h>
#include <string.h>
#include <winrt/Windows.Foundation.Collections.h>
#include <winrt/Windows.system.h>
#include <winrt/windows.ui.xaml.hosting.h>
#include <windows.ui.xaml.hosting.desktopwindowxamlsource.h>
#include <winrt/windows.ui.xaml.controls.h>
#include <winrt/Windows.ui.xaml.media.h>
using namespace winrt;
using namespace Windows::UI;
using namespace Windows::UI::Composition;
using namespace Windows::UI::Xaml::Hosting;
using namespace Windows::Foundation::Numerics;
```

2. 위의 코드에 다음을 추가합니다. 이것은 앱의 WinMain 함수를 정의합니다. 기본 창을 초기화하고 XAML 호스팅 API 를 사용하여 UWP TextBlock 컨트롤을 호스팅합니다.

```
LRESULT CALLBACK WindowProc(HWND, UINT, WPARAM, LPARAM);
HWND _hWnd;
HWND _childhWnd;
HINSTANCE _hInstance;
int CALLBACK WinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_ LPSTRlpCmdLine, _In_ int nCmdShow)
{
_hInstance = hInstance;
// The main window class name.
const wchar_t szWindowClass[] = L"Win32DesktopApp";
WNDCLASSEX windowClass = { };
windowClass.cbSize = sizeof(WNDCLASSEX);
windowClass.lpfnWndProc = WindowProc;
windowClass.hInstance = hInstance;
windowClass.lpszClassName = szWindowClass;
windowClass.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
windowClass.hIconSm = LoadIcon(windowClass.hInstance, IDI_APPLICATION);
if (RegisterClassEx(&windowClass) == NULL)
{
MessageBox(NULL, L"Windows registration failed!", L"Error", NULL);
return 0;
}
_hWnd = CreateWindow(
szWindowClass,
L"Windows c++ Win32 Desktop App",
WS_OVERLAPPEDWINDOW | WS_VISIBLE,
CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT,
NULL,
NULL,
hInstance,
NULL
);
if (_hWnd == NULL)
{
MessageBox(NULL, L"Call to CreateWindow failed!", L"Error", NULL);
return 0;
}
// Begin XAML Island section.
// The call to winrt::init_apartment initializes COM; by default, in a multithreaded apartment.
winrt::init_apartment(apartment_type::single_threaded);
// Initialize the XAML framework's core window for the current thread.
WindowsXamlManager winxamlmanager = WindowsXamlManager::InitializeForCurrentThread();
// This DesktopWindowXamlSource is the object that enables a non-UWP desktop application
// to host WinRT XAML controls in any UI element that is associated with a window handle (HWND).
DesktopWindowXamlSource desktopSource;
// Get handle to the core window.
auto interop = desktopSource.as<IDesktopWindowXamlSourceNative>();
// Parent the DesktopWindowXamlSource object to the current window.
check_hresult(interop->AttachToWindow(_hWnd));
// This HWND will be the window handler for the XAML Island: A child window that contains XAML.
HWND hWndXamlIsland = nullptr;
// Get the new child window's HWND.
interop->get_WindowHandle(&hWndXamlIsland);
// Update the XAML Island window size because initially it is 0,0.
SetWindowPos(hWndXamlIsland, 0, 200, 100, 800, 200, SWP_SHOWWINDOW);
// Create the XAML content.
Windows::UI::Xaml::Controls::StackPanel xamlContainer;
xamlContainer.Background(Windows::UI::Xaml::Media::SolidColorBrush{
Windows::UI::Colors::LightGray() });
Windows::UI::Xaml::Controls::TextBlock tb;
tb.Text(L"Hello World from Xaml Islands!");
tb.VerticalAlignment(Windows::UI::Xaml::VerticalAlignment::Center);
tb.HorizontalAlignment(Windows::UI::Xaml::HorizontalAlignment::Center);
tb.FontSize(48);
xamlContainer.Children().Append(tb);
xamlContainer.UpdateLayout();
desktopSource.Content(xamlContainer);
// End XAML Island section.
ShowWindow(_hWnd, nCmdShow);
UpdateWindow(_hWnd);
//Message loop:
MSG msg = { };
while (GetMessage(&msg, NULL, 0, 0))
{
TranslateMessage(&msg);
DispatchMessage(&msg);
}
return 0;
}
```

3. 이어서 다음 코드를 입력합니다. 창에서 창 프로시저를 정의합니다.

```
LRESULT CALLBACK WindowProc(HWND hWnd, UINT messageCode, WPARAM wParam, LPARAM lParam)
{
    PAINTSTRUCT ps;    HDC hdc;
    wchar_t greeting[] = L"Hello World in Win32!";    RECT rcClient;
    switch (messageCode)
    {       case WM_PAINT:
            if (hWnd == _hWnd)
            {
                hdc = BeginPaint(hWnd, &ps);
                TextOut(hdc, 300, 5, greeting, wcslen(greeting));
                EndPaint(hWnd, &ps);
            }          break;       case WM_DESTROY:          PostQuitMessage(0);          break;
      // Create main window       case WM_CREATE:
            _childhWnd = CreateWindowEx(0, L"ChildWClass", NULL, WS_CHILD | WS_BORDER, 0, 0, 0, 0, hWnd, NULL, _hInstance, NULL);          return 0;
      // Main window changed size       case WM_SIZE:
            // Get the dimensions of the main window's client
            // area, and enumerate the child windows. Pass the          // dimensions to the child windows during enumeration.
            GetClientRect(hWnd, &rcClient);
            MoveWindow(_childhWnd, 200, 200, 400, 500, TRUE);
            ShowWindow(_childhWnd, SW_SHOW);          return 0;
            // Process other messages.
      default:          return DefWindowProc(hWnd, messageCode, wParam, lParam);          break;
    }
    return 0;
}
```

4. 코드파일을 저장하고 앱을 빌드하고 실행합니다. UWP 텍스트블록 컨트롤이 앱 창에 나온 것을 확인합니다.

> **NOTE**
> 다음과 같은 빌드 경고를 볼 수도 있습니다.
> warning C4002: too many arguments for function-like macro invocation 'GetCurrentTime'
> manifest authoring warning 81010002: Unrecognized Element "maxversiontested" in namespace "urn:schemas-microsoft-com:compatibility.v1"
> 이 경고들은 현재 도구와 NuGet 패키지에서 알려진 이슈인데, 당장은 신경 쓸 필요 없습니다.

XAML 호스팅 API 를 사용하는 완전한 예제를 보려면 다음 코드 파일을 보십시오.

- **C++ Win32**: Xaml Islands 코드 샘플 리포지토리의 XamlBridge.cpp
- **WPF:** Windows 커뮤니티 툴킷의 WindowsXamlHostBase.cs, WindowsXamlHost.cs
- **Windows Forms:** Windows 커뮤니티 툴킷의 WindowsXamlHostBase.cs, WindowsXamlHost.cs

앱 패키지로 만들기

원한다면 앱을 MSIX 패키지로 만들어 배포할 수 있습니다. MSIX 는 Windows 의 현대적 앱 패키징 기술이며 MSI, .appx, App-V, ClickOnce 설치 기술의 결합을 기반으로 만들어졌습니다.

다음 안내는 Visual Studio 2019 의 Windows 애플리케이션 패키징 프로젝트를 이용하여 솔루션 전체를 MSIX 패키지로 만드는 방법을 보여줍니다. MSIX 패키지로 패키징하고 싶은 경우에만 보시면 됩니다.

> **NOTE**
> MSIX 로 패키징하지 않는 경우, 앱을 실행하는 컴퓨터는 Visual C++ 런타임이 설치되어 있어야 합니다.

1. Windows 애플리케이션 패키징 프로젝트를 솔루션에 추가합니다. 이때 대상 및 최소 버전으로 Windows 10 1903 버전 이상을 선택해야 합니다.

2. 패키징 프로젝트에서 애플리케이션 노드를 우클릭하고 참조 추가를 선택합니다. C++/Win32 데스크톱 앱 프로젝트를 선택하고 확인을 클릭합니다.

3. 패키징 프로젝트를 빌드하고 실행합니다. 앱이 실행될 때 WinRT XAML 컨트롤이 잘 작동하는지 확인합니다.

이후 단계

이 자료의 코드 예제에서는 C++ Win32 앱에서 표준 WinRT XAML 컨트롤을 호스팅하는 기본적인 시나리오부터 시작할 수 있습니다. 다음 섹션에서는 응용 프로그램이 지원해야 할 수 있는 추가 시나리오를 소개합니다.

커스텀 WinRT XAML 컨트롤 호스팅하기
대부분의 경우 함께 작동하는 여러 개별 컨트롤이 포함된 사용자 지정 UWP XAML 컨트롤을 호스트해야 할 수 있습니다. C++ Win32 앱에서 사용자 지정 컨트롤(직접 만든 컨트롤 또는 타사에서 제공하는 컨트롤)을 호스팅하는 프로세스는 표준 컨트롤 호스팅보다 더 복잡하며 추가 코드가 필요합니다.

완전한 가이드는 XAML Hosting API 를 사용하여 C++ Win32 앱에 커스텀 WinRT XAML 컨트롤 호스팅하기를 참조하십시오.

응용 시나리오
XAML Islands 를 호스팅하는 대부분의 데스크톱 애플리케이션은 원활한 사용자 환경을 제공하기 위해 추가 시나리오를 처리해야 합니다. 예를 들어, 데스크톱 애플리케이션은 XAML Islands 의 키보드 입력, XAML Islands 와 다른 UI 요소 간의 집중 탐색, 레이아웃 변경 등을 처리해야 할 수 있습니다.

이러한 시나리오에 대한 처리 방법 및 코드 샘플은 응용 시나리오 부분을 참조하십시오.

관련 주제

데스크톱 앱에서 UWP XAML 컨트롤 호스팅 (XAML Islands)
C++ Win32 앱에서 UWP XAML 호스팅 API 사용하기
C++ Win32 앱에서 커스텀 WinRT XAML 컨트롤 호스팅하기
C++ Win32 앱에서 XAML Islands 의 응용 시나리오
XAML Islands 코드 샘플